

Flexera

This graduation project is formally submitted to the Department of Information Technology, Faculty of Computers and Information Technology, The National Egyptian E-Learning University, Cairo, Egypt, in partial fulfillment of the academic requirements for the bachelor's degree in computer and information technology.

Supervised and Guided by

Prof.Sayed Al-Dahshan

Professor, Department of Information Technology
Faculty of Computers and Information Technology
The National Egyptian E-Learning University

AL.Omnia Atef

Assistant Lecturer, Department of Information Technology
Faculty of Computers and Information Technology
The National Egyptian E-Learning University

Team

Shahd Ahmed Sayed (2200397)

Yasmeen Waleed Gamal (2200392)

Shahinaz Samy Mohamed (2200369)

Abdelraheem Mohamed (2200633)

Razan Mohamed Kamal (2200367)

Nour Hatem Mohamed (2200882)

Mariam Nakhla Ageeb (2200962)

Gannatullah Mohamed (2100718)

Academic Year 2025-2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ
"وَمَا تَوْفِيقِي إِلَّا بِاللَّهِ عَلَيْهِ تَوَكَّلْتُ وَإِلَيْهِ أُنِيبُ"

(سورة هود - الآية 88)

Table of content:

Abstract.....	7
Acknowledgments	8
Chapter 1	9
Introduction	9
1.1 Introduction.....	10
1.2 Background of problem.....	10
1.3 Importance of problem	11
1.4 Problem statement	11
1.5 Data Source and Measures	12
1.6 Objectives 14	
1.6.1 Simplifying Appointment Booking and Saving Patients’ Time	14
1.6.2 Strengthening Doctor–Patient Communication.....	15
1.6.3 Enhancing Rehabilitation Through AI-Based Posture Detection	15
1.6.4 Tracking Patient Progress and Performance	16
1.6.5 Supporting Doctors with Remote Management Tools	16
1.7 Brief Overview of the Proposed Solution.....	17
1.7.1 Smart Booking System	17
1.7.2 Continuous Doctor–Patient Interaction Module	18
1.7.3 AI-Powered Posture Detection and Progress Tracking	18
1.7.4 Doctor Dashboard and Remote Management System	19
Chapter 2	20
Related Work	20
2.1 Introduction.....	21
2.7 Real-Time Posture Correction Systems	21
2.2 Spatio-Temporal Deep Learning Models.....	22
2.3 Hybrid Machine Learning and Fuzzy Logic.....	22
2.4 Lightweight Algorithmic Mobile Framework	23
2.5 Digital Rehabilitation Platform with Adherence Tracking	23
2.6 Graph Convolutional Network-Based Exercise Assessment.....	23

2.8 Adaptive Image Enhancement for Pose Detection	24
2.9 Comparative Analysis.....	24
2.10 Research Gap	27
Chapter 3	28
Proposed system.....	28
3.1 Approach used to solve the problem	29
3.1.1 Architectural Overview	29
3.1.2 The Proposed Solution: Core Pillars	30
3.1.3 AI Model Selection & Methodology	31
3.1.4 Dataset & Functionality Validation	32
3.2 System architecture	36
3.2.1 User Use Case Diagram	36
3.2.2 Doctor Use Case Diagram.....	37
3.2.3 Class Diagram	38
3.2.4 Sequence Diagram.....	39
1.2 Account Provisioning.....	40
1.3 Login.....	40
1.4 Forget password	41
1.5 My Appointments Feature.....	42
1.6 My Patients Feature	42
1.7.1 Doctor Adds Available Slots.....	43
1.7.2 Remove Expired Slots.....	43
2.1 Account Provisioning.....	44
2.2 Health Status	44
2.3 Booking System	45
2.4 Service Tips	45
2.6 Progress Module.....	46
2.7 Start Exercise + Camera Permission	47
3.2.5 Entity Relationship	49
3.3 Frameworks	50
Chapter 4	54

Implementation	54
4.1 Introduction	55
4.2 UI/UX Design	55
4.2.1 Design Tools and Frameworks	55
4.2.2 System Interface Screens	56
4.2.3 Interface Design Elements	61
4.2.4 Brand Identity	63
4.3 Implementation	64
4.3.1 Mobile Application	64
4.3.2 Front-End	67
4.3.3 Back-End “API Implementation”	69
4. Database Integration & Schema Design	70
4.3.4 AI	71
4.4.1 Mobile Application	73
4.4.2 Front End	74
4.4.3 Back-End	76
4.4.4 AI	82
4.5 Challenges and Resolutions	83
4.5.1 Mobile Applications	83
4.5.2 Front End	84
4.5.3 Back-End	85
4.5.4 AI	87
Chapter 5	89
Testing	89
5.1 Testing Strategies	90
5.1.1 API Testing	90
5.1.2 System Testing	95
5.1.3 System Integration Testing	143
5.1.4 GUI Testing	146
Testing Approach	147
5.1.5 Usability Testing	148

5.2 Performance Metrics.....	153
5.2.1 Accuracy.....	153
5.2.2 Scalability.....	153
5.2.3 Compatibility	154
Chapter 6	155
Result & Discussion	155
6.1 Chapter Introduction.....	156
6.2 Comparative Analysis of AI Model Performance	156
6.2.1 Selection Rationale and Hybrid Architecture	158
6.3 Real-Time Pose Detection and Kinematic Calculation	158
6.3.1 Mathematical Methodology for Kinematic Calculation	159
6.3.2 Repetition-Counting Logic (Finite State Machine)	160
6.4 Data-Driven Corrective Feedback Logic.....	161
6.4.1 Statistical Grounding.....	161
6.4.2 Directional Corrective Instructions.....	161
6.5 System Responsiveness and Architectural Efficacy	162
6.5.1 Service Decomposition.....	163
6.5.2 Empirical Performance Results	163
6.6 Environmental Validation and Occlusion Management	165
6.6.1 Deployment Requirements.....	165
6.7 Summary and Discussion	166
Chapter 7	168
Conclusion and Future Work.....	168
7.1 Summary of Contributions	169
7.2 Possible Improvements and Future Work.....	170

Abstract

Patients experience difficulties performing these exercises correctly outside clinical environments, particularly during home-based rehabilitation, which may lead to delayed recovery and increased reliance on repeated clinical visits. This challenge highlights the need for accessible and reliable rehabilitation support systems.

In this project, a smart mobile-based physiotherapy application is developed to assist patients in performing exercises correctly and consistently. The system is implemented using a **mobile application built with Flutter**, integrated with a **backend server** responsible for data management, user authentication, and communication between system components. The application analyzes patient movements during exercise sessions and provides real-time feedback to guide users toward correct posture and motion execution. Additionally, it enables tracking of rehabilitation progress over time through structured data storage and monitoring features.

The proposed system also supports personalized rehabilitation plans tailored to individual patient needs and facilitates direct communication between patients and healthcare providers. This integration enhances remote supervision and improves patient engagement throughout the rehabilitation process.

Acknowledgments

I would like to express my deepest gratitude to **Prof. Sayed Al-Dahshan** for his honorable supervision and invaluable guidance. His profound expertise and strategic insights provided the essential foundation for this project, and I am deeply grateful for his trust and the academic freedom he granted us to bring our vision for **Flex Era** to life.

I also wish to extend my sincere appreciation to **AL. Omnia Atef** for her dedicated mentorship and continuous support. Her constant presence, technical follow-ups, and constructive feedback were vital to our daily progress. I am truly thankful for the time and effort she invested in guiding the team through the practical challenges of this work.

Chapter 1

Introduction

1.1 Introduction

model of care. The core challenge remains ensuring that rehabilitation plans are followed accurately outside of scheduled clinical visits. This documentation details a Smart Mobile Physiotherapy Platform that bridges the distance between therapists and patients. Through integrated movement analysis and real time corrective monitoring, the system transforms a standard mobile device into an interactive rehabilitation tool. By focusing on precision and engagement, the platform not only tracks progress but actively guides the user through every motion, ensuring a higher standard of home-based recovery and better long-term clinical outcomes

1.2 Background of problem

Rehabilitation is a fundamental component of modern healthcare systems, aimed at optimizing individuals' physical, mental, and social functioning after illness, injury, or disability. It encompasses a wide range of services, including physiotherapy, occupational therapy, and speech therapy, all of which contribute to improving a person's ability to perform daily activities independently.

In recent decades, the global demand for rehabilitation services has increased significantly. This rise is mainly driven by factors such as population aging, the growing prevalence of chronic diseases (cardiovascular diseases, diabetes), and the increasing number of injuries resulting from accidents and other causes. According to the World Health Organization, approximately 2.4 billion people worldwide, nearly one in three could benefit from rehabilitation services at some point in their lives.

Despite this growing need, rehabilitation services remain underdeveloped and unevenly distributed across different regions. Many healthcare systems still focus more on acute care (treatment of diseases) rather than long term recovery and functional improvement, which leads to a gap in service delivery.

1.3 Importance of problem

The importance of rehabilitation lies in its critical role in enhancing quality of life and promoting independence among individuals. Rehabilitation helps patients regain lost abilities, manage pain, and prevent further complications, which ultimately reduces long-term disability.

Physiotherapy is one of the most essential rehabilitation services, as it focuses on restoring movement, strength, and physical function. Without timely and effective physiotherapy interventions, many patients may experience permanent impairments that affect their daily lives, productivity, and social participation.

Furthermore, rehabilitation has a broader impact on society and healthcare systems. It reduces hospital reductions, shortens recovery time, and lowers overall healthcare costs. Investing in rehabilitation also supports achieving global health goals such as Universal Health Coverage (UHC) and aligns sustainable development objectives aimed at improving health and well-being.

1.4 Problem statement

Despite the well-documented benefits of rehabilitation services, there is a substantial gap between the number of individuals who need these services and those who receive them. Studies indicate that more than 50% of people who require rehabilitation do not have access to adequate care, particularly in low- and middle-income countries.

Additionally, statistical analyses reveal that the utilization of physiotherapy services is significantly lower than the actual demand. This discrepancy highlights systemic issues such as lack of awareness, insufficient healthcare infrastructure, shortage of trained professionals, and limited financial resources allocated to rehabilitation services. [1] [2] [3] [4] [5]

As a result, many individuals continue to live with preventable or manageable disabilities, leading to decreased quality of life and increased socio-economic burden. Therefore, the core problem addressed in this study is the underutilization and limited accessibility of physiotherapy and rehabilitation services despite the high and growing global need.

1.5 Data Source and Measures

Researchers used data from the Global Burden of Disease Study, which is one of the most comprehensive global studies for measuring disease burden. The analysis focused on health conditions that can benefit from rehabilitation services.

Two main indicators were calculated for each country:

- Prevalence
- Years Lived with Disability (YLDs)

To better understand the evolution of global rehabilitation needs, a comparison was conducted between estimates from 2019 and 2021, along with future projections.

Aspect	2019	2021	Post-2021 / Forecast (2025-2036)
Data source	GBD 2019 (25 conditions)	GBD 2021 (27 conditions and multidimensional analysis)	Estimator GBD 2021
Number of people needing rehabilitation	2.41 billion	2.56 billion	2.64 billion (projected 2036)
YLDs	310 million	340 million	Not specified directly, but expected to rise
Most affected region	Western Pacific (610 million and 83 million YLDs)	High-SDI regions still dominant, trend toward greater global equity	LMICs (low- and middle-income countries)
Most prevalent condition	Musculoskeletal disorders (1.71 billion), mainly low back pain	Neurological disorders, sensory impairments, chronic respiratory, cardiovascular, neoplasms also rising	Stroke, cerebral palsy, Alzheimer's disease & dementia
Key drivers	Population growth, aging, high prevalence of musculoskeletal disorders	Population growth, aging, COVID-19 impact, social development	Population growth, aging, COVID-19 impact, social development

Table 1: Global Rehabilitation Needs Comparison (2019–2036)

1.6 Objectives

Main Objective

The primary goal of the **Flex Era** project is to develop an intelligent, AI-assisted physiotherapy application that enhances the quality, accessibility, and efficiency of rehabilitation services. The system aims to bridge the gap between traditional in-clinic physiotherapy and modern digital healthcare by enabling remote monitoring, real-time feedback, and continuous doctor and patient interaction.

The application is designed to improve patient outcomes by ensuring correct exercise execution, increasing adherence to treatment plans, and reducing the need for frequent physical visits. At the same time, physiotherapists with advanced tools for patient management, progress tracking, and data-driven clinical decision-making.

Specific Objectives

To achieve the main goal, the project is structured around the following specific objectives:

1.6.1 Simplifying Appointment Booking and Saving Patients' Time

The system aims to eliminate inefficiencies in traditional booking methods by providing a fully digital and user-friendly appointment scheduling platform. This includes:

- Allowing patients to browse physiotherapists based on specialization, experience, and cost.
- Enabling real-time selection of available appointment slots.
- Providing transparent consultation fees before booking.
- Supporting secure online payments within the application.
- Reducing reliance on phone calls and in-person scheduling.

Outcome:

This objective improves accessibility, reduces waiting time, and enhances patient convenience while minimizing administrative workload.

1.6.2 Strengthening Doctor–Patient Communication

The application aims to establish a continuous and effective connection between physiotherapists and patients beyond physical sessions.

This includes:

- Providing personalized rehabilitation plans tailored to each patient’s condition and recovery stage.
- Allowing patients to access exercises, instructions, and session details at any time.
- Reducing the need for repeated consultations through structured and easily accessible guidance.
- Supporting patients throughout their recovery journey both medically and psychologically.

Outcome:

This leads to better engagement, stronger trust, and improved adherence to treatment plans.

1.6.3 Enhancing Rehabilitation Through AI-Based Posture Detection

A core objective is to improve the quality of home-based physiotherapy using artificial intelligence technologies.

This includes:

- Detecting patient posture in real time while performing exercises.
- Providing instant corrective feedback to ensure proper movement.
- Identifying incorrect exercise execution may lead to injury.
- Improving the accuracy and safety of rehabilitation exercises.

Outcome:

This minimizes risks, enhances treatment effectiveness, and ensures professional-level guidance outside the clinical environment.

1.6.4 Tracking Patient Progress and Performance

The system aims to monitor and evaluate patient recovery using measurable and data-driven indicators.

This includes:

- Recording daily exercise performance and consistency.
- Tracking improvement over time using clear performance metrics
- Generating detailed progress reports for both patients and doctors.
- Supporting data-driven adjustments to treatment plans.

Outcome:

This enables better evaluation of recovery progress and more informed clinical decision-making.

1.6.5 Supporting Doctors with Remote Management Tools

The application aims to enhance the efficiency and productivity of physiotherapists through digital solutions.

This includes:

- Allowing doctors to monitor patients remotely without frequent clinic visits.
- Providing access to complete patient history and treatment records.
- Enabling the creation, modification, and management of rehabilitation plans.
- Offering a centralized dashboard for managing appointments and patient data.

Outcome:

This reduces workload, improves accuracy, and allows doctors to manage a larger number of patients effectively.

1.7 Brief Overview of the Proposed Solution

The proposed solution, **Flex Era**, is an integrated mobile and/or web-based application that leverages artificial intelligence and modern digital technologies to transform physiotherapy services into a more accessible, efficient, and intelligent system.

The application consists of four main components that work together to deliver a comprehensive rehabilitation experience:

1.7.1 Smart Booking System

Flex Era introduces a digital booking system that simplifies appointment scheduling through an intuitive and user-friendly interface.

- Patients can search for physiotherapists based on specialization and personal preferences.
- Real-time availability ensures accurate and flexible appointment selection.
- The system supports 24/7 booking without limitations.
- Integrated online payment functionality reduces additional steps and saves time.

This component eliminates traditional barriers such as long waiting times and manual scheduling processes.

1.7.2 Continuous Doctor–Patient Interaction Module

The system ensures ongoing engagement between patients and doctors beyond clinic visits.

- Patients can access their therapy plans, instructions, and session details at any time.
- All relevant information is stored in a centralized and easily accessible format.
- The need for constant direct communication is reduced while maintaining clarity and support.

This module enhances patient commitment and ensures continuity of care throughout the rehabilitation process.

1.7.3 AI-Powered Posture Detection and Progress Tracking

This is the core innovative feature of the system, utilizing artificial intelligence to enhance physiotherapy quality.

- The AI analyzes patient movements in real time using computer vision techniques.
- Provides instant corrective feedback to ensure proper exercise execution.
- Tracks patient progress daily for each exercise.
- Generates performance insights to evaluate improvement and treatment effectiveness.

This feature enables patients to perform exercises safely and effectively outside clinical environments while maintaining high-quality standards.

1.7.4 Doctor Dashboard and Remote Management System

Flex Era provides physiotherapists with a powerful dashboard for managing patients and treatment plans.

- Doctors can monitor patient progress remotely using real-time data.
- Access to complete patient profiles, including medical history and previous treatments.
- Ability to create and update rehabilitation plans efficiently.
- Centralized system for managing appointments, schedules, and patient records.
- AI-generated reports assist in evaluating patient performance.

This component improves efficiency, supports better decision-making, and reduces administrative effort.

Chapter 2

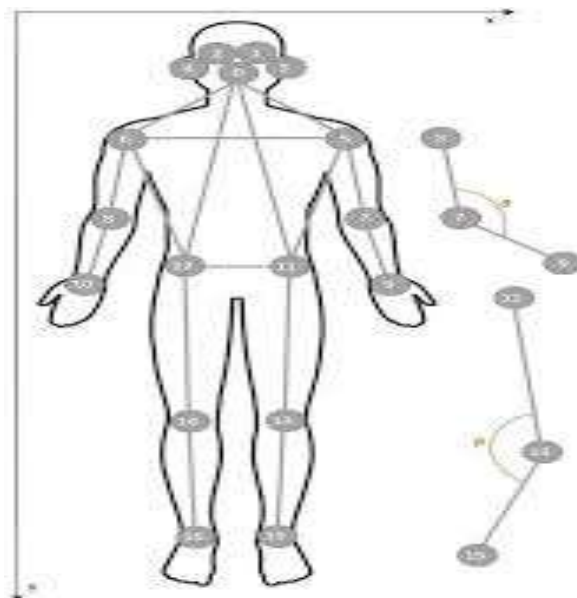
Related Work

2.1 Introduction

Recent advances in artificial intelligence (AI), computer vision (CV), and machine learning (ML) have significantly transformed automated physical rehabilitation systems. Researchers have proposed a broad spectrum of solutions, ranging from computationally intensive deep spatio-temporal models to lightweight mobile-first algorithms capable of running on consumer hardware. This chapter reviews seven recent contributions across the areas of rehabilitation exercise recognition, movement quality assessment, adaptive image processing for pose detection, and real-time corrective feedback systems, with the aim of identifying the gaps that Flexera addresses.

2.7 Real-Time Posture Correction Systems

Kotte et al. [6] developed a real-time posture monitoring system for gym exercises that pairs YOLOv7-Pose for anatomical key point detection with a rule-based angle evaluation engine to deliver immediate corrective feedback to users. The system demonstrates the feasibility of using a single RGB camera for on-the-spot form correction without specialist equipment. However, the work was evaluated in controlled gym settings and has not been validated at clinical scale, limiting confidence in its performance across the broader patient populations encountered in physiotherapy practice.

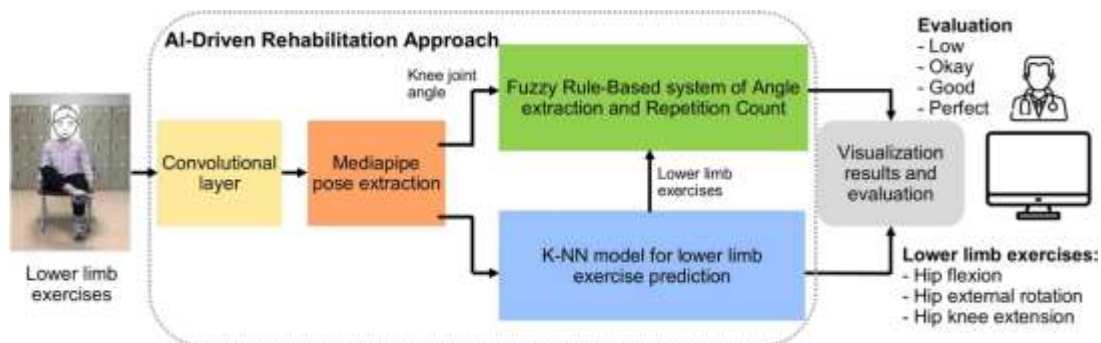


2.2 Spatio-Temporal Deep Learning Models

The STA-C3DL framework [7] integrates 3D Convolutional Neural Networks (3D CNNs) with Long Short-Term Memory (LSTM) networks to capture both the spatial configuration and temporal progression of rehabilitation movements from full video sequences. By jointly modeling motion across frames, the architecture demonstrates measurable improvements in recognition robustness against variations in execution speed and camera viewpoint compared to conventional 2D frame-wise classifiers. However, the approach demands extensive labeled training data and significant computational resources, making real-time deployment on mobile or edge devices impractical without hardware acceleration.

2.3 Hybrid Machine Learning and Fuzzy Logic

Das et al. [8] proposed a hybrid system that combines fuzzy logic with K-Nearest Neighbors (K-NN) classification for monitoring lower limb rehabilitation exercises in stroke patients. The system derives joint angles from standard 2D RGB video and applies clinician-aligned fuzzy inference rules to assess movement quality, achieving up to 97% classification accuracy. Despite these strong results, the evaluation was restricted to exercises performed by trained therapists, and the absence of 3D depth information limits its applicability to complex multi-plane movements that are common in real-world physiotherapy sessions.



2.4 Lightweight Algorithmic Mobile Framework

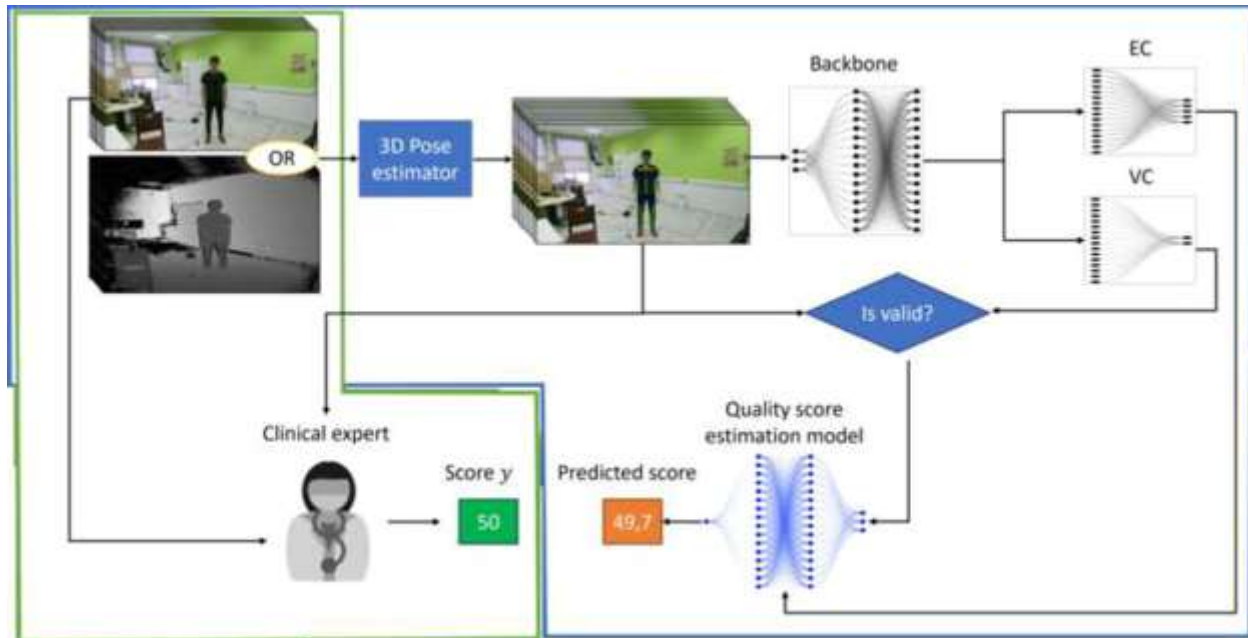
Kandylakis et al. [9] presented a mobile-health algorithmic framework that deliberately avoids deep learning in favor of dynamic programming sequence matching applied to pose-derived joint angle sequences. This design choice enables fully client-side real-time deployment on off-the-shelf smartphones without a server dependency. The trade-off is that the system is highly sensitive to the reliability of the underlying pose estimation model and requires exercise-specific calibration, reducing its generalizability to new movement types without re-tuning.

2.5 Digital Rehabilitation Platform with Adherence Tracking

Liu et al. [10] introduced a comprehensive mobile rehabilitation platform for patients recovering from spinal surgery, combining computer-vision-based exercise coaching with posture monitoring and cognitive behavioral therapy (CBT) modules. The platform meaningfully improves remote accessibility and patient engagement by integrating multiple therapeutic modalities in a single interface. Nevertheless, the primary focus on adherence tracking and general coaching means the system does not provide detailed quantitative scoring of movement quality, which is essential for clinical progress measurement in physiotherapy.

2.6 Graph Convolutional Network-Based Exercise Assessment

Kourbane et al. [11] proposed a sequential Graph Convolutional Network (GCN) architecture specifically designed to assess rehabilitation exercises from 3D skeletal input data. The model encodes the spatial relationships between body joints as graph edges, and the sequential design captures temporal motion dynamics, enabling both exercise classification and continuous quality scoring via regression. While the architecture achieves strong benchmark performance, its dependence on accurate 3D skeleton acquisition requires dedicated depth sensors such as Microsoft Kinect, which are not available in typical home environments.



2.8 Adaptive Image Enhancement for Pose Detection

Shrivastav et al. [12] addressed a frequently overlooked challenge in real-world pose estimation: the significant accuracy degradation caused by inconsistent or poor lighting conditions. Their system couples Media Pipe’s 33-keypoint pose estimator with a Convolutional Neural Network classifier that dynamically selects the most appropriate image enhancement technique per frame from a set including CLAHE, Multi-Scale Retinex with Color Restoration (MSRCR), Histogram Equalization, and Gamma Correction. Deployed as a Flask web application, the system achieves 94.2% accuracy in enhancement selection, a 15% improvement in pose detection under poor lighting, and 97.5% accuracy in repetition counting at an average latency of 55 MS per frame. The primary limitations are dependence on the InfiniteRep training dataset, which may not generalize to all clinical environments, and potential performance degradation on low-specification hardware.

2.9 Comparative Analysis

Table 2.1 provides a structured side-by-side comparison of all reviewed systems across the dimensions of input type, core methodology, target deployment environment, and key limitations.

Study	Year	Input Type	Core Method	Deployment	Limitations
YOLOv7 Posture [6]	2023	RGB Camera	Pose Key points + Rule-Based Angles	Real-Time App	No clinical-scale validation
STA-C3DL [7]	2024	RGB Video	3D CNN + LSTM	Server / High Compute	High computation cost; large data requirement
Hybrid ML + Fuzzy [8]	2025	2D Pose (RGB)	Fuzzy Logic + K-NN	Clinic / Home	No 3D depth; limited to therapist-performed exercises
M-Health Algorithm [9]	2025	2D Pose	Dynamic Programming Sequence Matching	Mobile (client-side)	Relies heavily on pose estimation quality; needs per-exercise calibration

Digital Rehab Program [10]	2025	Camera + IMU	CV + CBT Modules	Mobile Platform	Adherence-focused; lacks quantitative movement scoring
Sequential GCN [11]	2025	3D Skeleton	Spatiotemporal GCN	Research / Clinical	Requires dedicated 3D depth sensors
Adaptive Pose + Lighting [12]	2025	RGB Video (variable lighting)	Media Pipe + CNN Enhancement Selector	Web App (Flask)	Dataset-specific; performance drops on low-end hardware

Table 2.1. Comparative overview of reviewed rehabilitation systems.

2.10 Research Gap

The reviewed systems collectively demonstrate the growing maturity of AI-driven rehabilitation technology, yet several critical gaps persist. High-performing models such as STA-C3DL and Sequential GCN achieve strong accuracy but demand specialized hardware or substantial computational infrastructure that is incompatible with home-based deployment. Lightweight mobile approaches address the hardware barrier but sacrifice movement quality scoring and generalization. None of the reviewed systems simultaneously address adaptive image enhancement for variable real-world lighting, personalized rehabilitation plan management, integrated doctor-patient communication, and secure appointment scheduling within a single cohesive platform.

Furthermore, most systems are evaluated exclusively in controlled laboratory or clinical settings, and real patient populations with diverse physical conditions and home environments remain underrepresented in validation studies. The lack of integrated financial transaction management and role-based access control also limits the clinical deployment readiness of existing solutions.

Flexera is designed to bridge these gaps by adopting YOLOv8-Pose as its core pose estimation engine — an advancement over the YOLOv7-based approach of [6] integrated with a FastAPI-powered AI analysis engine and a Flutter mobile client, all within a unified, scalable platform. YOLOv8-Pose introduces a more streamlined anchor-free architecture, improved keypoint detection accuracy, and faster inference compared to YOLOv7, making it well-suited for real-time deployment in home rehabilitation settings on standard mobile hardware. The system further integrates personalized rehabilitation plans, real-time corrective feedback, doctor-patient communication, and secure payment processing — delivering a clinically practical and home-ready physiotherapy solution that no single prior work has offered in its entirety.

Chapter 3

Proposed system

3.1 Approach used to solve the problem

To address the challenges in traditional physical therapy—such as patient-doctor disconnection, incorrect exercise execution at home, and scheduling inefficiencies—the Flexera system was engineered using a multi-layered, AI-integrated approach. The solution is built upon a Client-Server Architecture that prioritizes real-time feedback, data integrity, and remote accessibility.

3.1.1 Architectural Overview

The system is divided into four distinct logical layers to ensure scalability and efficient processing:

1. **Presentation Layer (Frontend):** Developed using the Flutter framework, providing a cross-platform, responsive interface for Patients, Doctors, and Admins. It handles real-time data visualization and collects camera feed data for AI analysis.
2. **Application Logic Layer (Backend):** Powered by Node.js and Express.js, acting as the central nervous system. It manages business logic through specialized controllers:
 - **AuthController:** Secure access and OTP management.
 - **ScheduleController:** Prevents race conditions in booking.
 - **PaymobController:** Ensures secure financial transactions.
3. **Artificial Intelligence Layer (AI Engine):** A specialized Python (FastAPI) module that runs complex Deep Learning models to provide corrective feedback on patient posture.
4. **Data Persistence Layer (Database):** A MongoDB (NoSQL) database designed to store semi-structured medical logs, high-frequency exercise data, and user profiles.

3.1.2 The Proposed Solution: Core Pillars

The Flexera approach solves the problem through four fundamental pillars:

1. Optimization of Patient Time & Effort:

Unlike traditional methods requiring phone calls and physical visits, the system provides a seamless digital booking experience. Patients can browse specialized doctors, view budgets, and book appointments instantly. The integration of Paymob eliminates administrative delays, allowing for immediate confirmation.

2. Strengthening the Doctor-Patient Continuity:

The approach creates a persistent link beyond the clinic. Doctors assign digital rehabilitation plans, and patients access these instructions anytime. This continuous support loop increases treatment commitment and improves recovery outcomes.

3. AI-Powered Posture Detection & Progress Tracking:

This is the system's primary differentiator. The AI analyzes the patient's posture in real-time while performing exercises at home.

- **Correction:** It provides immediate feedback to ensure safety and prevent injury.
- **Monitoring:** It tracks daily progress, allowing clinicians to observe how a patient responds to the treatment plan remotely.

4. Remote Clinical Management for Doctors:

The system provides a comprehensive dashboard for doctors to manage a larger volume of patients efficiently. It digitizes medical histories and provides data-driven insights into patient performance, saving time and increasing diagnostic accuracy.

3.1.3 AI Model Selection & Methodology

As an initial research phase to determine the most effective solution for mobile deployment, a comparative analysis was conducted between three deep learning classification architectures to evaluate the feasibility of exercise recognition using the UCO Physical Rehabilitation Dataset. The models incorporated, all utilizing BiLSTM layers for temporal movement sequencing and Multi-Head Attention for spatial focus on relevant joint regions.

Feature	Model A (VGG16 Backbone)	Model B (MobileNetV2 Backbone)	Model C (EfficientNetB3 Backbone)
Suitability	Heavy-duty, high computational cost	Lightweight, optimized for Mobile	High-accuracy and efficient feature extraction
Training Accuracy	94.31%	97.70%	99.96%
Test Accuracy	91.40%	94.00%	98.38%
Generalization	Weaker subject generalization	Superior performance & efficiency	Excellent generalization with stable cross-validation performance
Architecture Strength	Deep but computationally expensive	Lightweight and fast inference	Balanced scalability, accuracy, and efficiency

Selection: Although Model B, based on MobileNetV2, demonstrated superior accuracy and computational efficiency, further analysis revealed that a classification-only approach could not fully satisfy the core requirements of the Flexera mobile application, particularly the need for real-time directional corrective feedback and precise object localization. To overcome this limitation, the proposed system adopted a hybrid deep learning architecture that use

EfficientNetb3. EfficientNetb3 was incorporated to further enhance feature representation and improve the extraction of high-level semantic information while preserving efficiency. The extracted pretrained weights and learned feature maps were then integrated into the YOLO model to support accurate and fast object detection. Furthermore, a one-stage detection strategy was employed within the YOLO framework, enabling simultaneous object localization and classification in a single forward pass. This approach significantly reduced inference time while maintaining high detection accuracy, making the system highly suitable for the real-time mobile rehabilitation and corrective feedback requirements of the Flexera application.

3.1.4 Dataset & Functionality Validation

The UCO Physical Rehabilitation dataset is a specialized collection of multi-view video sequences designed for the evaluation of Human Pose Estimation (HPE) methods in the context of clinical rehabilitation. It addresses the scarcity of public RGB datasets for monitoring musculoskeletal recovery.

1. Core Specifications and Scale Total Sequences: The dataset contains 2,160 video recordings. Frame Count: Approximately 1.6 million individual frames. Average Duration: Each sequence lasts roughly 30.4 seconds. Video Quality: Recorded at 1280\times 720(720p) resolution.

2. Participants and Recording Environment and Subjects: 27 healthy individuals participated (20 males and 7 females). Age Demographic: Participants ranged from 23 to 60 years old. Environment: Data collection took place in a controlled indoor laboratory with standardized lighting.



(a)

(b)

(c)

(d)



(e)

(f)

(g)

(h)

Table 1. List of exercises included in the UCO Physical Rehabilitation dataset. They are classified by position, side and upper/lower body.

#	Exercise Name	Position	Side	Upper/ Lower Body
01	Bending the knee without support while sitting	Seated	Left	Lower
02	Bending the knee with support while sitting			
03	Lift the extended leg	Supine		
04	Bending the knee with bed support			
05	Bending the knee without support while sitting	Seated	Right	
06	Bending the knee with support while sitting			
07	Lift the extended leg	Supine		
08	Bending the knee with bed support			
09	Shoulder flexion	Seated	Left	Upper
10	Horizontal weighted openings	Standing		
11	External rotation of shoulders with elastic band			
12	Circular pendulum	Seated		
13	Shoulder flexion		Right	
14	Horizontal weighted openings	Standing		
15	External rotation of shoulders with elastic band			
16	Circular pendulum			

3. Exercise Protocol the dataset includes 16 distinct activities, categorized into eight exercises performed on both the left and right sides of the body. These were specifically selected by surgeons and physiotherapists to simulate recovery from joint surgeries. Repetitions: Each activity consists of four repetitions performed at a low speed to mimic a patient with recent injuries. Movement Types: Lower Body (Exercises 1–8): Bending the knee (with/without support) and straight leg lifts in seated or supine (lying down) positions. Upper Body (Exercises 9–16): Shoulder flexions, horizontal weighted openings, elastic band rotations, and pendulum movements in seated or standing positions.

4. Multi-View Setup RGB Cameras: Five cameras recorded each exercise simultaneously from different viewpoints. Angles & Heights: Cameras were positioned at three different horizontal angles. Heights varied from 15 cm (near ground) to 180 cm

5. Monitoring: This dataset enables the development of tools for patients to perform 90% of their recovery at home while still providing doctors with accurate mobility data

6. Objective Metrics: Instead of subjective visual checks, this system calculates the precise flexion angle of limbs (e.g., knee or shoulder) to track progress over time.

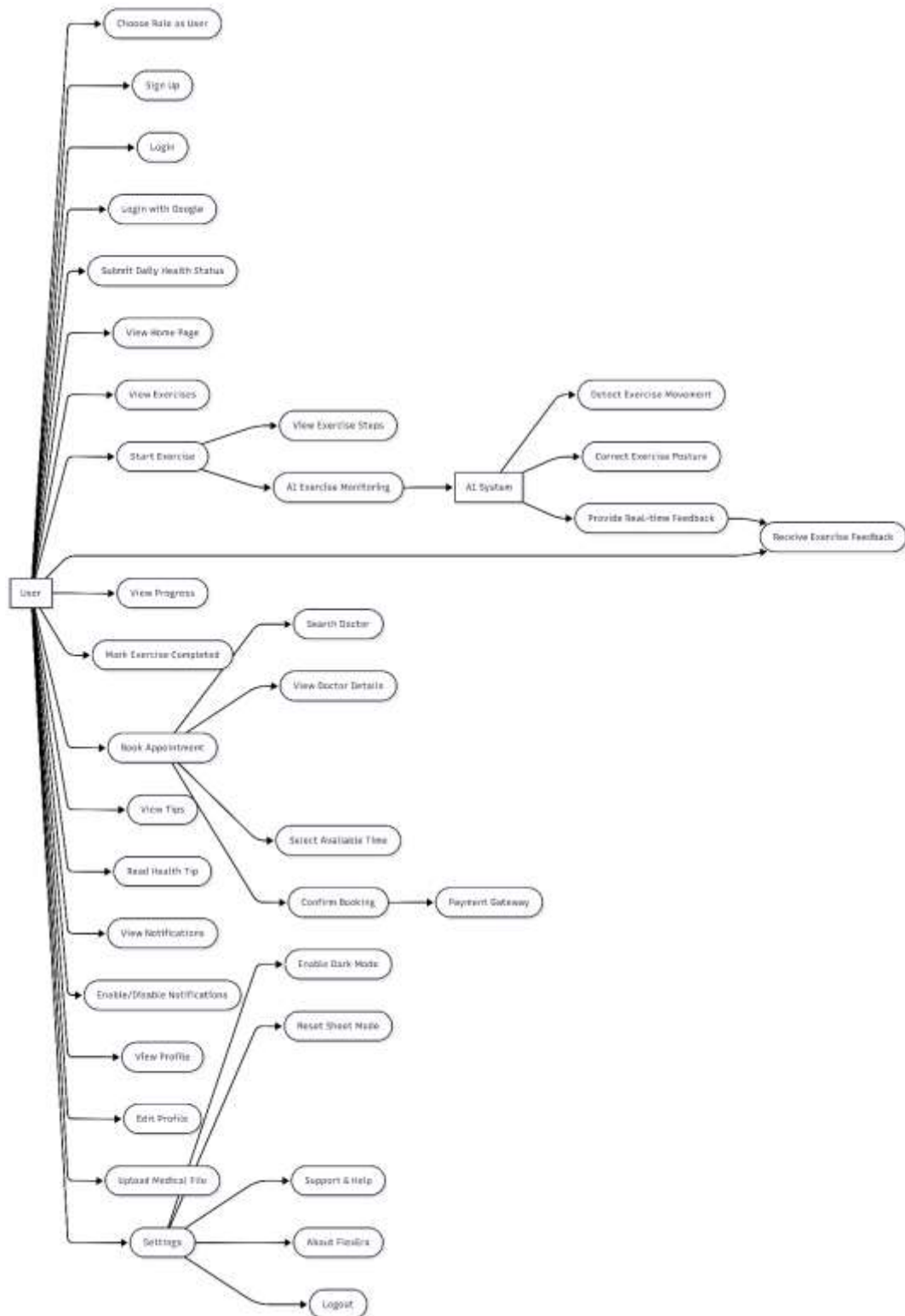
7. Limitations of the dataset

A) Controlled Environment: The current data was recorded with consistent backgrounds and lighting. Future versions may need testing in varied "real-world" home settings.

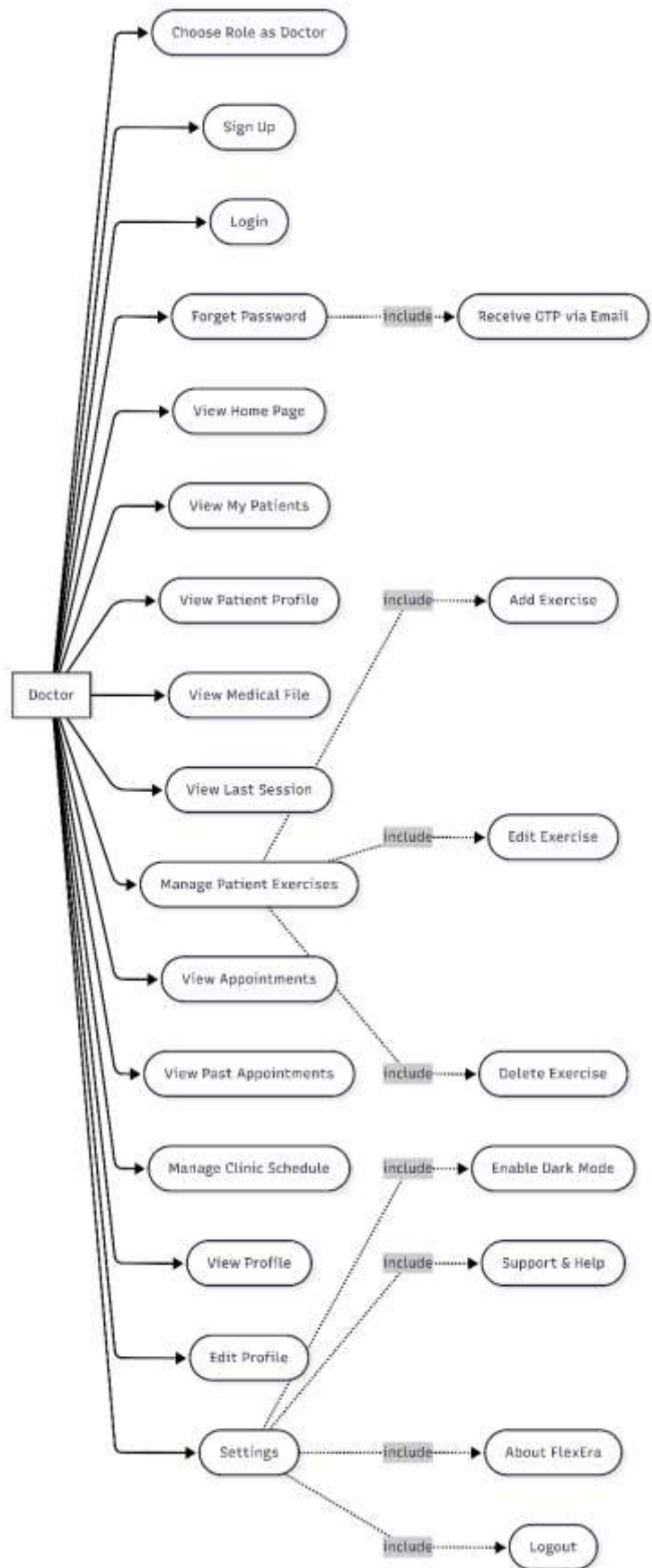
B) Occlusion Issues: Even the best models can be misled by objects in the room (like the legs of a treatment couch) or when one limb covers another.

3.2 System architecture

3.2.1 User Use Case Diagram



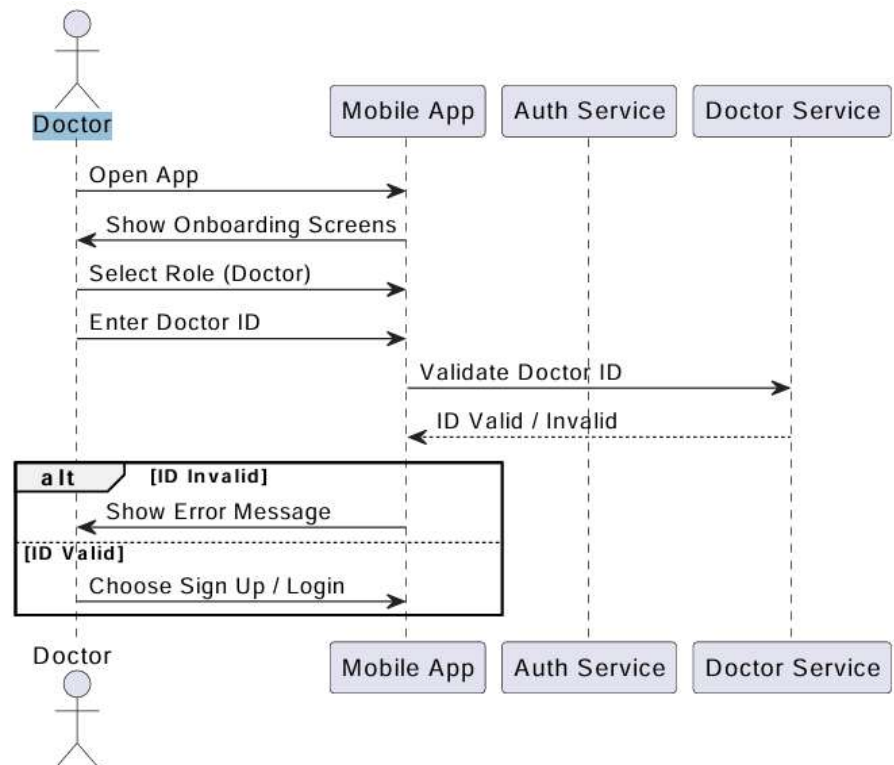
3.2.2 Doctor Use Case Diagram



3.2.4 Sequence Diagram

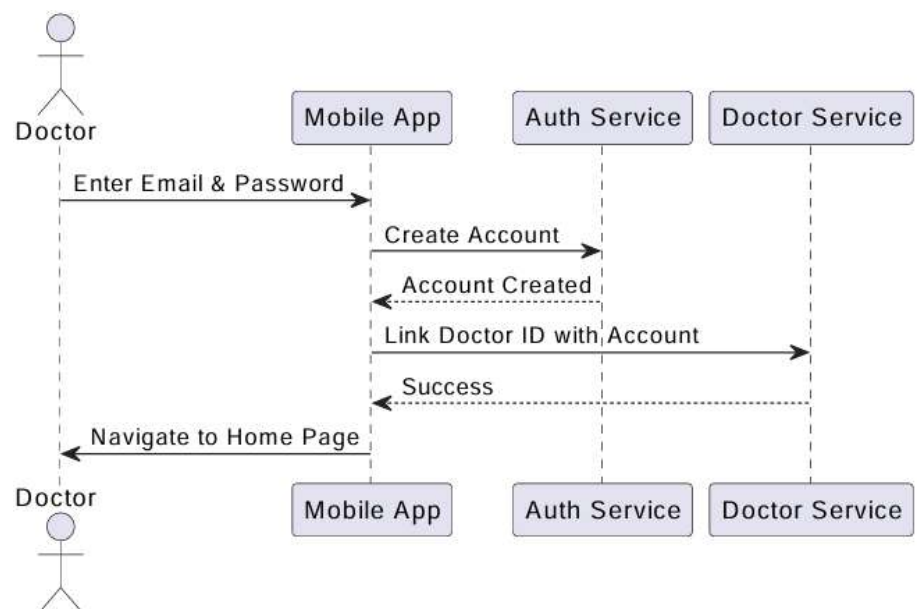
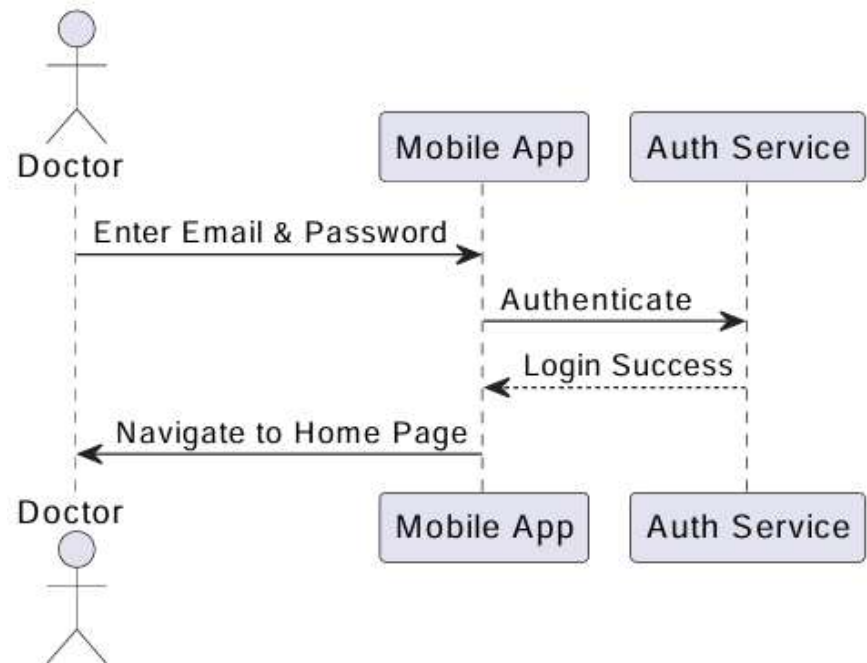
1- As a doctor

1.1 Doctor Identity Validation

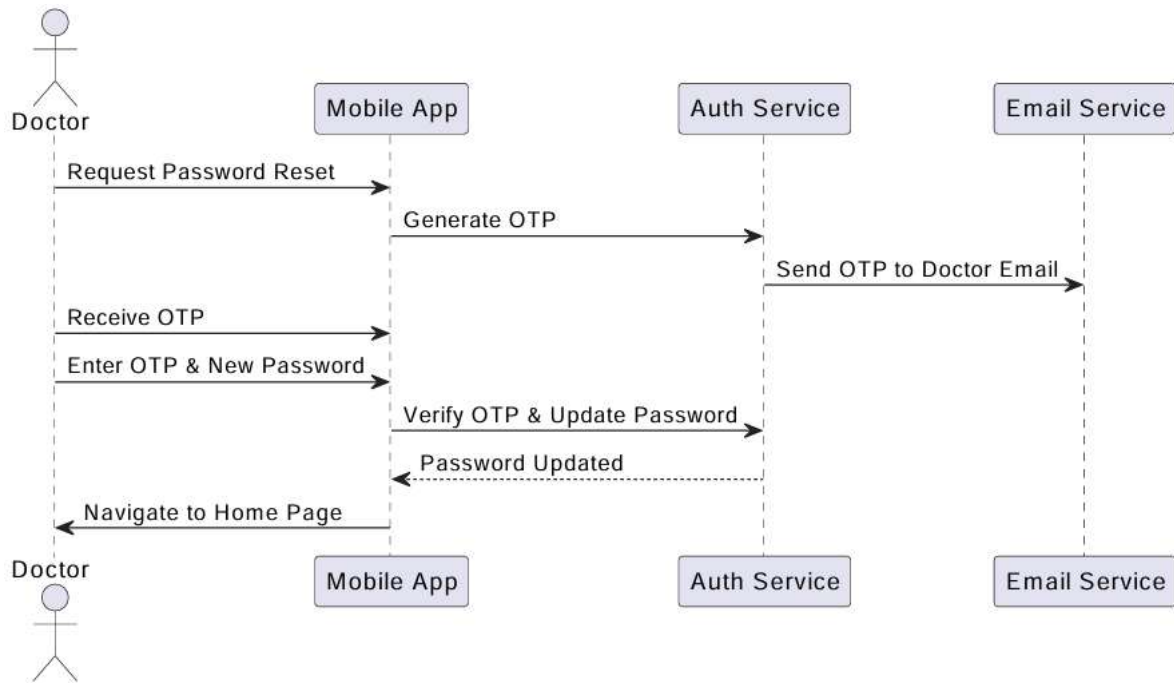


1.2 Account Provisioning

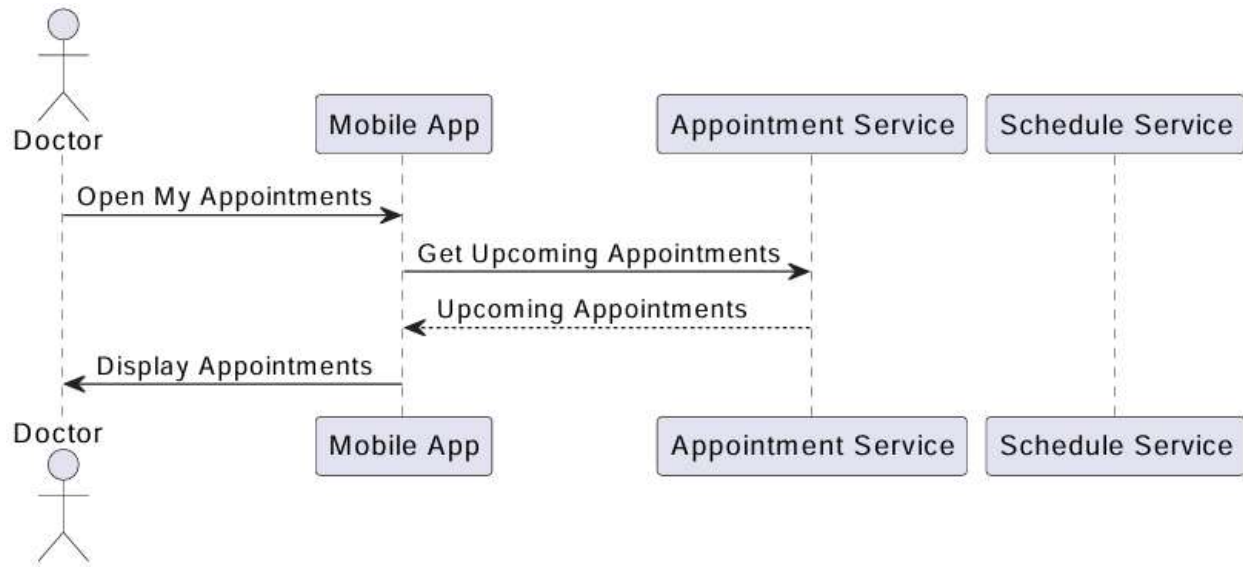
1.3 Login



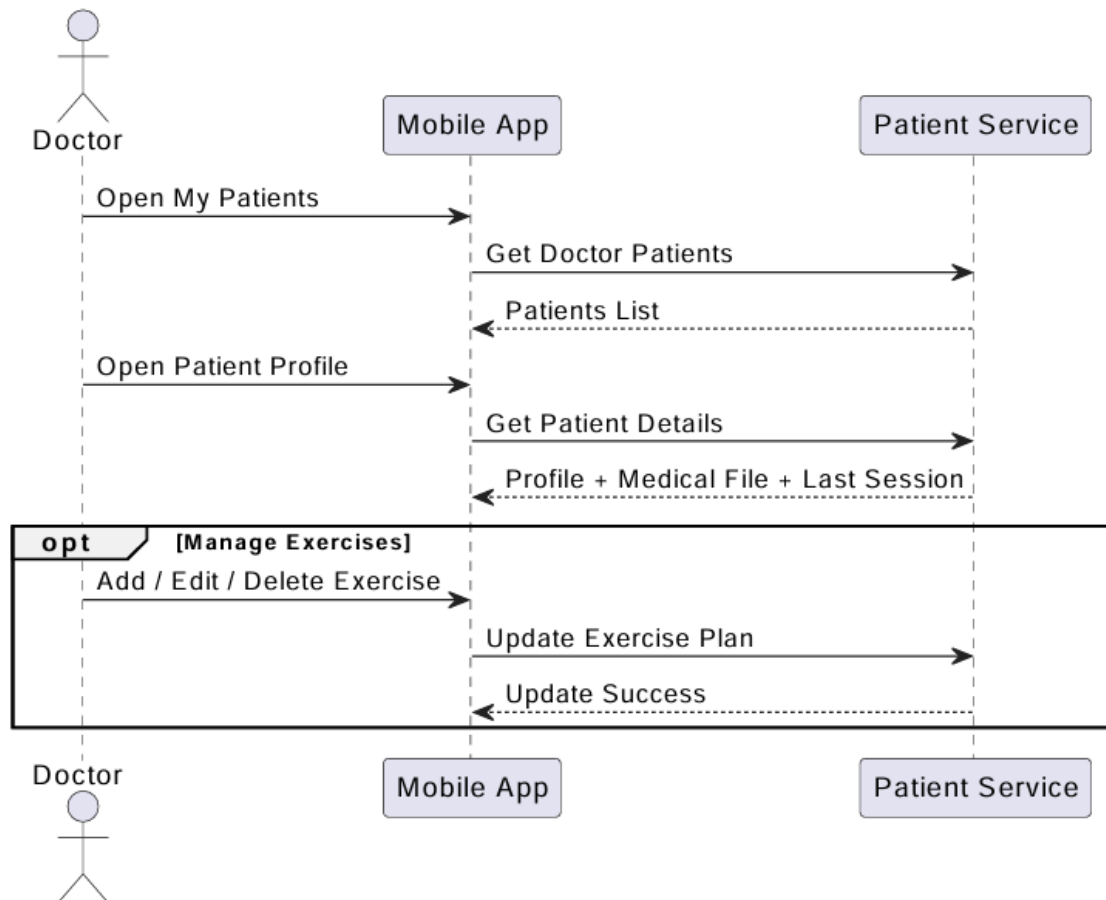
1.4 Forget password



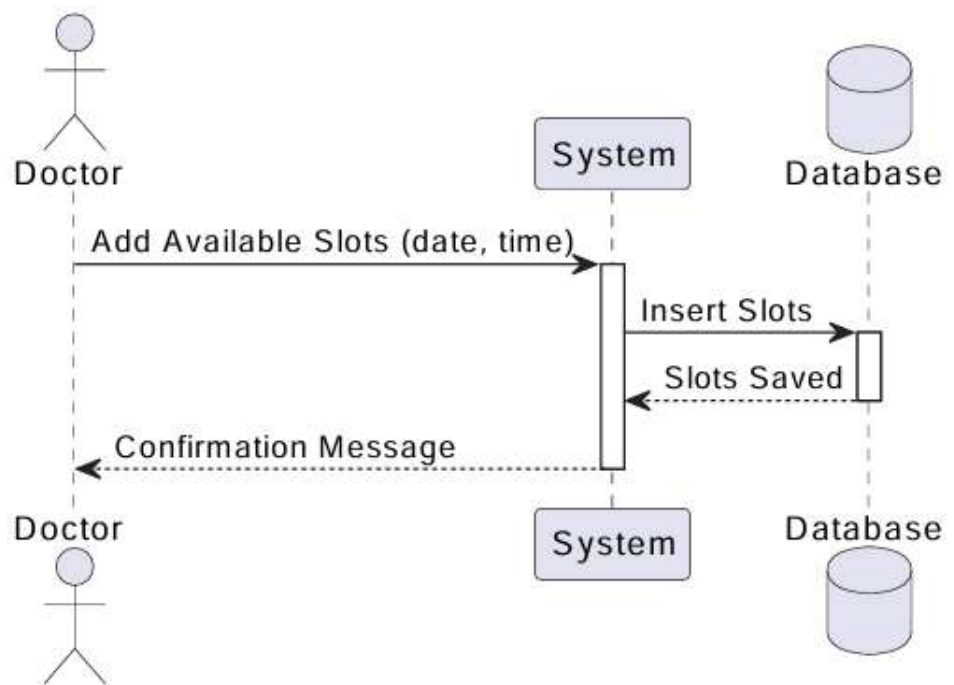
1.5 My Appointments Feature



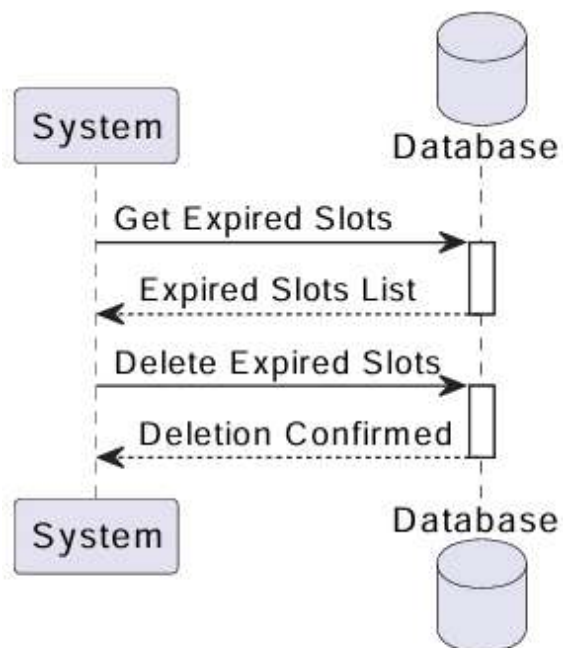
1.6 My Patients Feature



1.7.1 Doctor Adds Available Slots

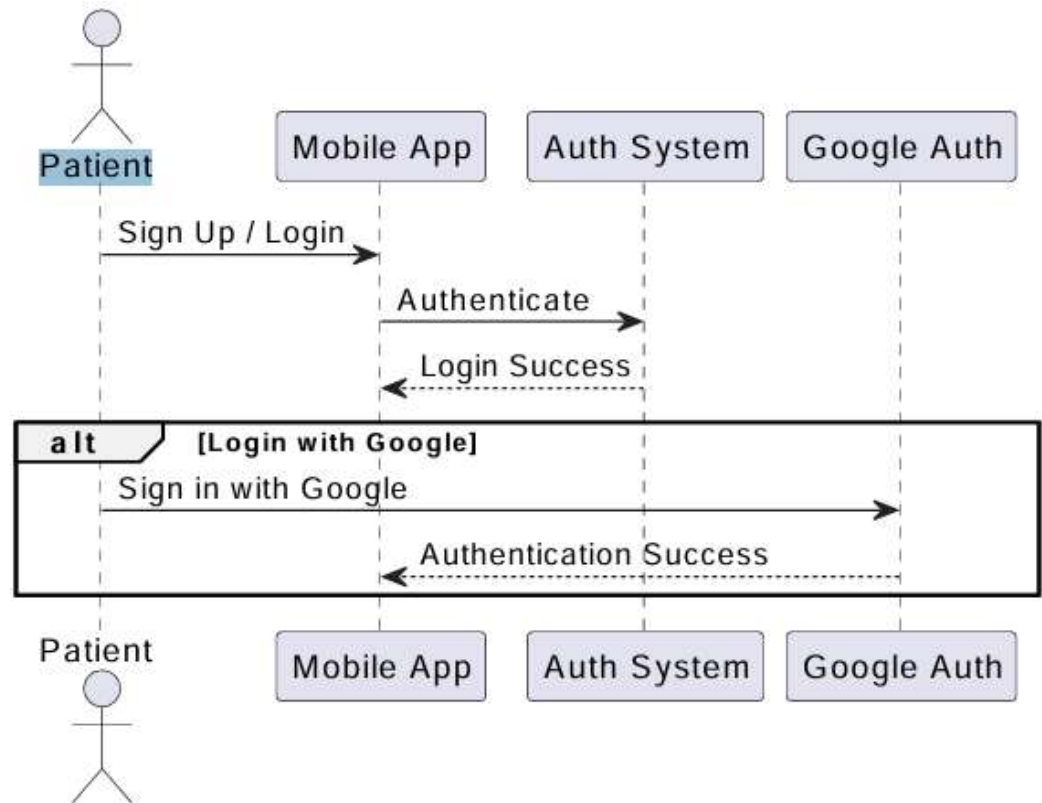


1.7.2 Remove Expired Slots



1-As a Patient

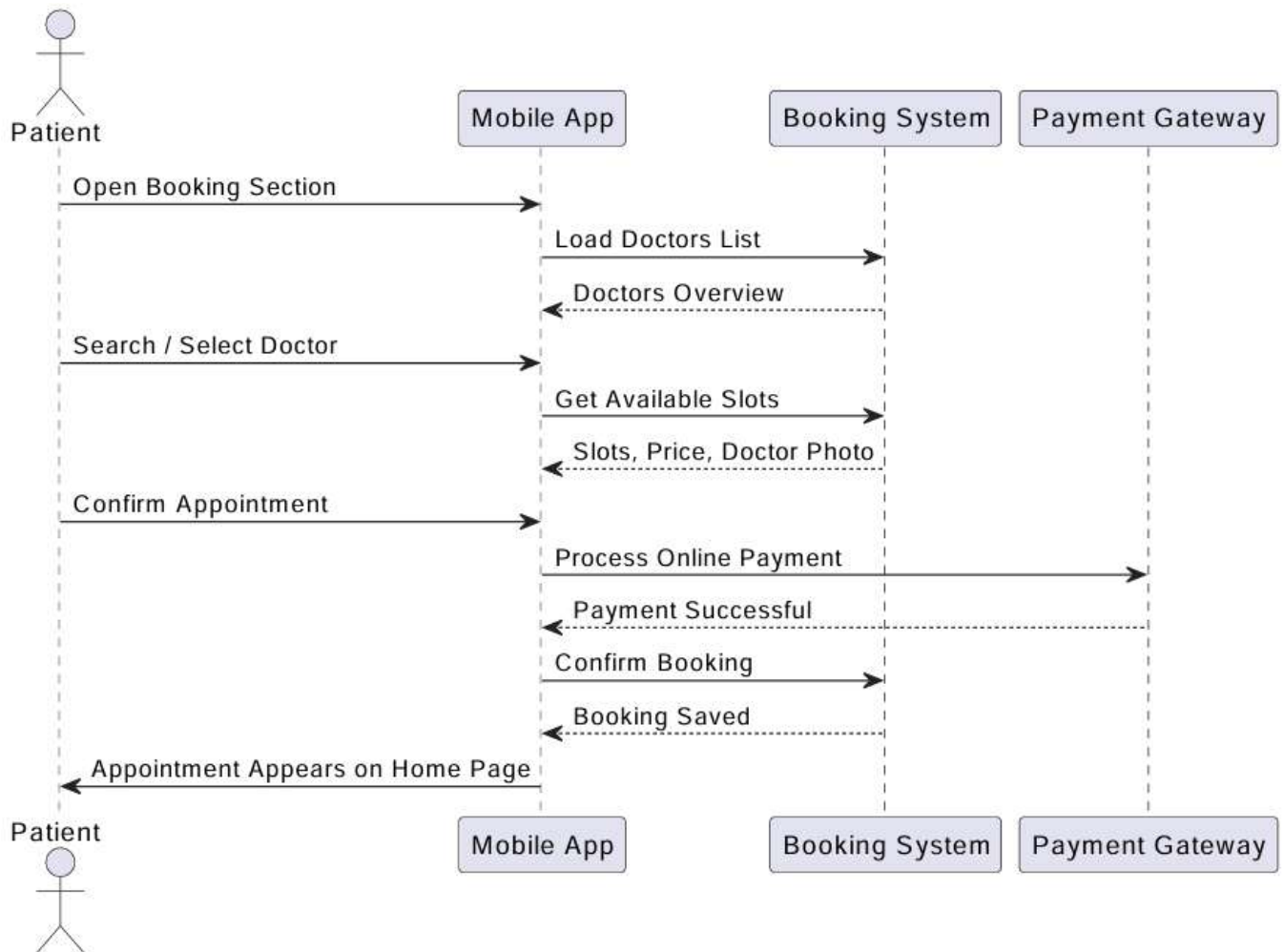
2.1 Account Provisioning



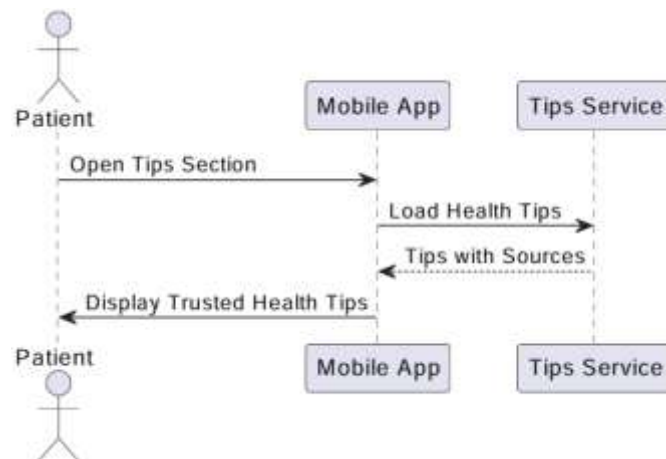
2.2 Health Status



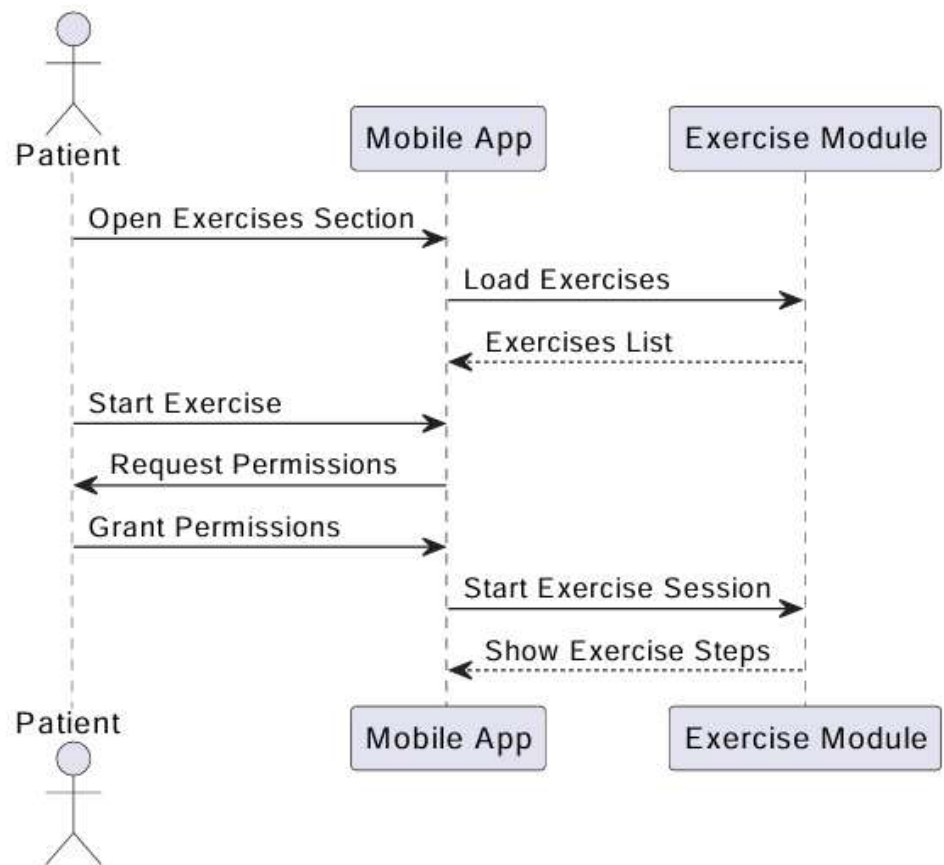
2.3 Booking System



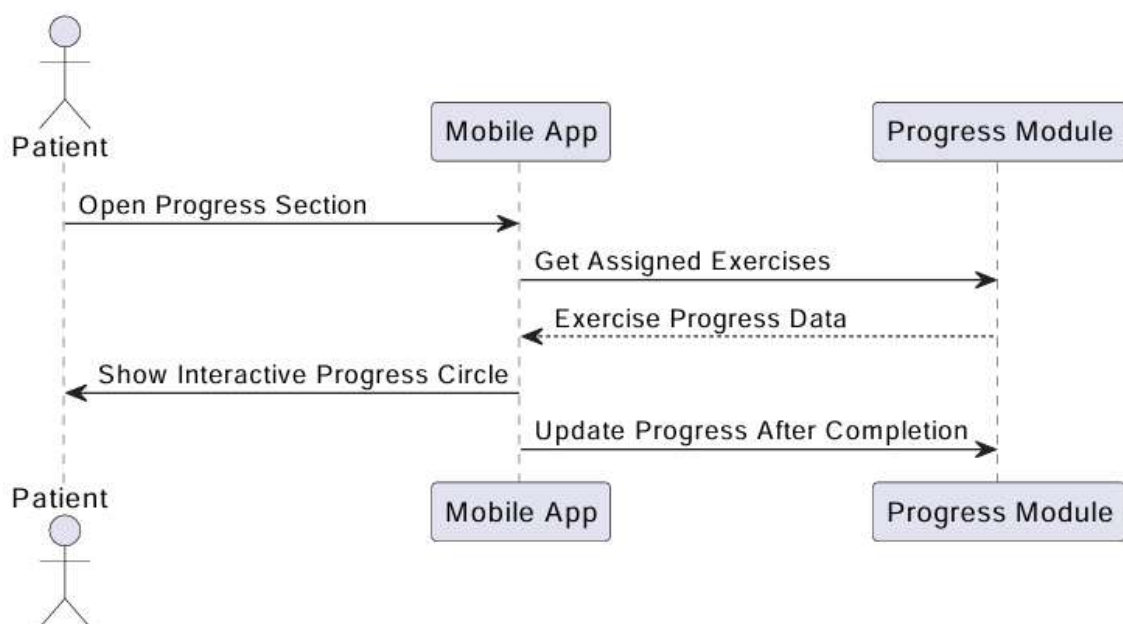
2.4 Service Tips



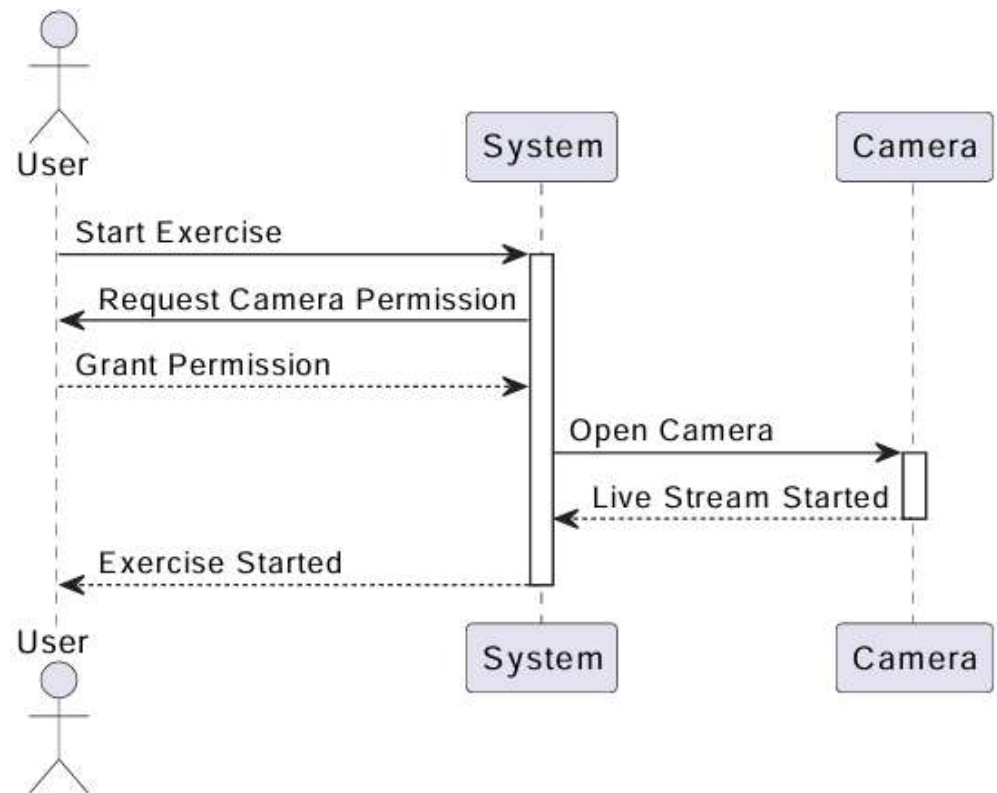
2.5 Exercises Module



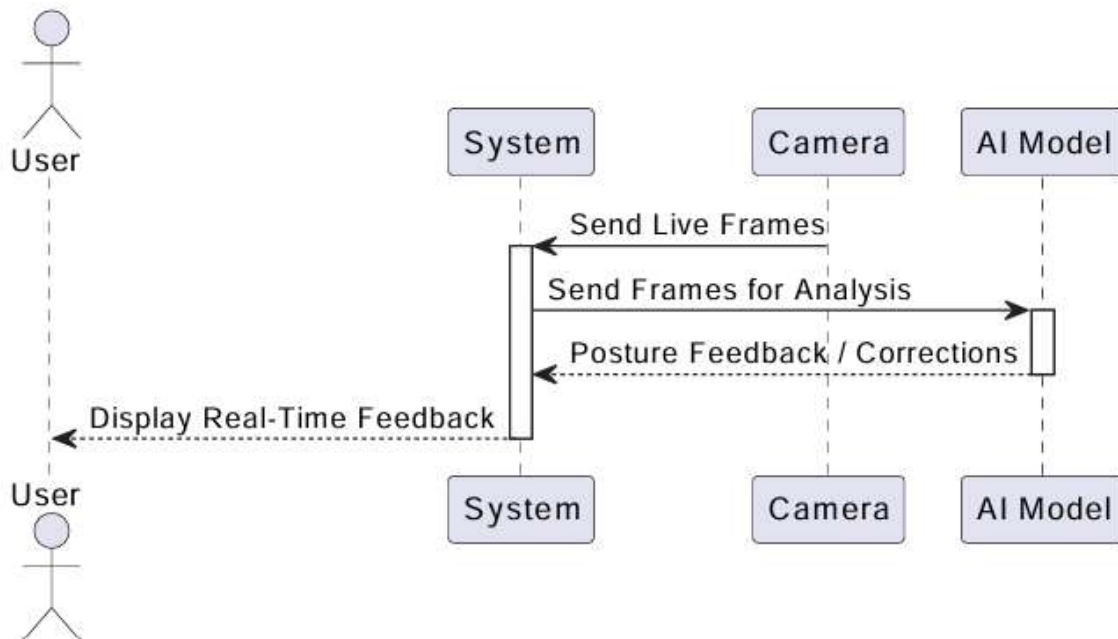
2.6 Progress Module



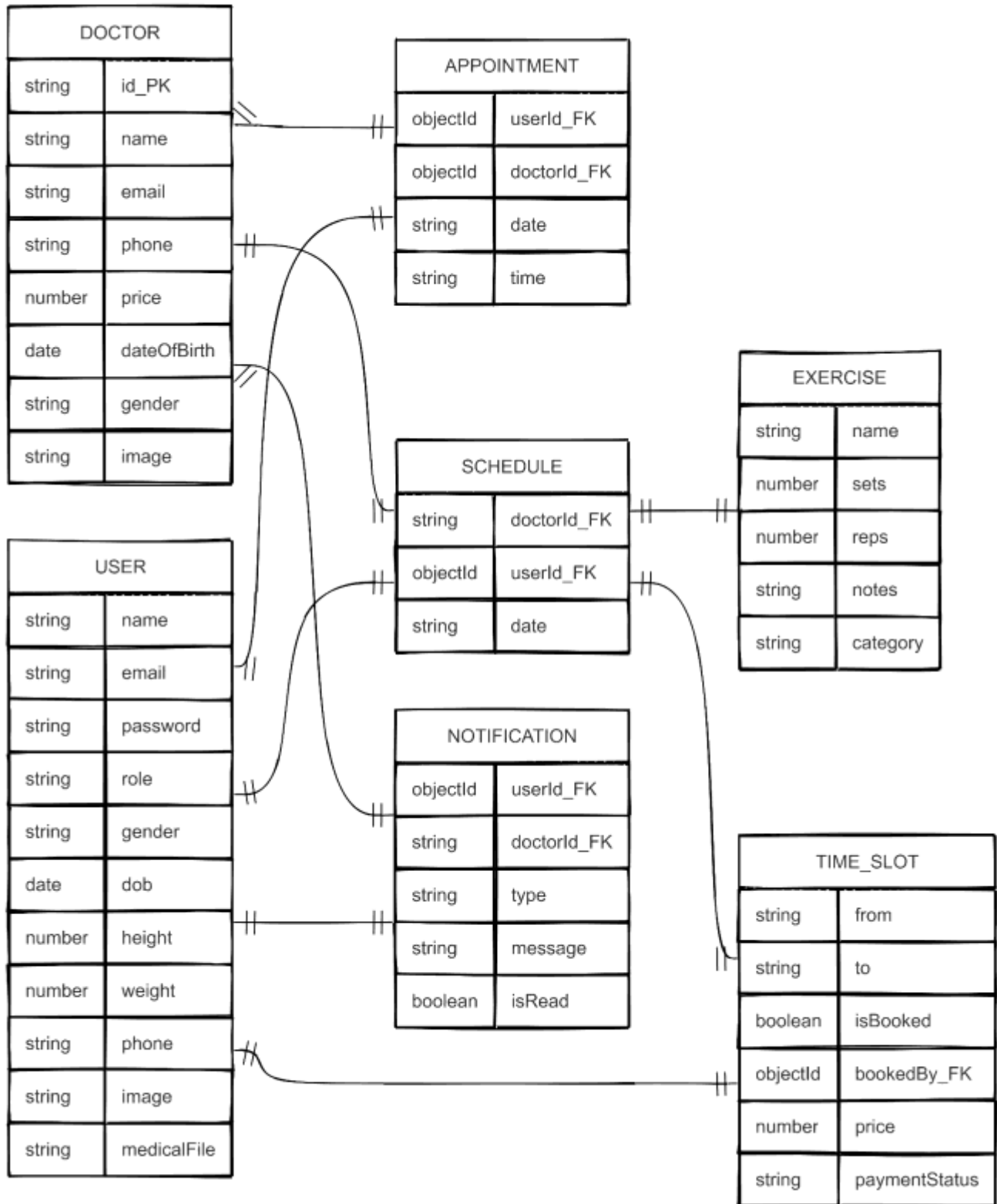
2.7 Start Exercise + Camera Permission



2.8 AI Real-Time Analysis & Feedback



3.2.5 Entity Relationship



3.3 Frameworks

This section provides detailed descriptions of the key frameworks and tools used to build *FlexEra*:

- **Express.js & Node.js[13]:** The core backend framework used to build a scalable RESTful API. It handles the application's routing logic (such as Auth, Admin, and Doctor modules), integrates essential middleware for security (JWT), and manages file uploads using Multer, ensuring a robust and organized server-side architecture.
- **MongoDB & Mongoose[14]:** A NoSQL database utilized for its flexible schema, ideal for storing dynamic medical records, user profiles, and complex appointment schedules. **Mongoose[14]** is employed as an ODM (Object Data Modeling) to enforce structure through models, enabling efficient data validation and relationship management between different collections.
- **Docker:[15]** Used to containerize the entire backend environment, including Node.js and all dependencies. This guarantees environment consistency across development, testing, and production, eliminating deployment issues and streamlining the CI/CD workflow.
- **Postman[16]:** is used for rigorous API testing, simulating HTTP requests, and debugging complex flows like **Paymob[17]** payment callbacks. For documentation.
- **Git & GitHub[18]:** Git serves as the primary version control system to track code changes, while GitHub acts as the collaborative hub for codebase hosting, issue tracking, and version management, ensuring a synchronized development cycle.
- **Third-Party Integrations:** The system is enhanced with **Paymob[17]** for secure payment processing, **Cloudinary [19]** for high-performance cloud image management, and **Firebase** for real-time push notifications, providing seamless and professional user experience.

- **Flutter & Dart Framework:** The primary cross-platform framework used to build a high-performance UI from a single codebase. It facilitates fast rendering and smooth animations, ensuring a consistent look and feel across Android and iOS devices.
- **MVVM Architectural Pattern:** A structural framework used to decouple the user interface from the business logic. By separating the **View** from the **ViewModel**, the application becomes more modular, making it easier to maintain and test.
- **Provider (State Management):** A reactive framework used to manage the application's state and data flow. It ensures that UI components are updated in real-time when data changes, without the need for manual refreshes.
- **Persistent Session Algorithm:** A logical sequence implemented to manage user authentication states. By checking locally cached JWT tokens and role flags (isDoctor), the system autonomously determines the user's entry point, ensuring secure and personalized routing.
- **RESTful Client (Dio):** A specialized networking framework configured to communicate with the Node.js API. It handles asynchronous data fetching, manages global authorization headers, and provides standardized error handling for all mobile-to-server interactions.
- **Responsive Scaling Algorithm:** A geometry-based framework (ScreenUtil) used to adapt the UI to various screen densities. It calculates relative dimensions based on a fixed design baseline, ensuring the Figma design remains precise on any mobile hardware.
- **React.js:** serves as the core frontend framework. It is a declarative, component-based JavaScript library that enables the construction of dynamic and interactive user interfaces.
- **Component-Based Architecture:** Each feature is encapsulated into reusable components (e.g., Navbar, Booking, Dashboard).

- **Virtual DOM Rendering:** Ensures efficient UI updates by minimizing direct DOM manipulation.
- **Hooks (useState, useEffect):** Provide functional state management and lifecycle control without relying on class components.
- **React Route :** is employed for navigation and routing across the application.
- **Role-Based Routing:** Implements access control for patients, doctors, and administrators.
- **Dynamic Routing:** Supports passing parameters (e.g., doctor ID, appointment slot) between pages.
- **Nested Routes:** Allows hierarchical navigation structures, improving maintainability.
- **Axios:** is the HTTP client library used for API communication.
- **Centralized API Handling:** All GET, POST, PUT, DELETE requests are routed through a single helper.
- **Global Configuration:** Manages base URLs, authorization headers, and timeout settings.
- **Error Handling:** Provides standardized responses and retry logic.
- **Integration:** Ensures seamless communication with backend services (authentication, booking, progress tracking).
- **WebSocket:** is integrated for real-time communication.
- **Bidirectional Streaming:** Enables continuous data exchange between frontend and backend.
- **CSS Frameworks & Material Design:**
 - The frontend design leverages CSS3 and Material Design principles.
 - **Responsive Layouts:** Achieved using Flexbox and Grid systems.
 - **Consistency:** Material Design provides a unified visual language across components.
 - **Accessibility:** Compliance with WCAG 2.1 ensures usability for diverse users, including those with disabilities.
- **Development & Testing Tools**
 - **Visual Studio Code (VS Code):** Primary IDE for coding, debugging, and extensions.
 - **Git & GitHub:** Version control and collaborative development. GitHub also manages issues, pull requests, and CI/CD pipelines.

- **Postman:** Used for API testing, simulating HTTP requests, and validating backend responses.
- **Browser DevTools:** Performance profiling, network inspection, and UI debugging.
- **Deployment & Build Tools**
- **Vite:** Modern build tool used for fast bundling and hot module replacement.
- **Vercel:** Deployment platform ensuring scalability and global distribution.
- **Configuration Files:** vite.config.js and vercel.json streamline build optimization and deployment workflows.

Chapter 4

Implementation

4.1 Introduction

This section presents the technologies, tools, and programming languages used in the development of the system. It highlights the key software and frameworks that support the design and implementation process, ensuring efficient development, good performance, and a user-friendly experience across the application.

4.2 UI/UX Design

The UI design in Flexera aims to provide a simple and user-friendly interface that enhances usability, supports clear navigation, and ensures consistency across all screens for smoother experience.

Therefore, the design was created to achieve key goals that improve interaction and accessibility.

- Provide a simple and user-friendly interface.
- Enhance usability and user interaction with the system.
- Maintain consistency across all screens.
- Ensure easy and clear navigation.
- Deliver a smooth user experience for all users regardless of technical background.

4.2.1 Design *Tools and Frameworks*

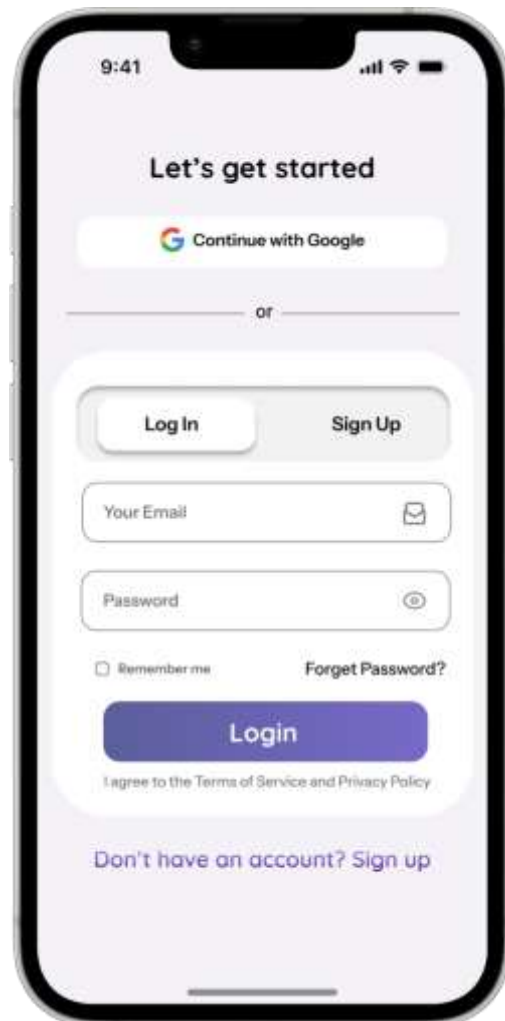
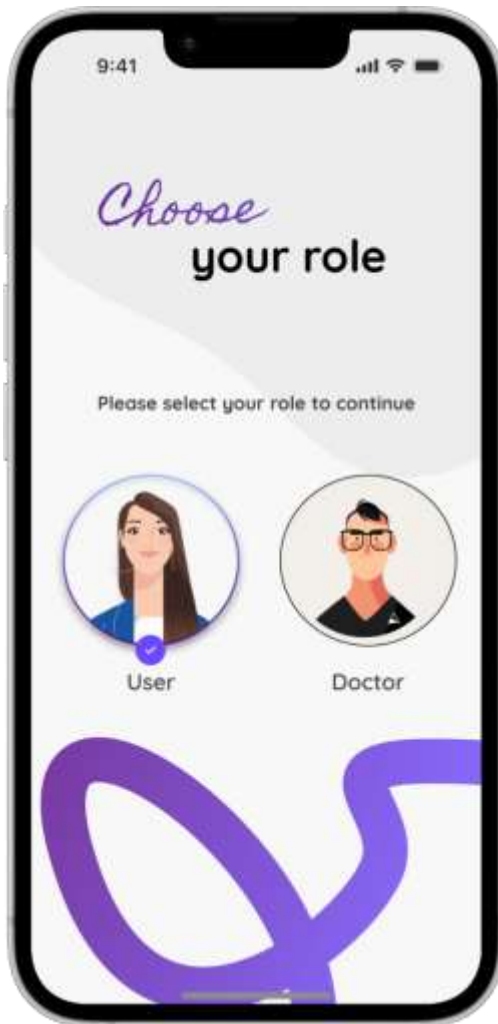
The interface design in Flexera was created using Figma and Adobe XD to develop and visualize the system's screens before implementation. These tools were used to build structured layouts and interactive prototypes that helped refine the design and improve usability.

For development, React was used to build the web application due to its flexibility and component-based structure, while Flutter was used for mobile development to ensure consistent and responsive experience across platforms.

The overall design approach focused on simplicity, consistency, and delivering a user-friendly interface across all versions of the system.

4.2.2 System Interface Screens

Login Screen: Allows users to securely access the system using their credentials

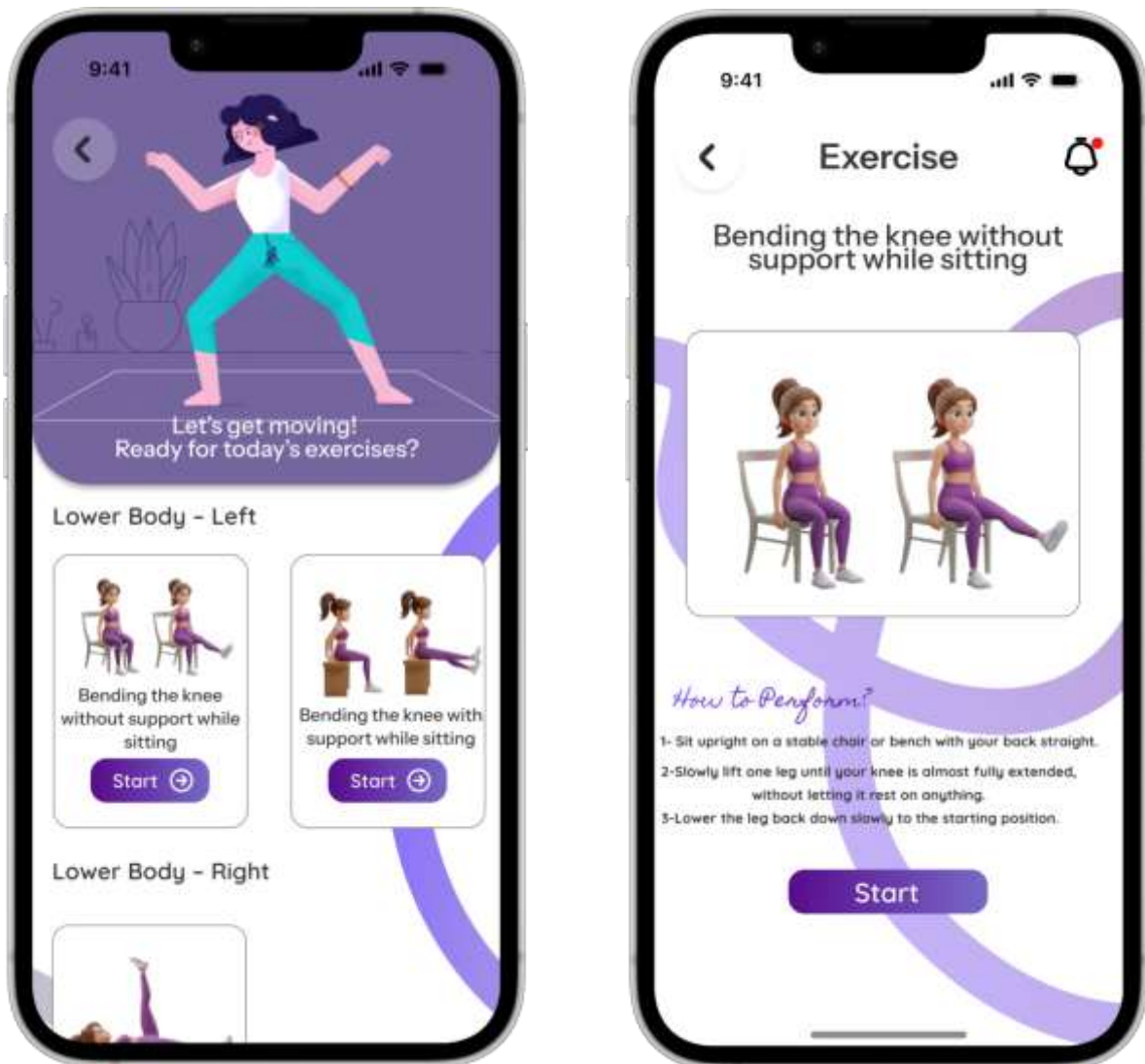


Home screen: Provides users with access to the main features of the application.



Features Screen: This screen provides access to the main functionalities of the system, allowing users to easily navigate between different features

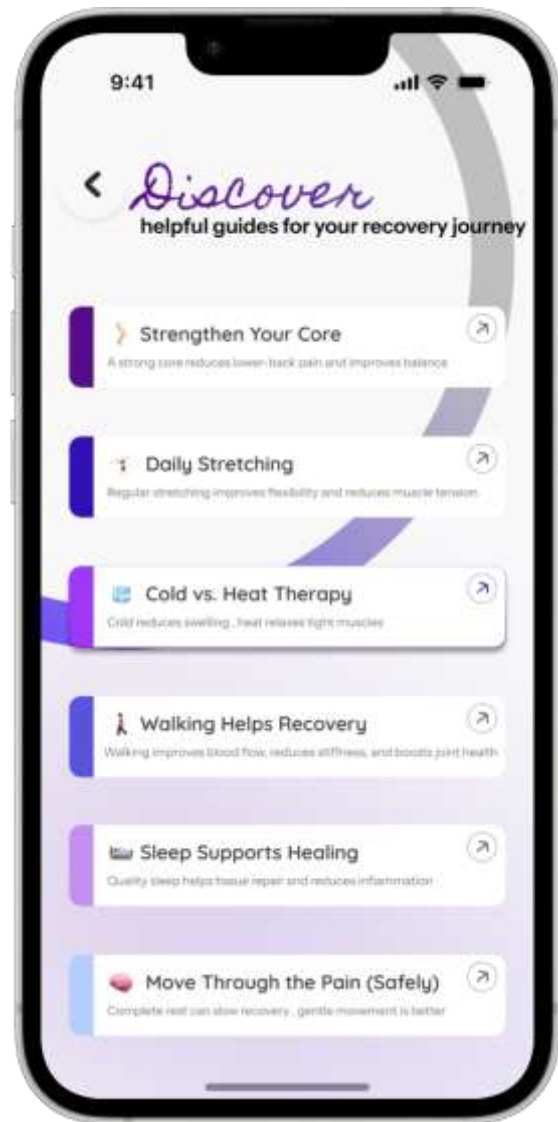
- **Exercises:** Allows users to view and perform rehabilitation exercises.



- **Appointments:** Enables users to book and manage their sessions



- **Progress & Tips:** Provides useful health tips while tracking user progress and improvement over time



- **Dashboard Screen:** Displays booked users, appointment schedules, and relevant information about doctors.



4.2.3 Interface Design Elements

This section describes the main visual elements used in the user interface, focusing on how we create a clear and consistent design.

- **Colors:** A purple color palette was used as the main visual theme, specifically the shades **#786AC8** and **#5B5F9C**, as they provide a calm and visually comfortable appearance that aligns with the nature of physiotherapy. These tones help create a balanced and modern interface while avoiding a strict clinical look

In addition, **#FFFFFF** (white) was used as a base color to enhance clarity, improve readability, and maintain a clean and simple layout



Furthermore, a Dark Mode was implemented to reduce eye strain and improve usability, especially for users with visual sensitivity and older adults.



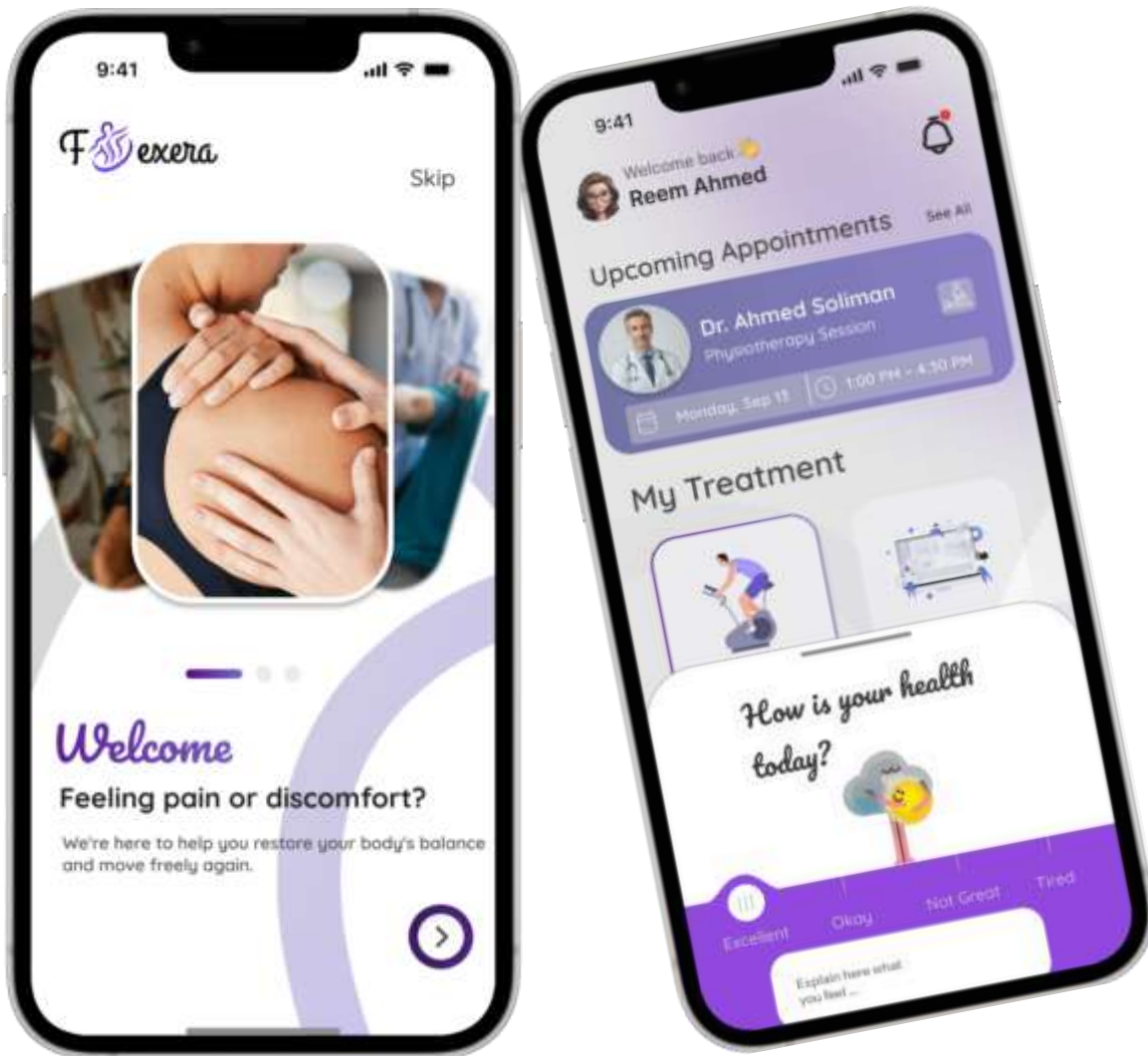
- Typography:**
 The **Quicksand** font was used as the main typeface across the application to ensure clarity and readability.
 In addition, the **Pacifico** font was specifically used in the onboarding section to create a more engaging and welcoming first impression.
- Logo Design:** The Flexera logo features the letter “L” designed in the shape of a spinal column, representing physiotherapy and rehabilitation. This design reflects the application’s focus on improving movement and spinal health while maintaining a simple and clear visual identity.



- ***Layout Consistency:*** was maintained across all screens to ensure a uniform structure and improve user experience

4.2.4 Brand Identity

The identity of Flexera is designed to represent a professional yet approachable healthcare application centered on physiotherapy and rehabilitation. It combines a minimal visual style with meaningful design elements to create a sense of trust, clarity, and ease of use. Overall branding ensures a cohesive and recognizable experience across all parts of the system.



4.3 Implementation

4.3.1 Mobile Application

1. Technologies, Tools, and Programming Languages

The mobile application was developed using a modern technology stack to ensure cross-platform compatibility, high performance, and scalability.

- **Flutter[21] SDK Implementation:** The application was built using the Flutter SDK to leverage its cross-platform capabilities, allowing for a single codebase to serve both Android and iOS users. The implementation focused on utilizing Flutter's high-performance rendering engines, **Impeller** and **Skia**, to ensure that complex physiotherapy exercise animations and real-time UI updates remain fluid and responsive.
- **Dart Programming Language:** Dart was employed as the core programming language, taking advantage of its strong typing and sound

null safety to reduce runtime errors. The development process utilized Dart's **Just-In-Time (JIT)** compilation for rapid UI iteration and **Ahead-Of-Time (AOT)** compilation for the final release to ensure maximum execution speed on mobile hardware.

- **Provider[22] Pattern:** Employed as the primary state management solution. It facilitates the separation of business logic from the UI, ensuring efficient rebuilding of widgets only when necessary data changes.
- **Dio [23]Package:** Used for handling HTTP requests. It provides advanced features such as interceptors, global configuration, and robust error handling compared to the standard HTTP library.
- **SharedPreferences[24]:** Implemented for persistent local data storage (via CacheHelper), used to manage user sessions (tokens), application settings (Theme mode), and onboarding status.
- **Firebase Core & Messaging:** Integrated to handle backend cloud services and cloud messaging (FCM) for real-time push notifications.
- **Flutter ScreenUtil:** Utilized to ensure a responsive UI design across different screen sizes and densities by adapting text and element sizes based on a standard design size (405x923).
- **FastAPI Integration (Mobile Perspective):** The application communicates with AI-driven backend services hosted via FastAPI to perform real-time motion detection and exercise validation through RESTful API endpoints.

2. Software Architectural Techniques & Principles

To ensure the code is maintainable and professional, several Object-Oriented Programming (OOP) and SOLID principles were applied:

- **Encapsulation:** This principle is applied in the DioHelper and CacheHelper modules to centralize network configurations and local storage logic. By concealing these internal implementation details, the application protects its global state and ensures that sensitive configurations, such as API headers and persistent tokens, are not exposed to unintended modifications.
- **Abstraction:** Through the use of the **Repository Pattern**, specifically in AccountRepository, the system creates a necessary layer between business logic and data sources. This allows ViewModels to execute data operations

without handling the complexities of API communication or data formatting, significantly simplifying future updates to the backend services.

- **Single Responsibility[25] Principle (SRP):** The architecture maintains a clear division of labor by assigning specific functionalities to dedicated classes, such as LoginViewModel for authentication processes. This ensures that each module has only one reason to change, which enhances code readability and streamlines the debugging process for the development team.
- **Dependency Injection:** Utilizing the MultiProvider in main.dart, the application injects required dependencies and state management objects directly into the widget tree. This approach eliminates the need for hardcoding dependencies within individual screens, resulting in a decoupled architecture that improves overall performance and testability.
- **Separation of Concerns (SoC):** By adopting the **MVVM (Model-View-ViewModel)** pattern, the application enforces a strict separation between the user interface, business logic, and data structures. This organization ensures that the UI layer remains independent of the core application logic, allowing for a more professional, modular, and organized development workflow.

4.3.2 Front-End

1. Technologies, Tools, and Programming Languages

The frontend was developed using a **multi-layered technology stack**, carefully selected to balance performance, scalability, and usability.

- **React.js Framework**
 - Component-based architecture enabling modular design and reusability.
 - Virtual DOM rendering ensures responsiveness and efficient updates.
- **JavaScript (ES6+)**
 - Modern features such as arrow functions, async/await, and destructuring improved readability and reduced redundancy.
 - Supported dynamic rendering and API integration.
- **CSS & Material Design Principles**
 - Ensured responsive layouts across devices.
 - Applied accessibility guidelines (contrast ratios, typography, spacing).
 - Material Design provided a consistent visual language, reducing cognitive load.
- **Axios (HTTP Client)**
 - Centralized API handler for GET, POST, PUT, DELETE requests.
 - Configured base URLs, headers, and token authentication.
 - Ensured consistent error handling and retry logic.
- **WebSocket Protocol**
 - Enabled real-time bidirectional communication between frontend and backend.
 - Critical for streaming gesture data with minimal delay.
- **Development Tools**
 - **Visual Studio Code:** IDE for efficient coding and debugging.
 - **Git/GitHub:** Version control and collaborative development.
 - **Postman:** API testing and validation.
 - **Browser DevTools:** Performance monitoring and UI debugging.

2. Software Architectural Techniques & Principles

The frontend of **FlexEra** was designed and implemented according to established software architectural techniques and engineering principles. These ensured modularity, scalability, maintainability, and a responsive, accessible user experience.

1. Modular Component-Based Design : The architecture followed a component-based approach with strict adherence to the Single Responsibility Principle (SRP). Each component was independent, reusable, and focused on a single functionality (e.g., authentication, booking, dashboard). Shared elements such as navigation bars and buttons were abstracted into a common library, ensuring consistency, reducing duplication, and simplifying future extensions.

2. Separation of Concerns & Client-Server Model :

Responsibilities were clearly divided into layers:

- UI Layer: React components.
- Logic Layer: Hooks and functions.
- Data Layer: API communication via Axios and WebSocket.

This separation minimized coupling, improved maintainability, and allowed independent evolution of frontend and backend. The client-server model reinforced this independence, with the frontend interacting only through standardized RESTful APIs and WebSocket streams, enhancing scalability and security.

3. Usability, Accessibility & Performance

Human-centered design principles were applied, following WCAG 2.1 guidelines to ensure inclusivity for diverse users. Responsive layouts using CSS Flexbox and Grid guaranteed adaptability across devices. Performance was optimized through lazy loading and code splitting, while accessibility-first practices ensured a consistent and inclusive user experience.

4.3.3 Back-End “API Implementation”

The backend architecture of Flexera is engineered using a hybrid service-oriented approach, specifically designed to handle intensive computational tasks while maintaining high-speed responsiveness for user management. The system is split into two specialized engines:

1. Programming Languages:

- **JavaScript (Node.js & ES6+):** Chosen for the core business logic due to its event-driven, non-blocking I/O model. This allows Flexera to handle hundreds of concurrent user interactions—such as booking and notifications—with minimal latency.
- **Python (FastAPI):** Utilized as the high-speed engine for AI processing, leveraging Python's superior ecosystem for computer vision and mathematical modeling.

2. AI Performance & Pose Estimation Engine (FastAPI)

The core "intelligence" of the system is built using **FastAPI**, a high-performance Python framework. This service is dedicated to real-time physical therapy exercise analysis.

3. Central Business & Data Logic (Node.js & Express.js)

To complement the AI capabilities, a robust **RESTful API** was developed using **Node.js** and **Express.js**. This service acts as the central hub for the application's business operations.

- **Identity & Security Management:** Implements a multi-layered authentication system using **JWT (JSON Web Tokens)** for session persistence, **Bcrypt.js** for secure password hashing, and **OTP (One-Time Password)** verification via **Nodemailer** for secure password resets.
- **Administrative & Doctor Management:** (As seen in `adminController.js` and `doctorController.js`), this module provides a comprehensive

dashboard for staff to manage medical personnel, verify doctor credentials, and oversee all paid appointments across the platform.

- **Automated Scheduling Engine:** Manages complex time-slot logic. It allows doctors to define dynamic availability and ensures transactional integrity during bookings to prevent race conditions.
- **Financial Transaction Hub (Paymob Integration):** Manages secure handshakes with the **Paymob** gateway. It utilizes **HMAC (Hash-based Message Authentication Code)** to verify the authenticity of transaction callbacks, ensuring that appointments are only confirmed upon validated payment.
- **Real-Time Communication Hub:** Coordinates with **Firestore Cloud Messaging (FCM)** for push notifications and **Pusher** for instant in-app updates, ensuring patients never miss a session.

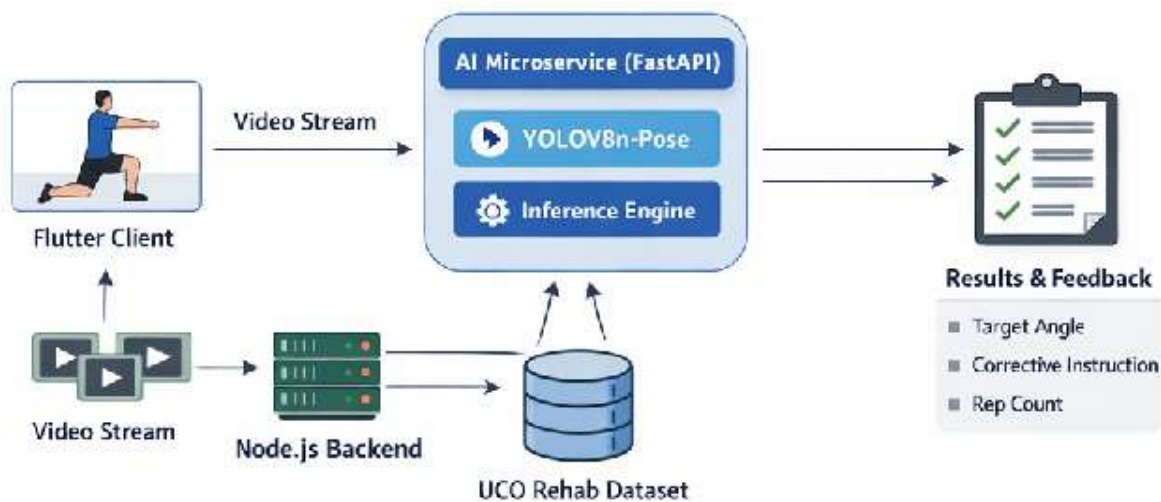
4. Database Integration & Schema Design

All application data has persisted using **MongoDB**, a NoSQL database that offers the flexibility required for multimedia-rich medical data. The integration is managed via **Mongoose**, providing a structured schema-based modeling environment:

- **Dynamic Medical Profiles:** Stores detailed user information, including biological data, medical files (hosted on **Cloudinary**[20]), and historical exercise performance.
- **Relational Mapping in NoSQL:** Despite being a NoSQL database, the system uses **Mongoose Population** to link Doctors, Patients, and Appointment slots, ensuring a relational-like data integrity for scheduling.
- **Notification Logs:** Persistent storage for user notifications, categorized by type (Broadcasts, Reminders, and Alerts), allowing for a synchronized notification history across devices.

4.3.4 AI

AI Architecture



The AI component of Flexera is implemented as a dedicated Python microservice built on Fast API, deliberately separated from the Node.js business logic layer to prevent computationally intensive inference from blocking user-facing operations such as scheduling and payment processing.

The pose estimation pipeline is built on **YOLOv8n-Pose**, the nano variant of Ultralytics' YOLOv8 architecture, which detects 17 anatomical key points per video frame using the COCO key point schema. The nano variant was selected for its optimized balance between inference speed and accuracy, enabling real-time processing without requiring dedicated GPU hardware on the server.

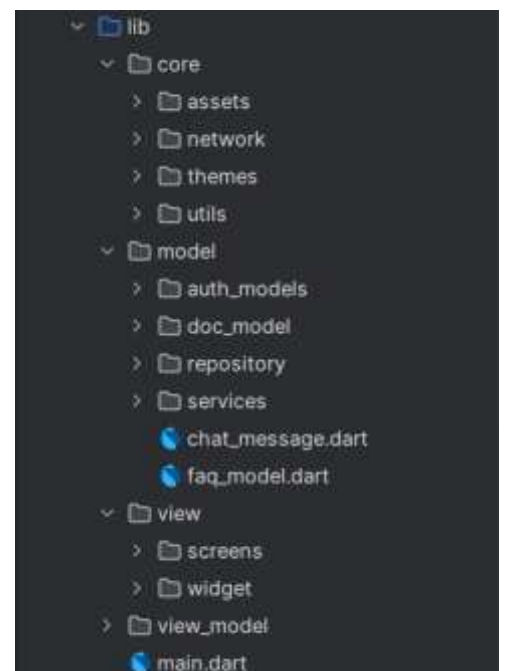
Joint angles are computed from fixed key point triplets specific to each exercise type. For each triplet — defined as three anatomical landmarks where the middle point is the joint of interest — the angle is calculated using the dot product of the two vectors formed by the surrounding points. The 16 supported exercises are grouped by joint type.

Rather than relying on clinically prescribed angle ranges from literature, the system derives its correction thresholds empirically from the UCO Physical Rehabilitation Dataset. The calibration process ran YOLOv8n-Pose across camera videos for all 27 subjects, sampling every 5 frames at a key point confidence threshold of 0.5. For each exercise, the 5th and 95th percentiles of the observed angle distribution were computed across all subjects and expanded by a margin to produce a statistically grounded valid range.

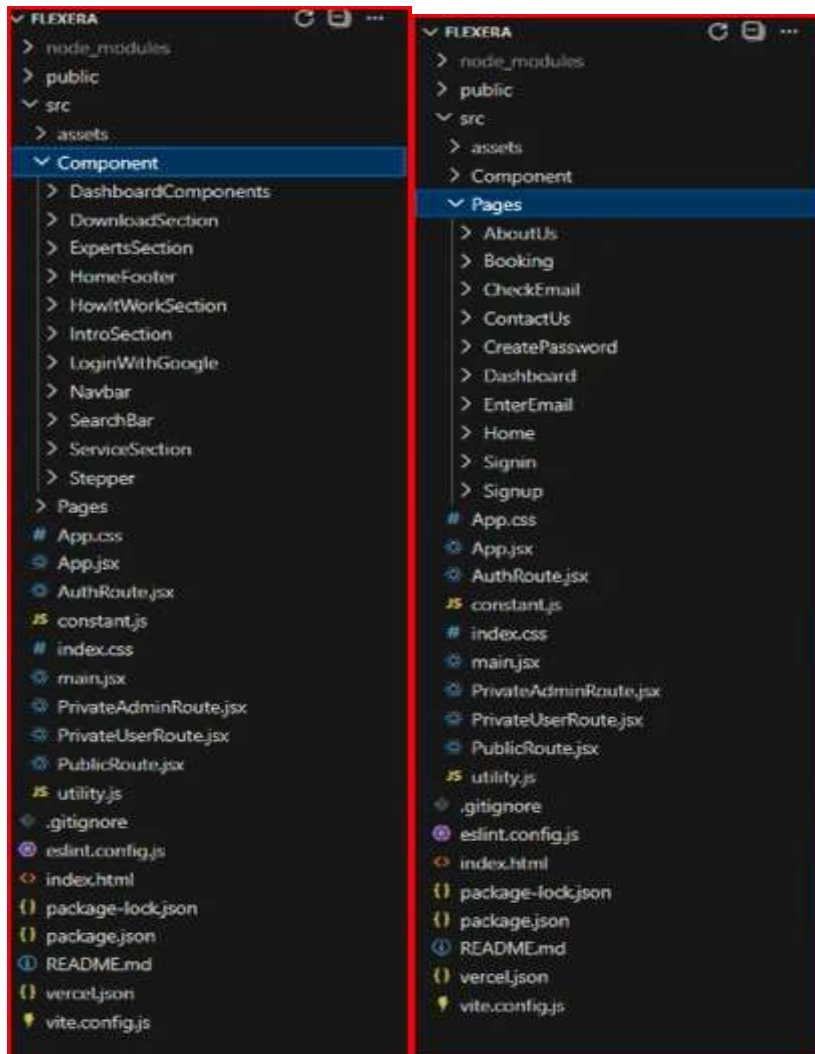
4.4.1 Mobile Application

The system architecture is organized into distinct modules to maintain the "Separation of Concerns" principle:

- **Network Layer (DioHelper):** A centralized module that encapsulates all API interactions (GET, POST, PUT, DELETE). It handles global configurations like BaseOptions, timeouts, and authorization headers using Bearer tokens.
- **Persistence Layer (CacheHelper):** A wrapper around SharedPreferences that provides a unified interface for storing and retrieving primitive data types and string lists, ensuring data consistency throughout the app lifecycle.
- **View-Model Layer:** This layer acts as the intermediary between the models and the views. For instance:
 - **LoginViewModel:** Manages authentication logic, form validation (Regex-based email validation), and coordinates with NotificationRepo to sync FCM tokens upon successful login.
 - **AccountViewModel & Repository:** Handles profile updates and data fetching, utilizing a repository pattern to abstract the data source from the UI.
- **Navigation & Routing:** The application uses a central routing table in main.dart, supporting both named routes and dynamic routing with arguments (e.g., passing Patient or Exercise objects between screens).
- **UI Components (Custom Widgets):** Reusable components such as CustomNavBar were developed to maintain visual consistency. The navigation bar dynamically adjusts based on the active index and theme (Light/Dark mode).



4.4.2 Front End



- **Layered Architecture**

The system is structured into distinct layers, each with a specific responsibility:

- **Network Layer (Axios Helper)**

Centralized API handler for GET, POST, PUT, DELETE requests

Configure base URLs, headers, and token authentication.

Provides consistent error handling and retrieve logic across modules.

- **Persistence Layer (Local Storage / Cache)**

Stores authentication tokens, user preferences, and session data.

Ensure continuity across reloads and sessions

- **View-Model Layer (Hooks & State Management)**

LoginViewModel: Manages authentication logic, form validation, and token storage.

BookingViewModel: Handles doctor search, slot selection, and appointment confirmation.

ProgressViewModel: Synchronizes exercise completion data with backend APIs.

DashboardViewModel: Abstracts admin operations such as adding/editing doctors and managing users.

- **Navigation & Routing**

Centralized routing table with public and protected routes.

Implements role-based access control (User vs Admin vs Doctor).

- **UI/UX Layer**

Built with reusable components (Navbar, Cards, Buttons).

Implements responsive design principles (Flexbox/Grid).

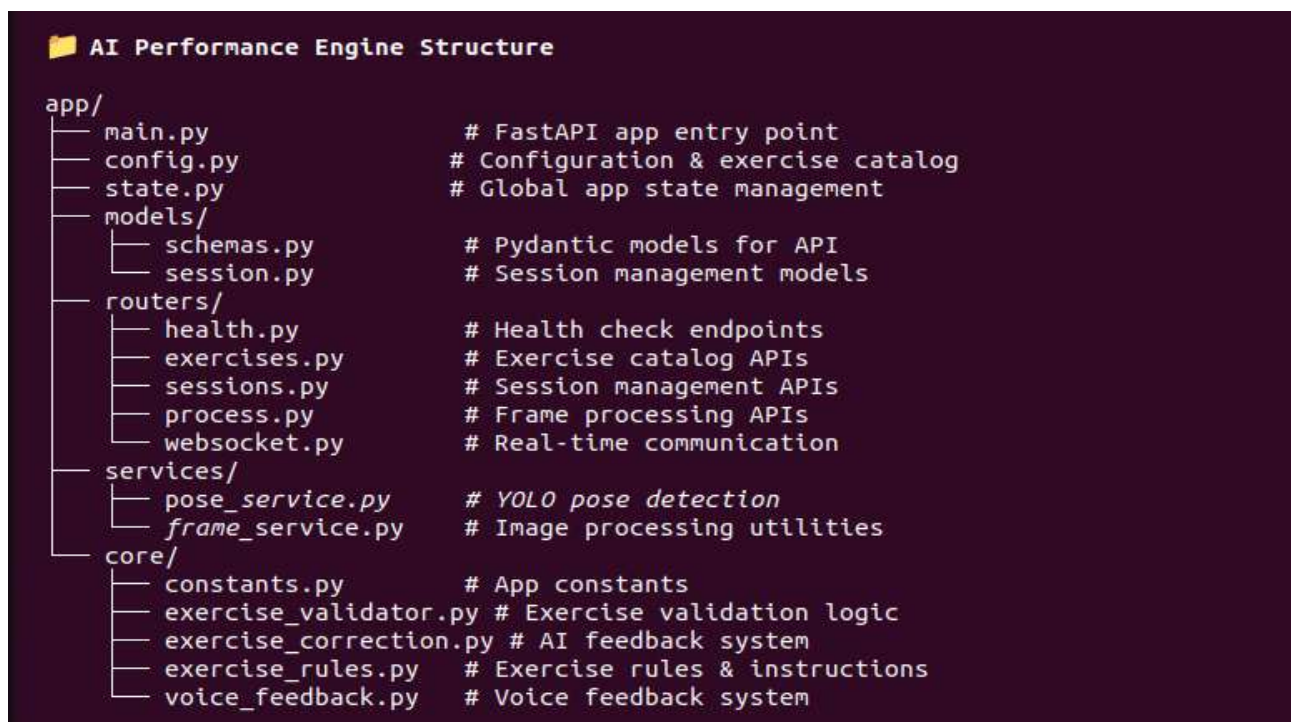
Ensures accessibility compliance with WCAG 2.1 guidelines.

4.4.3 Back-End

- **Identity & Access Management:** Manages multi-role logins, Google OAuth, and OTP verification (authController)
- **Scheduling & Appointment Engine:** A dynamic system for managing time slots and avoiding booking conflicts (scheduleController.js).
- **Financial Integration Module:** Handles the secure payment handshake with **Paymob**[17] and verifies transaction integrity (paymobController.js).
- **Smart Notification Hub:** Manages FCM tokens and sends automated reminders or administrative broadcasts (notificationController.js).



- **Fast API:**



AI Performance Engine Architecture and Workflow

Overview

The AI Performance Engine is an advanced physiotherapy assistance system developed using the FastAPI framework. It integrates computer vision and artificial intelligence technologies to provide real-time monitoring and evaluation of rehabilitation exercises. By leveraging the YOLOv8 pose estimation model and an intelligent exercise validation engine, the system analyzes user movements, validates exercise performance, counts repetitions, detects errors, and provides immediate corrective feedback.

The platform is designed to support clinical rehabilitation by improving exercise accuracy, tracking patient progress, and delivering personalized guidance throughout therapy sessions.

- **System Architecture**

The AI Performance Engine follows a modular architecture that separates responsibilities into independent components, making the system scalable, maintainable, and efficient.

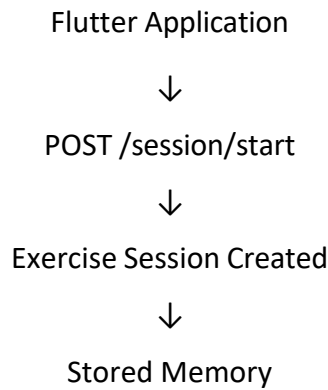
The major components include:

- FastAPI application server
- YOLOv8 pose detection service
- Exercise validation engine
- AI correction and feedback module
- Voice feedback system
- Session management module
- Configuration and exercise catalog
- Global state management

This architecture enables seamless communication between the mobile application and the backend AI services

- **Data Processing Pipeline**

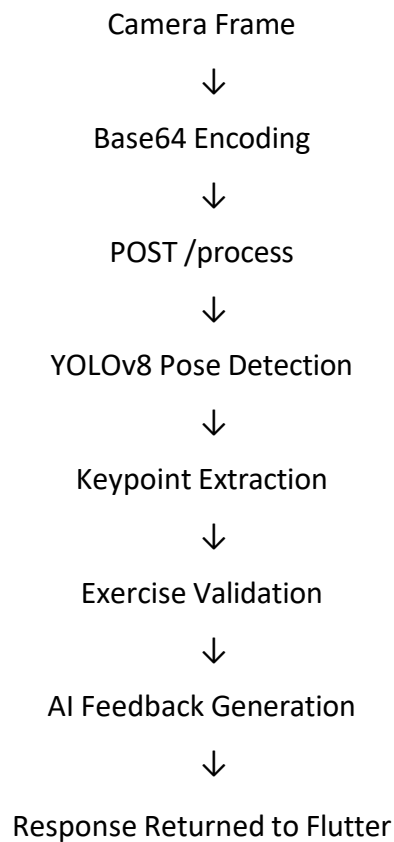
Before exercise processing begins, the Flutter mobile application sends a request to initialize a new exercise session.



The session stores exercise information, repetition goals, progress, and user-specific tracking data.

- **Frame Processing Pipeline**

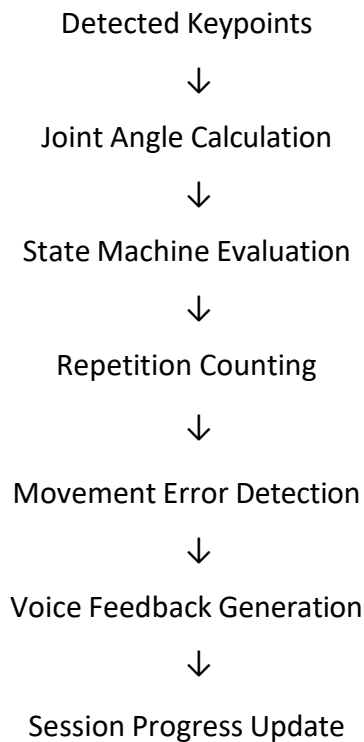
For every captured camera frame, the following sequence is executed:



This pipeline allows continuous monitoring with low latency suitable for real-time rehabilitation assistance

- **Real-Time Validation Workflow**

After body keypoints are extracted, they are processed through multiple validation stages.



This workflow ensures that every movement is evaluated according to predefined biomechanical rules.

- **RESTAPI Endpoints**

System Endpoints

GET /health

Returns the overall system status, including:

- Model loading status
- Server availability
- Number of active exercise sessions

GET /docs

Provides automatically generated interactive API documentation using Swagger UI.

- **Exercise Management Endpoints**

GET /exercises

Returns the complete catalog of supported physiotherapy exercises.

GET /exercises/category/{category}

Filters exercises based on rehabilitation category such as:

- Knee rehabilitation
- Shoulder rehabilitation

GET /exercises/{exercise_id}

Returns detailed information about a specific exercise, including instructions and validation rules.

- **Session Management Endpoints**

POST /session/start

Creates a new exercise session using user-defined parameters.

GET /session/{session_key}

Retrieves the current session status and progress.

POST /session/{session_key}/next-set

Moves the user to the next exercise set.

POST /session/{session_key}/reset

Resets all counters and progress for the current session.

DELETE /session/{session_key}

Terminates the exercise session and releases allocated resources.

- **Frame Processing Endpoints**

POST /process

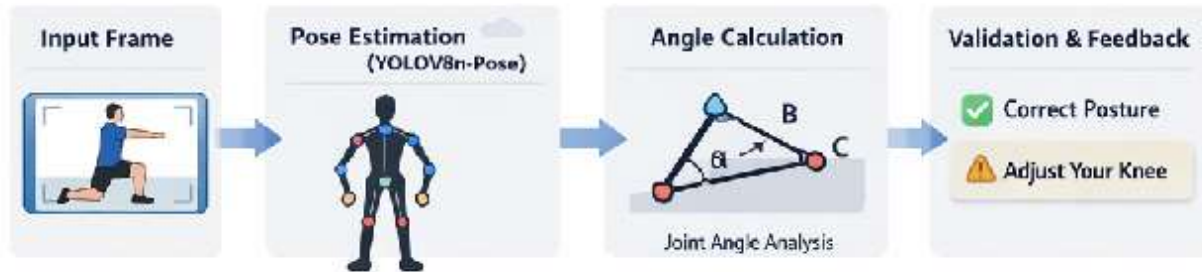
Processes an individual camera frame and performs AI-based exercise analysis.

WebSocket /ws

Provides low-latency bidirectional communication for continuous real-time processing

4.4.4 AI

AI Processing Pipeline



The AI engine is composed of three tightly integrated modules that operate sequentially to analyze movement and generate feedback.

The ExerciseValidator loads the calibrated minimum and maximum angle thresholds for a specified exercise and evaluates each incoming frame against the valid range.

The correction module generates directional corrective instruction when a deviation is detected for example, instructing the patient to extend the knee further or reduce shoulder elevation rather than producing a binary pass/fail signal.

The repetition counter tracks the angle trajectory across frames to detect completed movement cycles, incrementing the rep count when the joint completes a full range-of-motion excursion within the valid bounds. Together, these modules return a structured response to the Flutter client containing the current angle, the target range, the corrective message if applicable, and the updated repetition count.

4.5 Challenges and Resolutions

4.5.1 Mobile Applications

- **Challenge: Real-time Notification Synchronization.** Ensuring the backend stays updated with the user's latest FCM token across different login methods (Email/Google).
 - **Resolution:** Implemented a synchronized call within the LoginViewModel that triggers NotificationRepo.sendFcmToken immediately after a successful authentication event.
- **Challenge: UI Consistency Across Devices.** Maintaining the exact proportions of the Figma design on various Android and iOS screen aspect ratios.
 - **Resolution:** Integrated Flutter_ScreenUtil to wrap the MaterialApp, allowing for the use of .w, .h, and .sp extensions for all dimensions, ensuring a scalable UI.
- **Challenge: Complex Role-Based Navigation.** Managing different entry points for Patients and Doctors based on cached session data.
 - **Resolution:** Developed a conditional logic sequence in the main() function that checks token, onboarding, and is Doctor flags from Cache Helper before the app renders, ensuring a seamless user experience.
- **Challenge: Maintaining State Consistency in Multi-Step Forms.** Managing user input and validation states across complex, multi-step processes like sign-up and appointment booking required a robust mechanism to prevent data loss or UI desynchronization.
 - **Resolution:** This was addressed by adopting the **Provider** state management pattern. By housing the logic within dedicated ViewModels, the application maintains a persistent state that survives widget rebuilds, ensuring that user data is validated in real-time and remains consistent throughout the entire user journey.

4.5.2 Front End

- **Challenge: Real-Time Responsiveness.** Maintaining low latency for booking confirmations and progress updates.
 - **Resolution:** WebSocket streaming and optimized API calls.
- **Challenge: Complex State Management.** Multiple states (auth, booking, progress) required modular handling.
 - **Resolution:** React Hooks and Context API for structured state separation.
- **Challenge: Dynamic Forms.** Validation for login, booking, and dashboard forms.
 - **Solution:** Controlled components with regex-based validation.
- **Challenge: Cross-Platform Responsiveness.** Ensuring consistent rendering across devices.
 - **Resolution:** CSS Grid/Flexbox and responsive breakpoints
- **Challenge: Security & Authentication.** Protecting sensitive data.
 - **Resolution:** Token-based authentication and Firebase integration.
- **Challenge: Scalability.** Preparing for future modules.
 - **Resolution:** Component-based architecture and SRP (Single Responsibility Principle).

4.5.3 Back-End

- **Challenge: Real-time Computational Load & Latency.** Running AI models for pose estimation (MediaPipe) requires significant CPU/GPU resources. Integrating this directly into the main Node.js server threatened to cause severe latency, resulting in a poor user experience and slow API responses.
 - **Resolution:** A Microservices-inspired architecture was adopted. By offloading the AI inference logic to a dedicated FastAPI server, the main business logic (Node.js) remained lightweight. This separation ensured that the intense mathematical calculations for joint angles did not block the scheduling and payment services.
- **Challenge: Payment Synchronization & "Ghost Bookings".** Ensuring that a time slot is only officially booked after a successful financial transaction. Relying on the mobile app to confirm payment is insecure, as a user might close the app before the transaction is finalized.
 - **Resolution:** Implemented a Server-to-Server Webhook system using Paymob's callback mechanism. The backend (managed in `paymobController.js`) listens to an asynchronous digital signature (HMAC) from the payment gateway. The appointment status is only updated to "Paid" once the server cryptographically verifies the transaction's authenticity.
- **Challenge: Data Consistency & Race Conditions.** In a high-traffic environment, two patients might attempt to book the exact same time slot at the same millisecond.
 - **Resolution:** Developed a Conflict Detection Algorithm within the `bookTimeSlot` controller. By utilizing Atomic Operations in MongoDB, the system performs a final check on the `isBooked` status immediately before committing the transaction, effectively preventing "double-booking" scenarios.

- **Challenge: Secure Multi-Role Authorization.** The system serves three distinct user roles (Patients, Doctors, and Staff). Managing access control to sensitive medical files and administrative functions requires a strict security protocol.
 - **Resolution:** Implemented Custom Middleware Layers (protect and authorize). These layers decode the JWT (JSON Web Token) to verify the user's identity and role before granting access to specific endpoints, ensuring that medical data remains private and administrative tools remain secure.
- **Challenge: Reliable Real-time Notifications.** Delivering instant updates (e.g., appointment reminders or new doctor alerts) regardless of whether the application is active or running in the background.
 - **Resolution:** A hybrid notification strategy was used by integrating Pusher for real-time in-app events and Firebase Cloud Messaging (FCM) for background push notifications. This dual-layer approach ensures 100% deliverability of critical alerts to the patient's device.

4.5.4 AI

- **Challenge: Deriving Correction Thresholds Without Clinical Ground Truth.** Establishing what constitutes a correct joint angle for each exercise without access to per-patient clinical prescriptions presents a significant design challenge. Hardcoding values from published range-of-motion norms risk being too rigid for the natural variation observed across patients.
 - **Resolution.** A data-driven calibration script was developed that processed the UCO dataset across all 27 subjects to derive exercise-specific angle ranges statistically. The thresholds reflect realistic population-level movement variation rather than idealized clinical values, making the correction logic both robust and generalizable.
- **Challenge: Keypoint Visibility in Supine Exercises.** For exercises performed lying down, such as the bed-support knee bends and extended leg lifts, standard camera angles caused critical keypoints to be occluded or fall outside the model's confidence threshold, producing unreliable angle readings.
 - **Resolution: Camera Positioning and Patient Alignment Requirements.**
 - To ensure reliable keypoint detection and accurate angle computation, the system imposes specific physical setup requirements on the patient's environment. The camera must be positioned at a fixed height of approximately 1 meter from the ground, aligned horizontally so that the lens is level with the patient's mid-body region. This height ensures that the full body — from shoulders to ankles — falls within the camera's field of view during exercise execution,

which is critical for exercises involving lower limb movements such as knee flexion and hip extension.

- The patient must be positioned directly facing the camera with no significant lateral offset, maintaining a frontal orientation throughout the session. This frontal alignment is required because the YOLOv8n-Pose model's keypoint detection performs most reliably on forward-facing subjects, and angular deviation from the frontal plane introduces geometric distortion in the computed joint angles. For supine exercises — such as the extended leg lifts and bed-support knee bends — the camera should be repositioned to a lateral side view at the same height, capturing the sagittal plane of movement where the relevant joint angles are most accurately measured.

These constraints are communicated to the patient through the Flutter application prior to beginning an exercise session, ensuring that the physical environment is correctly configured before AI analysis is initiated

Following the design and analysis of the AI module, the next section evaluates the system through functional and real-time testing scenarios to assess its performance, responsiveness, and practical usability

Chapter 5

Testing

5.1 Testing Strategies

Testing activities were conducted incrementally throughout the development process. Since the backend APIs formed the foundation of the system and were developed before the user interface, API testing was performed first, followed by system testing, system integration, GUI testing, and usability testing.

5.1.1 API Testing

API testing was conducted to verify the functionality, reliability, security, and performance of the system's backend services. Since the APIs serve as the communication layer between the frontend application and the database, validating their correctness was essential before proceeding with further testing stages.

The APIs were tested independently of the user interface using Postman. Each endpoint was evaluated under normal, invalid, and edge-case conditions to ensure that the system handled requests and responses correctly.

Objectives

The objectives of API testing were:

- Verify that all API endpoints function as intended.
- Validate request and response formats.
- Ensure proper handling of valid and invalid inputs.
- Verify authentication and authorization mechanisms.
- Confirm appropriate HTTP status codes are returned.
- Detect and resolve backend defects before integration with the frontend.

Testing Environment

Item	Description
Testing Tool	Postman
Backend Framework	Node.js
Database	MongoDB.
API Format	Fast API, RESTful API

Testing Approach

- The following types of API tests were performed:

Functional Testing

- Verifies that each endpoint performs its intended operation correctly.

Validation Testing

- Checks whether invalid or missing data is properly rejected.

Authentication Testing

- Ensures protected endpoints require valid credentials or access tokens.

Error Handling Testing

- Verifies that meaningful error messages and appropriate status codes are returned.

Response Verification

- Confirms that response data structures match the expected JSON schema.

User Authentication APIs Test Cases

Test Case ID Scenario Input Expected Result Actual Result Status

Test Case ID	Scenario	Input	Expected Result	Actual Result	Status
API-001	Valid Login	Valid email and password	User authenticated and token returned	Token returned successfully	Pass
API-002	Invalid Password	Valid email, incorrect password	Authentication error	Authentication error displayed	Pass
API-003	Invalid Email	Non-existing email	User not found error	User not found error displayed	Pass
API-004	Empty Fields	Empty email and password	Validation error	Validation error returned	Pass
API-005	Missing Password	Email only	Validation error	Validation error returned	Pass

During API testing, a defect was identified in the doctor registration process. The system allowed a doctor to create multiple accounts using the same Doctor ID and email address while providing different passwords. This behavior violated the business rules of the system, which require that a doctor must first be added by a staff member before completing the registration process.

Defect ID	API-BUG-001
Module	Doctor Authentication / Registration
Severity	High
Priority	High
Status	Fixed

Description:

- The registration API did not properly validate whether a doctor account already existed in the system before creating a new account. As a result, duplicate accounts could be created using the same Doctor ID and email address.

Preconditions:

- A doctor record exists in the system.
- The doctor has already completed the registration process successfully.

Steps to Reproduce:

1. Register a doctor account using a valid Doctor ID, email address, and password.
2. Complete the registration successfully.
3. Attempt to register again using the same Doctor ID and email address.
4. Enter a different password.
5. Submit the registration request.

Expected Result:

- The system should reject the registration request and display an error message indicating that an account already exists with the provided Doctor ID or email address.

Actual Result:

- The system accepted the registration request and created another account with the same Doctor ID and email address but a different password.

Impact:

- This defect could lead to duplicate doctor accounts, data inconsistency, authentication conflicts, and unauthorized access to doctor-related information.

Root Cause:

- The registration API lacked validation checks to verify the uniqueness of the Doctor ID and email address before creating a new account.

Resolution:

Additional validation rules were implemented in the registration API to verify that:

- The Doctor ID exists in the staff-managed doctor records.
- The Doctor ID has not already been registered.
- The email address is unique within the system.

After applying the fix, duplicate registration attempts were correctly rejected, and the defect was closed.

HTTP Status Code Verification

Status Code	Meaning	Tested
200	Success	✓
201	Resource Created	✓
400	Bad Request	✓
401	Unauthorized	✓
403	Forbidden	✓
404	Not Found	✓
500	Internal Server Error	✓

Results and Findings

A total of 100 API test cases were executed using Postman.

Metric	Value
Total Test Cases	100
Passed	80
Failed	20
Pass Rate	80%

After fixing the functions that did not pass, retesting was conducted to ensure that all the test cases passed, and the pass rate is now 100%. The API testing phase successfully verified the correctness of the backend services. Several issues related to input validation and error handling were identified and corrected before proceeding to integration testing. After applying the fixes, all critical API endpoints passed their respective test cases.

5.1.2 System Testing

Introduction

System Testing was conducted to evaluate the complete integrated system against the specified functional and non-functional requirements. The testing process focused on validating end-to-end business workflows, ensuring that all system components worked together correctly, and verifying that the application met user expectations.

The testing was performed in an environment that closely resembled the production environment, allowing the system to be evaluated under realistic operating conditions.

Objectives

The objectives of System Testing were:

- Verify that all functional requirements are correctly implemented.
- Validate complete user workflows from start to finish.
- Ensure proper interaction between system components.
- Verify data consistency across the system.
- Detect defects that may arise in real-world usage scenarios.
- Confirm that the system satisfies business requirements.

Testing Environment

Item	Description
Testing Type	System Testing
Environment	Test Environment
Mobile Application	Flutter
Frontend	React.js
Backend	Node.js
Database	MongoDB
Testing Method	Manual Testing
Testing Tool	Browser / Application Interface

Testing Approach

- System testing was performed using scenario-based testing. Each major user workflow was executed to ensure that the system behaved as expected under normal and exceptional conditions.

The testing covered:

Patient Application

- User Authentication and Password Recovery
- Profile Management
- Settings Management
- Contact Us and AI Chat
- Doctor Booking and Payment Processing
- Upcoming Appointments Management
- Progress Tracking
- AI-Assisted Exercise Sessions
- Health Tips
- Notifications Management

Doctor Application

- Doctor Profile Management
- Patient Management

- Exercise Assignment and Modification
- Appointment Management
- Clinic Schedule Management

Integration Features

- Appointment Synchronization
- Exercise Synchronization
- Notification Synchronization
- Payment Integration
- AI Services Integration

Staff Member Web Portal

- Staff Authentication and Dashboard Access
- Patients Management (View patient records)
- Doctors Management (Add, Edit, Delete, View doctors)
- Doctor Information Management (Update and maintain doctor details across the system)
- Appointments Management (View all booked appointments with full details: doctor, patient, date, time)
- System-wide Data Monitoring and Administrative Control

Test Cases

Test Case ID: ST-001

Title: First-Time User Registration

Preconditions:

- User has completed the onboarding flow.
- User is on the Authentication screen.
- User does not already have an account.

Steps:

1. Choose the manual sign-up option.
2. Enter personal information (full name, email, etc.).
3. Enter a valid password.
4. Confirm the password.
5. Press the Sign Up button.

Expected Result:

- User account is created successfully.
- User is redirected to the Home screen.

Status: Pass

Test Case ID: ST-002

Title: Existing User Login Using Email

Preconditions:

- User has already completed registration and has an active account.
- User is on the Authentication/Login screen.

Steps:

1. Enter the registered email address.
2. Enter the correct password.
3. Press the Log In button.

Expected Result:

- User is authenticated successfully.
- User is redirected to the Home screen.

Status: Pass

Test Case ID: ST-003

Title: Authentication via Google Sign-In

Preconditions:

- User has completed the onboarding flow.
- User is on the Authentication screen.
- User has an active Google account on their device.

Steps:

1. Press the "Continue with Google" button.
2. Select the desired Google account from the native prompt.

Expected Result:

- User (whether first-time or existing) is authenticated instantly via Google.
- User is redirected to the Home screen.

Status: Pass

Test Case ID: ST-004

Title: Forgot Password - OTP Request and Verification

Preconditions:

- User has an existing registered account.
- User is on the Login screen.

Steps:

1. Press the "Forgot Password" button.
2. Enter the registered email address.
3. Press the button to send the OTP.
4. Retrieve the OTP code from the email inbox.
5. Enter the received OTP code on the verification screen.

Expected Result:

- OTP code is accepted as valid.
- User is redirected to the Create New Password screen.

Status: Pass

Test Case ID: ST-005

Title: New Password Validation and Reset

Preconditions:

- User has successfully verified their OTP.
- User is on the Create New Password screen.

Steps:

1. Verify the "Reset Password" button is initially disabled.
2. Enter a new password that meets all rules (at least 8 characters, 1 uppercase, 1 lowercase, 1 number or symbol).
3. Enter a mismatched password in the confirmation field (Verify button remains disabled).
4. Enter the exact matching password in the confirmation field (Verify button becomes enabled).
5. Press the "Reset Password" button.
6. Log in from the Login screen using the newly created password.

Expected Result:

- Button enables only when all validation rules and matching conditions are met.
- Password is changed successfully and user is redirected to the Login screen.
- User logs in successfully with the new password

Status: Pass

Test Case ID: ST-006

Title: Bottom Navigation Bar & Default Screen Validation

Preconditions:

- User has successfully logged into the application.

Steps:

1. Observe the default screen immediately after login.
2. Verify the bottom navigation bar contains four sections: Home, Settings, Profile, and Contact Us.
3. Press the "Settings" tab.
4. Press the "Profile" tab.
5. Press the "Contact Us" tab.
6. Press the "Home" tab.

Expected Result:

- The default screen after login is the Home screen.
- The user is successfully navigated to the respective screens upon pressing each tab in the bottom navigation bar.

Status: Pass

Test Case ID: ST-007

Title: Settings - User Information Card Redirect

Preconditions:

- User is logged in and on the Settings screen.

Steps:

1. Verify the user information card at the top displays the user's profile picture, name, and email address.
2. Press the User Information Card.

Expected Result:

- The card displays the correct user details.

- User is successfully redirected to the Profile screen upon tapping the card.

Status: Pass

Test Case ID: ST-008

Title: Settings - Preferences & Controls

Preconditions:

- User is logged in and on the Settings screen.

Steps:

1. Toggle the "Notification" switch to enable and disable notifications.
2. Toggle the "Dark Mode" switch to switch between Light and Dark mode.
3. Press the "Reset Mode Sheet" option and reset the current settings.

Expected Result:

- Notification toggle successfully updates the notification state.
- Dark Mode toggle immediately reflects UI changes (Light to Dark, and vice versa).
- Reset Mode Sheet opens successfully and reverts modes/settings to their default state.

Status: Pass

Test Case ID: ST-009

Title: Settings - Support & Help Navigation

Preconditions:

- User is logged in and on the Settings screen.

Steps:

1. Press the "Support & Help" option.
2. Verify the presence of FAQs and their answers.

3. Verify the presence of the company website button and the "Contact Us" screen button.
4. Navigate back from the Support screen.

Expected Result:

- User is redirected to the Support screen successfully.
- FAQs, Website button, and Contact Us button are clearly displayed.
- User is safely returned to the Settings screen upon leaving the Support screen.

Status: Pass

Test Case ID: ST-010

Title: Settings - About Flexera

Preconditions:

- User is logged in and on the Settings screen.

Steps:

1. Scroll to the "About Flexera" section.
2. Press or view the "About Flexera" option.

Expected Result:

- The application accurately displays the relevant information, version, and details about Flexera.

Status: Pass

Test Case ID: ST-011

Title: Settings - Logout Confirmation Flow

Preconditions:

- User is logged in and on the Settings screen.

Steps:

1. Scroll to the bottom of the Settings screen.
2. Press the "Logout" button.
3. Observe the confirmation dialog.
4. Press the "Cancel" option.
5. Press the "Logout" button again.
6. Press the "Confirm" option.

Expected Result:

- Pressing "Logout" triggers a confirmation dialog.
- Pressing "Cancel" dismisses the dialog and keeps the user on the Settings screen.
- Pressing "Confirm" successfully logs the user out and redirects them to the Authentication/Login screen.

Status: Pass

Test Case ID: ST-012

Title: Profile Screen Navigation and UI Display

Preconditions:

- User is logged into the application.

Steps:

1. Press the "Profile" section in the bottom navigation bar.
2. Observe the elements displayed on the screen.

Expected Result:

- The Profile / Account Information screen opens successfully.
- The screen displays the user's Profile picture, Full name, Email address, Phone number, Date of birth, Gender, Height, and Weight.

Status: Pass

Test Case ID: ST-013

Title: Profile - Email Address Read-Only Validation

Preconditions:

- User is logged in and on the Profile screen.

Steps:

1. Locate the "Email Address" field.
2. Attempt to tap on or type inside the Email Address field to edit it.

Expected Result:

- The Email Address field is strictly read-only or disabled.
- The system does not allow the user to modify the email address.

Status: Pass

Test Case ID: ST-014

Title: Profile - Edit Personal Information and Submit

Preconditions:

- User is logged in and on the Profile screen.

Steps:

1. Tap on the Profile picture and select a new image.
2. Modify the text in the "Full name" and "Phone number" fields.
3. Change the selections for "Date of birth", "Gender", "Height", and "Weight".
4. Press the "Submit" button.
5. Navigate away from the Profile screen and return to it to verify data persistence.

Expected Result:

- All specified fields accept the new inputs or selections without error.
- Pressing "Submit" successfully saves the changes to the user's account.
- The updated information remains visible upon returning to the screen.

Status: Pass

Test Case ID: ST-015

Title: Profile - Medical File Upload Validation

Preconditions:

- User is logged in and on the Profile screen.
- A valid test document/file is available on the device.

Steps:

1. Scroll to the bottom of the Profile screen to locate the medical file upload section.
2. Press the upload button/area.
3. Select a valid medical file from the device's storage.
4. Verify the file is attached to the screen.
5. Press the "Submit" button.

Expected Result:

- The native device file picker opens successfully.
- The selected file is attached and visually confirmed on the UI.
- Pressing "Submit" saves the uploaded medical file to the user's profile successfully.

Status: Pass

Test Case ID: ST-016

Title: Contact Us Screen Navigation and Content Validation

Preconditions:

- User is logged into the application.

Steps:

1. Press the "Contact Us" section in the bottom navigation bar.
2. Observe the elements displayed on the screen.
3. Verify the presence of the company's phone numbers.
4. Verify the presence of links/icons for social media pages.
5. Verify the presence of any other listed communication channels.

Expected Result:

- The Contact Us screen opens successfully.
- All required company contact information (phone numbers, social media pages, and communication channels) is clearly displayed.

Status: Pass

Test Case ID: ST-017

Title: Live Chat Button Navigation

Preconditions:

- User is logged in and on the Contact Us screen.

Steps:

1. Locate the "Live Chat" button in the center of the screen.
2. Press the "Live Chat" button.

Expected Result:

- The user is successfully redirected to the "Chat with AI" screen.

Status: Pass

Test Case ID: ST-018

Title: AI Chat - In-Scope Topic Validation

Preconditions:

- User is logged in and on the "Chat with AI" screen.

Steps:

1. Type and send a question related to the application's features and usage.
2. Type and send a question asking for physical therapy guidance.
3. Type and send a question asking for recommended exercises.

Expected Result:

- The AI assistant comprehends the queries and replies with relevant, helpful information and assistance for all three topics.

Status: Pass

Test Case ID: ST-019

Title: AI Chat - Out-of-Scope Topic Validation

Preconditions:

- User is logged in and on the "Chat with AI" screen.

Steps:

1. Type and send a question completely unrelated to the app, physical therapy, or exercises (e.g., asking for a cooking recipe or financial advice).

Expected Result:

- The AI assistant gracefully handles the out-of-scope query.
- The AI responds with a standardized message stating that it cannot assist with that specific request.

Status: Pass

Test Case ID: ST-020

Title: AI Chat - Options Menu Functionality

Preconditions:

- User is logged in and on the "Chat with AI" screen.
- The user has some existing chat history.

Steps:

1. Locate and press the three-dot menu icon at the top of the chat screen.
2. Verify the options displayed in the dropdown menu.
3. Press the "Start a New Chat" option.
4. Re-open the menu and press the "View Chat History" option.

Expected Result:

- The menu expands to reveal exactly two options: "Start a New Chat" and "View Chat History".
- Pressing "Start a New Chat" clears the current active screen and starts a fresh conversation instance.
- Pressing "View Chat History" successfully redirects the user to a screen or modal displaying their previous AI conversations.

Status: Pass

Test Case ID: ST-021

Title: Navigate to Booking Screen and Verify Doctors List

Preconditions:

- User is logged into the application.
- User is on the Home screen.

Steps:

1. Locate and select the "Booking" feature/section from the Home screen.
2. Observe the redirected screen.
3. Verify that the Doctors List is displayed.
4. Check that each doctor's entry shows their name and profile picture.

Expected Result:

- User is successfully redirected to the Doctors Booking screen.
- The screen displays a list of doctors, including their names and profile pictures.

Status: Pass

Test Case ID: ST-022

Title: Doctor Search and "See All" Functionality

Preconditions:

- User is logged in and on the Doctors Booking screen.

Steps:

1. Locate the search bar at the top of the screen.
2. Enter the name of a specific, known doctor in the search bar.
3. Clear the search bar.
4. Locate and press the "See All" option.

Expected Result:

- The search bar successfully filters and displays the specific doctor by name.
- Pressing "See All" successfully expands the list or redirects to a full view of all available doctors.

Status: Pass

Test Case ID: ST-023

Title: Doctor Details and Appointment Selection

Preconditions:

- User is logged in and on the Doctors Booking screen.

Steps:

1. Select a specific doctor from the list.
2. Observe the Doctor Details screen.
3. Verify the display of available booking days, time slots, and the booking price.
4. Select a preferred appointment date and time slot.

Expected Result:

- The application displays the correct doctor details, including available days, times, and price.
- The user can successfully select a specific date and time slot for the appointment

Status: Pass

Test Case ID: ST-024

Title: Booking Confirmation Screen Validation

Preconditions:

- User has selected a doctor, date, and time slot.

Steps:

1. Proceed to the confirmation step after selecting the appointment details.
2. Review the information displayed on the Booking Confirmation screen.

3. Verify the presence and accuracy of the Doctor's name, selected date and time, and the booking price.

Expected Result:

- The user is redirected to the confirmation screen.
- All appointment details (Doctor name, date, time, and price) match the user's previous selections accurately.

Status: Pass

Test Case ID: ST-025

Title: Payment Process - Failed Transaction Handling

Preconditions:

- User is on the Booking Confirmation screen.
- User has selected the option to confirm the booking.

Steps:

1. Confirm the booking to initiate the online payment process.
2. Observe the redirection to the Paymob payment gateway.
3. Enter invalid bank card details or simulate a failed payment response from the gateway.

Expected Result:

- The payment process fails.
- The user is automatically redirected back to the booking confirmation screen to review the appointment details again.

Status: Pass

Test Case ID: ST-026

Title: Payment Process - Successful Transaction & Navigation

Preconditions:

- User is on the Booking Confirmation screen.
- User has valid bank card details for testing.

Steps:

1. Confirm the booking and proceed to the Paymob gateway.
2. Complete the payment using valid bank card details.
3. Observe the redirection to the Success screen.
4. Verify the presence of post-booking navigation options.
5. Test the "Back to Home" button.

Expected Result:

- Payment is processed successfully.
- User is redirected to a success screen confirming the booking.
- The screen provides options to stay, return to the doctors list, or go back to Home.
- Pressing "Back to Home" successfully returns the user to the Home screen.

Status: Pass

Test Case ID: ST-027

Title: Home Screen - Upcoming Appointments (No Active Bookings)

Preconditions:

- User is logged into the application.
- User has absolutely no booked appointments on their account.
- User is on the Home screen.

Steps:

1. Scroll to the "Upcoming Appointment" section on the Home screen.
2. Verify the UI state for empty bookings.

3. Press the "Book Now" button.

Expected Result:

- The section correctly identifies that there are no bookings and displays a "Book Now" button.
- Pressing "Book Now" successfully redirects the user to the Doctors Booking screen.

Status: Pass

Test Case ID: ST-028

Title: Home Screen - Upcoming Appointments (Existing Bookings)

Preconditions:

- User is logged in and has at least one active booked appointment.
- User is on the Home screen.

Steps:

1. Scroll to the "Upcoming Appointment" section on the Home screen.
2. Verify the displayed details of the nearest upcoming appointment (Doctor name, appointment day, and date).
3. Press the appointment card.
4. Observe the bottom sheet that appears.

Expected Result:

- The section displays the details of the nearest upcoming appointment correctly.
- Pressing the card opens a bottom sheet successfully.
- The bottom sheet is clearly divided into two sections: "Completed Appointments" and "Upcoming/Future Appointments".

Status: Pass

Test Case ID: ST-029

Title: Progress Screen Navigation and Exercise Card Content

Preconditions:

- User is logged into the application.
- The user's doctor has assigned at least one exercise.
- User is on the Home screen.

Steps:

1. Locate and select the "Progress" section from the Home screen.
2. Observe the screen layout and the list of assigned exercises.
3. Verify the information displayed on individual exercise cards.

Expected Result:

- User is successfully redirected to the Progress screen.
- Each assigned exercise card correctly displays the Exercise name, Number of repetitions, Exercise duration/time, and a Checkbox status indicator.

Status: Pass

Test Case ID: ST-030

Title: Progress Tracking Indicator Accuracy

Preconditions:

- User is logged in and on the Progress screen.
- User has a mix of completed and pending assigned exercises.

Steps:

1. Locate the interactive circular progress indicator at the top of the screen.
2. Note the current numerical completion percentage and the circle's colour fill level.
3. Simulate or trigger the completion of a pending exercise.
4. Observe the progress indicator.

Expected Result:

- The progress circle gradually changes colour corresponding to the new completion percentage.
- The numerical percentage displayed above the circle updates automatically and accurately reflects the new ratio of completed exercises.

Status: Pass

Test Case ID: ST-031

Title: Navigation from Progress to Specific Exercise Details

Preconditions:

- User is logged in and on the Progress screen.

Steps:

1. Tap on a specific assigned exercise card from the Progress list.
2. Observe the redirection.
3. Verify the contents of the newly opened screen.

Expected Result:

- The user is redirected directly to the specific Exercise Details screen for the selected exercise.
- The screen displays exercise instructions/tips and a "Start" button.

Status: Pass

Test Case ID: ST-032

Title: Exercises Feature - Browse and Select from Home

Preconditions:

- User is logged in and on the Home screen.

Steps:

1. Locate and select the "Exercises" section from the Home screen.
2. Browse the list of all available exercises.
3. Select any exercise from the list.
4. Verify the details displayed on the selected exercise screen.

Expected Result:

- The Exercises screen displays a comprehensive list of all available exercises.
- Selecting an exercise successfully opens its details screen.
- The details screen clearly shows exercise tips/instructions and a "Start" button

Status: Pass

Test Case ID: ST-033

Title: AI Exercise Session Initiation and Real-Time Guidance

Preconditions:

- User is logged in and on an Exercise Details screen.
- Device camera is functional.

Steps:

1. Press the "Start" button.
2. If prompted, grant camera permissions to the application.
3. Observe the initiation of the AI exercise model and camera feed.
4. Begin performing the exercise.
5. Observe the AI's guidance and tracking functionality.

Expected Result:

- The application successfully requests camera permission.
- The camera opens, and the AI exercise tracking model starts.

- The AI assistant accurately monitors performance and provides real-time voice guidance and on-screen text instructions to correct the user's form.

Status: Pass

Test Case ID: ST-034

Title: Exercise Completion and Progress Auto-Sync

Preconditions:

- User is logged in and currently in an active AI exercise session.

Steps:

1. Successfully complete the required repetitions/duration of the exercise according to the AI's tracking.
2. Observe the application's behaviour immediately after completion.
3. Review the completion confirmation screen.
4. Navigate back to the "Progress" section from the Home screen.
5. Check the status of the recently completed exercise and the circular progress indicator.

Expected Result:

- The AI camera session closes automatically upon completion.
- A completion screen appears confirming the successful exercise session.
- Inside the Progress section, the specific exercise is automatically marked with a checked status indicator.
- The overall progress percentage and circular indicator update accordingly.

Status: Pass

Test Case ID: ST-035

Title: Tips Feature Navigation and List Display

Preconditions:

- User is logged into the application.
- User is on the Home screen.

Steps:

1. Locate and select the "Tips" section from the Home screen.
2. Observe the redirected screen.
3. Verify that a list of health and exercise-related tips is displayed.
4. Check that each tip is presented as a selectable title or card.

Expected Result:

- User is successfully redirected to the Tips screen.
- The application displays a populated list of tip cards/titles prepared by the application team

Status: Pass

Test Case ID: ST-036

Title: Tip Details Screen Content and Source Reference

Preconditions:

- User is logged in and on the Tips screen.

Steps:

1. Select and press any specific tip card from the list.
2. Observe the opened Tip Details screen.
3. Verify the presence of the full tip/information content.
4. Verify the presence of a related image or illustration (e.g., stretching poses).
5. Check for the inclusion of the information source/reference at the bottom or alongside the text.

Expected Result:

- The Tip Details screen opens successfully.
- The full text content and contextual images/illustrations are clearly visible and relate to physical therapy, exercises, or health recommendations.
- The source reference for the tip is displayed to ensure credibility.

Status: Pass

Test Case ID: ST-037

Title: Medical Warning Display for High-Risk Tips

Preconditions:

- User is logged in and on the Tips screen.
- Test data includes a specific tip flagged in the backend as involving medical risks or requiring supervision.

Steps:

1. Locate and press the tip card known to involve medical risks.
2. Review the content on the Tip Details screen.
3. Check for the presence of a medical warning message.

Expected Result:

- A clear medical warning message is displayed prominently on the screen.
- The warning explicitly advises the user to consult a doctor before applying the recommendation.

Status: Pass

Test Case ID: ST-038

Title: Absence of Medical Warning for General/Safe Tips

Preconditions:

- User is logged in and on the Tips screen.

- Test data includes a general, safe tip that does not require medical supervision.

Steps:

1. Locate and press the general/safe tip card.
2. Review the content on the Tip Details screen.
3. Ensure no medical warning message is triggered or displayed.

Expected Result:

- The Tip Details screen opens showing the content, image, and source reference.
- No medical warning message is displayed, as the tip is safe for general application.

Status: Pass

Test Case ID: ST-039

Title: System Notification Triggers

Preconditions:

- User is logged into the application.
- Application notifications are enabled in the user's device settings.

Steps:

1. Trigger the addition of a new doctor in the backend system.
2. Trigger an existing doctor to add a new available appointment slot.
3. Trigger a doctor to assign or update an exercise for the user.
4. Simulate the system time reaching the threshold for an upcoming appointment reminder.
5. Check the application's notification tray/screen for received alerts.

Expected Result:

- User successfully receives separate notifications for: a new doctor being added, a new appointment becoming available, an exercise being assigned/updated, and an upcoming appointment reminder.

Status: Pass

Test Case ID: ST-040

Title: Notification Categorization by Time

Preconditions:

- User is logged in.
- System contains mock notifications dated for today, earlier within the current week, and earlier within the current month.

Steps:

1. Open the Notifications screen from the application.
2. Observe the structure and section headers grouping the notifications.
3. Verify the placement of specific notifications based on their timestamps.

Expected Result:

- Notifications are strictly categorized into three distinct sections: "Today", "This Week", and "This Month".
- Notifications fall into the correct category based on their received date.

Status: Pass

Test Case ID: ST-041

Title: Notification Read and Unread Behaviour

Preconditions:

- User is logged in and on the Notifications screen.
- There is at least one new, unread notification.

Steps:

1. Observe the visual state of the new, unread notification.
2. Press/open the unread notification.
3. Return to the Notifications screen.
4. Observe the visual state of the previously opened notification.

Expected Result:

- Unread notifications are visibly highlighted (active state).
- Once opened or read, the notification is marked as read and the highlight is permanently removed.

Status: Pass

Test Case ID: ST-042

Title: Swipe to Delete Single Notification

Preconditions:

- User is logged in and on the Notifications screen.
- There is at least one notification in the list.

Steps:

1. Select a specific notification.
2. Swipe the notification cell to the left.
3. Observe the list of notifications.

Expected Result:

- Swiping left triggers a delete action.
- The targeted notification is successfully removed from the list without affecting other notifications.

Status: Pass

Test Case ID: ST-043

Title: Bulk Action - Mark All as Read

Preconditions:

- User is logged in and on the Notifications screen.
- There are multiple highlighted (unread) notifications across different time categories.

Steps:

1. Locate the "Mark All as Read" button on the screen.
2. Press the "Mark All as Read" button.
3. Observe the visual state of all notifications in the list.

Expected Result:

- The system immediately marks all notifications as read.
- All active state highlights are turned off simultaneously across all sections.

Status: Pass

Test Case ID: ST-044

Title: Bulk Action - Delete All Notifications

Preconditions:

- User is logged in and on the Notifications screen.
- There are multiple notifications present in the list.

Steps:

1. Locate the "Delete All" button on the screen.
2. Press the "Delete All" button.
3. (Optional) Confirm any prompted deletion warning dialog.
4. Observe the Notifications screen.

Expected Result:

- The system removes all notifications from the screen at once.

- The Notifications screen displays an empty state.

Status: Pass

Test Case ID: ST-045

Title: Doctor Navigation Bar Validation

Preconditions:

- User is logged into the application with a valid Doctor account.

Steps:

1. Observe the default Home screen upon login.
2. Verify the bottom navigation bar contains exactly three sections: Home, Settings, and Profile.
3. Press the "Settings" tab.
4. Press the "Profile" tab.
5. Press the "Home" tab.

Expected Result:

- The bottom navigation bar displays only the three specified sections.
- The application successfully navigates to the respective screens upon pressing each tab.

Status: Pass

Test Case ID: ST-046

Title: Doctor Profile Display and Editable Fields Update

Preconditions:

- User is logged into the application with a Doctor account.
- User is on the Profile screen.

Steps:

1. Observe the elements displayed on the screen (Profile picture, Full name, Email address, Phone number, Date of birth, Gender, Doctor ID).
2. Tap on the Profile picture to select a new image.
3. Modify the text in the "Full name" and "Phone number" fields.
4. Change the selections for "Date of birth" and "Gender".
5. Press the "Submit" or "Save" button to apply changes.

Expected Result:

- The Profile screen correctly displays all listed information fields.
- The Profile picture, Full name, Phone number, Date of birth, and Gender fields accept modifications.
- Pressing the submit button successfully saves the updated information to the doctor's profile.

Status: Pass

Test Case ID: ST-047

Title: Doctor Profile - Read-Only Fields Validation

Preconditions:

- User is logged into the application with a Doctor account.
- User is on the Profile screen.

Steps:

1. Locate the "Email Address" field.
2. Attempt to tap on or edit the Email Address.
3. Locate the "Doctor ID" field.
4. Attempt to tap on or edit the Doctor ID.

Expected Result:

- Both the Email Address and Doctor ID fields are strictly read-only.

- The system does not allow the user to modify or type inside these fields under any circumstances.

Status: Pass

Test Case ID: ST-048

Title: Doctor Settings Screen - Included Features Validation

Preconditions:

- User is logged into the application with a Doctor account.
- User is on the Settings screen.

Steps:

1. Verify the presence of the User Information Card at the top.
2. Verify the presence of the "Dark Mode" toggle/option.
3. Verify the presence of the "Support & Help" option.
4. Verify the presence of the "About Flexera" section.
5. Verify the presence of the "Logout" button at the bottom.

Expected Result:

- All expected sections (User Info Card, Dark Mode, Support & Help, About Flexera, Logout) are correctly displayed and accessible on the Doctor Settings screen.

Status: Pass

Test Case ID: ST-049

Title: Doctor Settings Screen - Excluded Features Validation

Preconditions:

- User is logged into the application with a Doctor account.
- User is on the Settings screen.

Steps:

1. Scroll through the entire Settings screen.
2. Search for a "Notifications" settings toggle or section.
3. Search for a "Reset Mode Sheet" option.

Expected Result:

- The "Notifications" settings/toggle is completely absent from the screen.
- The "Reset Mode Sheet" option is completely absent from the screen.

Status: Pass

Test Case ID: ST-050

Title: My Patients List Navigation and Card Content

Preconditions:

- User is logged into the application with a Doctor account.
- The doctor has at least one assigned patient.
- User is on the Home screen.

Steps:

1. Locate and select the "My Patients" section from the Home screen.
2. Observe the redirected screen and the list of assigned patients.
3. Verify the information displayed on individual patient cards.

Expected Result:

- User is successfully redirected to the My Patients list.
- The screen displays all assigned patients.
- Each patient card correctly displays the patient's profile picture, patient name, and last session date.

Status: Pass

Test Case ID: ST-051

Title: Patient Profile and Medical File View

Preconditions:

- User is logged in as a Doctor and is on the "My Patients" list screen.

Steps:

1. Select a specific patient from the list.
2. Observe the Patient Profile screen.
3. Verify that the patient's basic details are visible.
4. Locate and view the patient's uploaded medical file.

Expected Result:

- The doctor is successfully navigated to the selected patient's profile screen.
- The patient's basic details and uploaded medical file are accessible and displayed correctly

Status: Pass

Test Case ID: ST-052

Title: Add New Exercise to Patient Profile

Preconditions:

- User is logged in as a Doctor and is on a specific Patient Profile screen.

Steps:

1. Locate and press the "Add Exercise" button.
2. Observe the popup/modal window.
3. Select an exercise category and exercise name.
4. Enter valid values in the Duration and Repetitions (Reps) fields.
5. Press the "Add" button inside the popup.

Expected Result:

- The Add Exercise popup opens successfully with all required fields.
- After pressing Add, the popup window closes automatically.
- The new exercise is instantly added to the patient's exercise list, displaying the correct name, reps, and duration.

Status: Pass

Test Case ID: ST-053

Title: Edit Existing Assigned Exercise

Preconditions:

- User is logged in as a Doctor and is on a Patient Profile screen.
- The patient has at least one assigned exercise in their list.

Steps:

1. Locate an existing exercise card and press the "Edit" button.
2. Verify that the popup window opens and automatically displays the existing exercise data.
3. Modify the exercise category, name, duration, and repetitions.
4. Save the changes inside the popup.

Expected Result:

- The popup loads with the current data pre-filled correctly.
- The application accepts the modifications.
- The popup closes and the exercise card on the main screen updates immediately to reflect the new values.

Status: Pass

Test Case ID: ST-054

Title: Delete Assigned Exercise

Preconditions:

- User is logged in as a Doctor and is on a Patient Profile screen.
- The patient has at least one assigned exercise in their list.

Steps:

1. Locate an existing exercise card.
2. Press the Delete (X) button on the exercise card.

Expected Result:

- The selected exercise is immediately removed from the patient's exercise list.

Status: Pass

Test Case ID: ST-055

Title: Save Profile Changes and Redirect

Preconditions:

- User is logged in as a Doctor and is on a Patient Profile screen.
- The doctor has made updates (added, edited, or deleted an exercise).

Steps:

1. Locate and press the "Save Changes" button on the Patient Profile screen.
2. Observe the application's navigation behaviour.

Expected Result:

- The application successfully saves all modifications to the database.
- The doctor is automatically redirected back to the My Patients list screen.

Status: Pass

Test Case ID: ST-056

Title: Patient Progress App Synchronization

Preconditions:

- A doctor has successfully added or updated an exercise for a specific patient and saved the changes.
- Access to the specific patient's account is available.

Steps:

1. Log into the application using the specific Patient account.
2. Navigate to the Home screen and open the "Progress" section.
3. Review the list of assigned exercises.

Expected Result:

- The exercises added or modified by the doctor are automatically synchronized.
- The patient sees the exact, updated exercise list (including correct names, reps, and durations) inside their Progress section.

Status: Pass

Test Case ID: ST-057

Title: Appointments Screen Navigation and Layout Validation

Preconditions:

- User is logged into the application with a valid Doctor account.
- User is on the Home screen.

Steps:

1. Locate and select the "Appointments" section from the doctor Home screen.
2. Observe the redirected screen.
3. Verify the presence of two distinct sections or tabs on the screen.

Expected Result:

- User is successfully redirected to the Appointments screen.
- The screen is clearly divided into two main sections: "Upcoming Appointments" and "Past Appointments".

Status: Pass

Test Case ID: ST-058

Title: Upcoming Appointments Data Display

Preconditions:

- User is logged into the application with a Doctor account.
- The doctor has at least one scheduled future appointment in the system.
- User is on the Appointments screen.

Steps:

1. Navigate to or view the "Upcoming Appointments" section.
2. Review the list of displayed appointment cards.
3. Verify the details shown on an individual upcoming appointment item.

Expected Result:

- The section accurately lists all future appointments that have not yet been completed.
- Each appointment item correctly displays the Patient name, Appointment date, and Appointment time.

Status: Pass

Test Case ID: ST-059

Title: Past Appointments (History) Data Display

Preconditions:

- User is logged into the application with a Doctor account.

- The doctor has at least one completed/previous appointment in the system.
- User is on the Appointments screen.

Steps:

1. Navigate to or view the "Past Appointments" section.
2. Review the list of displayed appointment cards.
3. Verify the details shown on an individual past appointment item.

Expected Result:

- The section successfully acts as an appointment history screen, listing completed/previous appointments.
- Each past appointment item correctly displays the Patient name, Previous appointment date, and Previous appointment time

Status: Pass

Test Case ID: ST-060

Title: Clinic Schedule Navigation and Monthly Calendar Display

Preconditions:

- User is logged into the application with a valid Doctor account.
- User is on the Home screen.

Steps:

1. Locate and select the "Clinic Schedule" section from the doctor Home screen.
2. Observe the redirected screen.
3. Verify the presence and functionality of the monthly calendar UI.

Expected Result:

- User is successfully redirected to the Clinic Schedule screen.

- A fully interactive monthly calendar is displayed prominently on the screen.

Status: Pass

Test Case ID: ST-061

Title: Add Single Available Appointment Time

Preconditions:

- User is logged in as a Doctor and is on the Clinic Schedule screen.

Steps:

1. Select a specific day/date from the monthly calendar (e.g., June 10).
2. Observe the popup/modal window that appears.
3. Use the time picker to select a preferred appointment time (e.g., 10:00 AM).
4. Press the "Add" button.

Expected Result:

- Selecting a day successfully opens a time picker popup.
- Pressing "Add" associates the selected time strictly with the chosen calendar day.

Status: Pass

Test Case ID: ST-062

Title: Add Multiple Appointment Times via "Add More"

Preconditions:

- User is logged in as a Doctor and is managing a specific day on the Clinic Schedule screen.
- At least one time slot has already been added to the selected day.

Steps:

1. Locate and press the "Add More" button for the currently selected day.
2. Use the time picker to select an additional, different appointment time.
3. Press the "Add" button.

Expected Result:

- The application allows the addition of multiple time slots for a single calendar day.
- The new time slot is saved alongside the previously added slots for that exact date.

Status: Pass

Test Case ID: ST-063

Title: Delete an Available Appointment Time

Preconditions:

- User is logged in as a Doctor and is on the Clinic Schedule screen.
- A specific day containing at least one available appointment time is selected.

Steps:

1. View the listed appointment times for the selected day.
2. Locate an incorrect or unwanted time slot.
3. Press the delete/remove icon or button associated with that specific time slot.

Expected Result:

- The selected time slot is immediately removed from the day's schedule.
- Other existing time slots for that day remain unaffected.

Status: Pass

Test Case ID: ST-064

Title: Schedule Overview and Management Panel

Preconditions:

- User is logged in as a Doctor and is on the Clinic Schedule screen.
- The doctor has previously added available appointments across multiple different days.

Steps:

1. Scroll to the Schedule Overview section located below the monthly calendar.
2. Verify that all days containing available appointments are listed.
3. Select one of the listed days from the overview.
4. Verify that the specific appointment times for that day expand/display.
5. Attempt to add a new time and delete an existing time from this overview state.

Expected Result:

- The overview accurately lists all active days with available slots.
- Selecting a day correctly displays its respective time slots.
- The doctor can freely manage (view, add, delete) the schedule directly from this overview panel.

Status: Pass

Test Case ID: ST-065

Title: Cross-App Synchronization - User Booking Visibility

Preconditions:

- The doctor has just added a new, specific appointment date and time (e.g., June 15 at 2:00 PM).
- Access to the Patient/User application is available.

Steps:

1. Log into the application using a Patient account.
2. Navigate to the Home screen and open the "Booking" feature.
3. Select the specific doctor who added the new time.
4. Check the available booking days and time slots for that doctor.

Expected Result:

- The newly added appointment time automatically synchronizes across the system.
- The patient can see and select the exact date and time (June 15 at 2:00 PM) on the doctor's booking details screen.

Status: Pass

Test Case ID: ST-066

Title: User Notification for New Appointment Availability

Preconditions:

- A patient has the application installed with notifications enabled.
- The doctor profile linked/viewable by the patient adds a new available appointment slot.

Steps:

1. As a Doctor, add a new available appointment time via the Clinic Schedule.
2. Observe the notification center or device lock screen of the Patient user.

Expected Result:

- Adding a new time slot triggers an automated system event.
- The user receives a push/in-app notification successfully informing them that the doctor has added a new available appointment.

Status: Pass

Test Case ID: ST-067

Title: Staff Member Login & Dashboard Access (Web Portal)

Preconditions:

- User has valid Staff Member credentials.
- User is accessing the application via the Web Portal URL.

Steps:

1. Navigate to the web portal login screen.
2. Enter the Staff Member email and password.
3. Press the "Log In" button.

Expected Result:

- System authenticates the staff member successfully.
- User is redirected to the staff management dashboard.

Status: Pass

Test Case ID: ST-068

Title: Patients Management - View Registered Patients

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- There are registered patients in the system database.

Steps:

1. Navigate to the "Patients Management" section from the dashboard.
2. Observe the displayed list of patients.
3. Select a specific patient to review their existing records and information.

Expected Result:

- The system displays a complete list of all registered patients.

- The staff member can successfully view detailed patient information and records.

Status: Pass

Test Case ID: ST-069

Title: Doctors Management - View Doctors List

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- There are existing doctors registered in the system.

Steps:

1. Navigate to the "Doctors Management" section.
2. Observe the displayed list of doctors.

Expected Result:

- ☐ The system displays a comprehensive list of all doctors currently in the system.

Status: Pass

Test Case ID: ST-070

Title: Doctors Management - Add New Doctor

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- User is on the Doctors Management screen.

Steps:

1. Press the "Add New Doctor" button.
2. Fill out the required doctor details (Name, Email, Phone, Specialization, etc.) in the provided form.

3. Press the "Save" or "Add" button.

Expected Result:

- The new doctor is created successfully.
- The doctor appears in the doctors list on the web portal.
- The doctor can now log into the mobile application using their new credentials.

Status: Pass

Test Case ID: ST-071

Title: Doctors Management - Edit Doctor Information

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- User is on the Doctors Management screen.

Steps:

1. Locate an existing doctor from the list.
2. Press the "Edit" button next to the doctor's name.
3. Modify specific details (e.g., Phone number, basic info).
4. Press the "Save Changes" button.
5. Check the doctor's profile on the mobile app to verify synchronization.

Expected Result:

- The web portal successfully saves the updated information.
- The modifications are applied globally, reflecting accurately when the doctor logs into their own application profile.

Status: Pass

Test Case ID: ST-072

Title: Doctors Management - Delete Doctor

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- User is on the Doctors Management screen.
- A test doctor account exists that is safe to delete.

Steps:

1. Locate the test doctor in the list.
2. Press the "Delete" button.
3. Confirm the deletion in the warning prompt/dialog.

Expected Result:

- The doctor is permanently removed from the system.
- The doctor is no longer visible in the staff portal's list.
- The doctor can no longer log into the application.

Status: Pass

Test Case ID: ST-073

Title: Appointments Management - View Booked Appointments

Preconditions:

- User is logged into the Web Portal as a Staff Member.
- There are active booked appointments between patients and doctors in the system.

Steps:

1. Navigate to the "Appointments Management" section.
2. Review the data displayed in the appointments table/list.

Expected Result:

- The screen displays all scheduled sessions.

- Each appointment record accurately shows: Doctor name, Patient name, Appointment date, Appointment time, and Booking details.

Status: Pass

Regression Testing

- To ensure that newly implemented features and bug fixes did not negatively affect existing functionality, regression testing was performed after each major update. Previously executed test cases were re-run to verify that core system functions continued to operate correctly.

Findings

- The System Testing phase verified that the complete application functioned according to the specified requirements. All critical user workflows were successfully executed. Any defects identified during testing were corrected and retested to ensure system stability and reliability.

Conclusion

System testing confirmed that all main features of the application work correctly when integrated together.

The system successfully handled different user scenarios including authentication, booking, AI features, and payments.

Overall, the application is stable, functional, and ready for deployment with only minor improvements needed.

5.1.3 System Integration Testing

Introduction

System Integration Testing (SIT) was performed to verify the interaction between different modules of the system after integrating all components. Unlike unit or API testing, this phase focuses on validating the communication

and data flow between multiple subsystems to ensure that the entire application works as a unified system.

In this project, special attention was given to the integration of external services, including the Paymob payment gateway and the AI model service integrated within the application. These integrations were critical to the system's functionality, as they directly affect payment processing and intelligent feature generation.

The main goal of this phase was to ensure that all internal modules and external services interact correctly, reliably, and securely under different conditions.

Objectives

The objectives of System Integration Testing were:

- Verify seamless interaction between frontend and backend modules.
- Ensure correct integration with the Paymob payment gateway.
- Validate successful payment processing and callback handling.
- Verify integration of the AI model with the application.
- Ensure correct data flow between system components.
- Detect interface mismatches between integrated modules.
- Ensure system stability after combining all components.

Integration Scope

- The system integration testing covered the following major integrations:

1. Paymob Payment Integration

- Payment initialization request from backend
- Redirect to Paymob payment page
- Payment confirmation callback handling
- Order status update after successful payment
- Handling failed or canceled transactions

2. AI Model Integration

- Sending user/system input to AI endpoint
- Receiving predictions/recommendations
- Displaying AI-generated results in the application
- Handling latency and API failure cases
- Ensuring correct formatting of AI responses

Testing Approach

- The following testing approaches were used during system integration testing:

End-to-End Testing

- Simulated complete user workflows from frontend interaction to backend processing and external service response.

Interface Testing

Verified correct communication between:

- Frontend ↔ Backend
- Backend ↔ Paymob
- Backend ↔ AI Model Service

Data Flow Validation

- Ensured that data passed between modules remained consistent, accurate, and correctly formatted.

Failure Handling Testing

Verified system behaviour when:

- Payment fails or is cancelled
- AI service is unavailable
- Network delays occur

- Invalid data is sent to external services

Findings

- Paymob integration successfully handled all payment scenarios including success, failure, and cancellation flows.
- Order status synchronization between Paymob and the backend was accurate and reliable.
- AI model integration successfully returned responses under normal conditions.
- The system properly handled AI service delays and failures without crashing.
- Error handling mechanisms ensured system stability during external service failures.

Conclusion

The System Integration Testing phase confirmed that all modules and external services are correctly integrated. Both Paymob payment integration and AI model integration functioned as expected, ensuring a stable and reliable end-to-end system workflow.

5.1.4 GUI Testing

Introduction

Graphical User Interface (GUI) Testing was conducted to ensure that the system's visual components function correctly and provide a consistent and user-friendly experience across all screens. This type of testing focuses on validating the layout, design elements, navigation flow, and overall usability of the application interface.

Since the GUI represents the primary interaction point between the user and the system, it was essential to verify that all interface components behave as expected across different modules, including authentication, doctor browsing, booking, Appointments, payment integration, and AI-based features.

The objective of this phase was to ensure that the application interface is visually consistent, responsive, and free from usability defects.

Objectives

The objectives of GUI Testing were:

- Verify that all UI components are displayed correctly
- Ensure consistency in design across all screens
- Validate navigation flow between pages/screens
- Check responsiveness of the interface on different screen sizes
- Ensure proper alignment of buttons, forms, and text fields
- Verify visibility and clarity of error and success messages
- Ensure smooth user interaction without UI glitches

Testing Approach

- GUI testing was performed using a **manual exploratory testing approach**, where each screen of the application was reviewed against the UI design specifications.

The following aspects were validated:

Layout Verification

- Ensuring that UI elements are properly aligned and visually structured.

Navigation Testing

- Verifying that users can move between screens without broken links or incorrect routing.

Form Validation UI

- Checking error messages, input highlights, and validation feedback.

Responsiveness Testing

- Ensuring UI adapts correctly to different screen sizes and resolutions.

Consistency Testing

- Verifying uniform fonts, colors, button styles, and spacing across the system.

Findings

- The user interface was consistent across all system modules
- Navigation between pages was smooth and error-free
- Form validation messages were clear and properly positioned
- Paymob payment pages were correctly redirected and displayed
- AI module interface handled loading states and responses effectively
- Minor alignment issues (if any) were identified and corrected during testing

Conclusion

The GUI Testing phase confirmed that the system provides a clean, responsive, and user-friendly interface. All visual components behaved as expected, ensuring a smooth user experience across web and mobile views. The application interface is considered stable and ready for deployment.

5.1.5 Usability Testing

Introduction

Usability Testing was conducted to evaluate the system from the end-user perspective, focusing on how easy and efficient it is to use the application. Unlike functional testing, which verifies system correctness, usability testing focuses on user experience, intuitiveness, and overall satisfaction when interacting with the system.

The primary goal of this phase was to ensure that users can complete tasks smoothly without confusion or unnecessary complexity. This includes interacting with features such as authentication, doctor browsing, booking, Appointments, and AI-based features, payment processing through Paymob.

Objectives

The objectives of Usability Testing were:

- Evaluate the ease of use of the system interface.
- Ensure that users can navigate the system intuitively.
- Assess the clarity of labels, buttons, and instructions.
- Verify that system workflows are simple and efficient.
- Identify any user experience difficulties or confusion points.
- Ensure smooth interaction with key features .
- Improve overall user satisfaction and accessibility.

Test Scenarios

Authentication & Password Reset

- Users registered a new account using manual sign-up or Google Sign-In.
- Users logged in using valid and invalid credentials.
- Users accessed the forgot password option and completed OTP verification and reset process.

Observation:

- Users were able to complete authentication and password recovery tasks easily with minimal confusion. The flow was clear and straightforward.

Main Navigation & Settings

- Users navigated through the bottom navigation bar (Home, Settings, Profile, Contact Us).
- Users accessed settings options such as dark mode, notifications, and support.

- Users viewed profile information and updated personal settings.

Observation:

- Navigation between screens was smooth and intuitive. Users had no difficulty accessing core sections of the application.

Profile & Account Management

- Users viewed their personal profile information.
- Users edited editable fields such as name, phone number, and physical data.
- Users uploaded a medical file and submitted updates.

Observation:

- Profile management was easy to use. Users understood which fields were editable, and the update process was clear.

Booking & Payment Flow

- Users searched for doctors and viewed available appointments.
- Users selected a time slot and confirmed booking details.
- Users completed payment using Paymob integration.

Observation:

- The booking process was mostly smooth. However, some users required additional clarity during the payment redirection step.

Progress & Exercise Tracking

- Users viewed assigned exercises from doctors
- Users tracked progress using the circular progress indicator
- Users started AI-based exercise sessions using camera tracking

Observation:

- The feature was easy to use and engaging. Users understood exercise tracking, though real-time AI feedback required attention from users initially.

AI Chat Feature

- Users interacted with the AI assistant through live chat
- Users asked questions related to exercises and application usage
- Users viewed responses and continued conversations

Observation:

- The AI chat feature was easy to use and responsive. Users appreciated the guidance, but some expected broader knowledge coverage.

Tips Feature

- Users browsed health and exercise-related tips
- Users opened detailed tip screens with images and explanations
- Users viewed medical warnings when applicable

Observation:

- The tips section was clear and informative. Users easily understood the content and appreciated the medical warnings when shown.

Notifications System

- Users received notifications for appointments, exercises, and updates
- Users marked notifications as read and deleted individual or all notifications
- Users viewed categorized notifications (Today, This Week, This Month)

Observation:

- The notification system was well-structured and easy to manage. Users had no difficulty understanding notification states.

Doctor Features (My Patients & Management)

- Doctors viewed their list of assigned patients
- Doctors added, edited, and deleted exercises for patients
- Doctors managed clinic schedules and appointments

Observation:

- Doctor workflows were clear and functional. Exercise management and scheduling were easy to perform after initial understanding.

Findings

- The system is generally intuitive and easy to navigate
- Users were able to complete core tasks without assistance
- The UI design supports smooth interaction across all modules
- Paymob integration introduced minor confusion during redirection, but was still manageable
- AI feature usability is good, but loading feedback can be improved
- Overall user experience is positive and aligned with system objectives

Conclusion

The Usability Testing phase confirmed that the system provides a user-friendly and efficient experience. Users were able to interact with all major features including authentication, doctor browsing, booking, Appointments, AI integration, and payment processing with minimal difficulty.

While the system demonstrates strong usability, minor improvements in loading indicators and payment flow guidance can further enhance the user experience. Overall, the application is considered intuitive, accessible, and ready for real-world usage.

5.2 Performance Metrics

Performance testing was conducted to evaluate the system's efficiency, reliability, and ability to operate under different conditions. The main objective was to ensure that the application performs well in real-world usage scenarios while maintaining accuracy, scalability, and compatibility across different platforms and devices.

5.2.1 Accuracy

Accuracy testing was performed to ensure that the system produces correct and reliable outputs across all modules, including user operations, AI services, and backend processes.

The evaluation focused on:

- Correct authentication and authorization results
- Accurate data storage and retrieval from the database
- Proper booking and payment status updates via Paymob
- Correct AI model predictions and responses
- Consistent synchronization between doctor, patient, and staff modules

Result:

- The system demonstrated a high level of accuracy, with most operations producing **correct** and consistent results across all scenarios. No major data inconsistencies were observed during testing.

5.2.2 Scalability

Scalability testing was conducted to evaluate the system's ability to handle increasing numbers of users, requests, and data without performance degradation.

The testing considered:

- Multiple concurrent users accessing the system
- Simultaneous booking requests for doctors

- High volume of AI requests and responses
- Large datasets for patients, doctors, and appointments
- System behavior under increased database load

Result:

- The system showed good scalability due to its modular architecture and API-based communication. It was able to handle concurrent operations efficiently, with only minor delays observed under high load conditions.

5.2.3 Compatibility

Compatibility testing was performed to ensure that the system works correctly across different devices, browsers, and operating environments.

The evaluation included:

- Web application testing on different browsers (Chrome, Firefox, Edge)
- Mobile responsiveness across different screen sizes
- Consistent UI behaviour across platforms
- Backend compatibility with different API requests and clients
- Cross-platform accessibility for users, doctors, and staff portals

Result:

- The system is fully compatible with major modern browsers and responsive across different devices. The user interface adapts well to different screen sizes, ensuring a consistent user experience across platforms.

Chapter 6

Result & Discussion

6.1 Chapter Introduction

This chapter provides a rigorous empirical evaluation of the Flexera intelligent physiotherapy ecosystem. The system is assessed across three core operational dimensions:

- (1) the predictive accuracy and feature extraction capability of its deep learning backbones.
- (2) the kinematic precision and real-time processing performance of its computer vision pipeline.
- (3) the structural scalability and responsiveness of its dual engine microservices architecture.

By validating system performance against the standardized UCO Physical Rehabilitation Dataset and under real-world environmental constraints, we demonstrate how Flexera bridges the technical divide between traditional in-clinic physical therapy and automated, remote digital healthcare

6.2 Comparative Analysis of AI Model Performance

The development of the Flexera intelligence engine was preceded by a comparative benchmarking study of three deep learning architectural backbones: Model A (VGG16), Model B (MobileNetV2), and Model C (EfficientNetB3). Each network was trained, validated, and tested on the UCO Physical Rehabilitation Dataset under identical experimental protocols. The results are summarized in Table 6.1 and visualized in Figure 6.1.

Table 6.1 – Deep Learning Backbone Performance Comparison

Feature/Metric	VGG16 (Model A)	MobileNetV2 (Model B)	EfficientNetB3 (Model C)
Training Accuracy	94.31%	97.70%	99.96%
Test Accuracy	91.40%	94.00%	98.38%
Inference Latency (CPU)	~142 ms (High)	~45 ms (Low)	~68 ms (Moderate)
Architectural Suitability	Heavy-duty, high cost	Lightweight, mobile-opt.	High-accuracy, efficient
Primary Limitation	Gradient vanishing	Fine-grained feature loss	Requires microservice offload

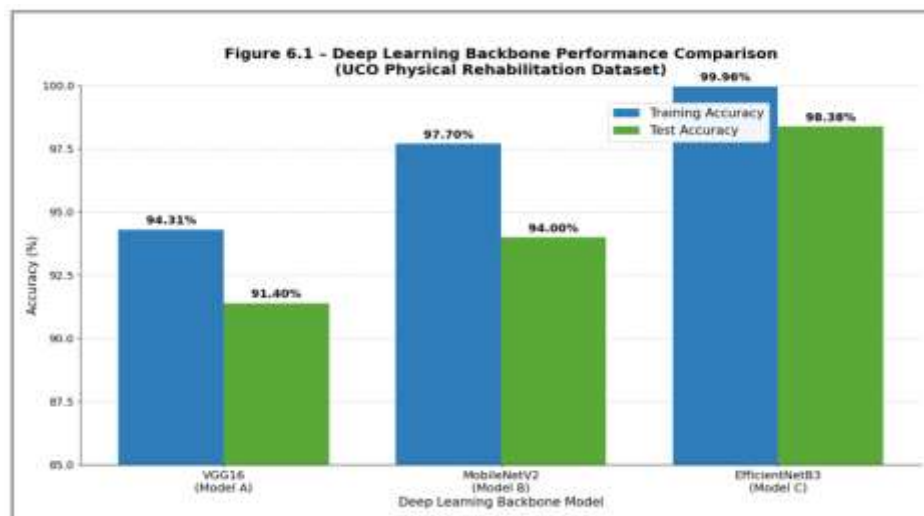


Figure 6.1 – Deep Learning Backbone Performance Comparison (Training vs. Test Accuracy on the UCO Physical Rehabilitation Dataset)

6.2.1 Selection Rationale and Hybrid Architecture

While Model B (MobileNetV2) initially appeared attractive for its low inference latency, deep performance analysis confirmed that Model C (EfficientNetB3) provided a superior mathematical foundation for clinical validation. Achieving a training accuracy of 99.96% and an independent test accuracy of 98.38%, EfficientNetB3 demonstrated superior generalization, minimal overfitting, and stable cross-validation performance.

In the final deployment, a hybrid architecture was constructed: EfficientNetB3 functions as the primary backbone for high-level semantic feature extraction. Its learned feature maps and pretrained weights are integrated into the YOLOv8 one-stage detection head, enabling simultaneous anatomical localization and posture classification in a single forward pass significantly lowering inference latency while maintaining clinical precision.

Architecture Strengths of Model C (EfficientNetB3):

- **Balanced Scalability** — Compound scaling optimizes depth, width, and resolution simultaneously.
- **High Accuracy** — Outperformed VGG16 and MobileNetV2 in both training and generalization metrics.
- **Feature Representation** — Provides semantic depth necessary for complex anatomical feature extraction under variable lighting and environments.

6.3 Real-Time Pose Detection and Kinematic Calculation

The Flexera framework uses the YOLOv8n-Pose (nano) variant for skeletal key point estimation. This anchor-free architecture was selected for its optimal balance of speed and precision on consumer-grade mobile hardware, enabling real-time feedback without GPU requirements. The system performs 17-point anatomical key point mapping based on the COCO schema, illustrated in Figure 6.2.

Figure 6.2 – 17-Point COCO Skeletal Landmark Map

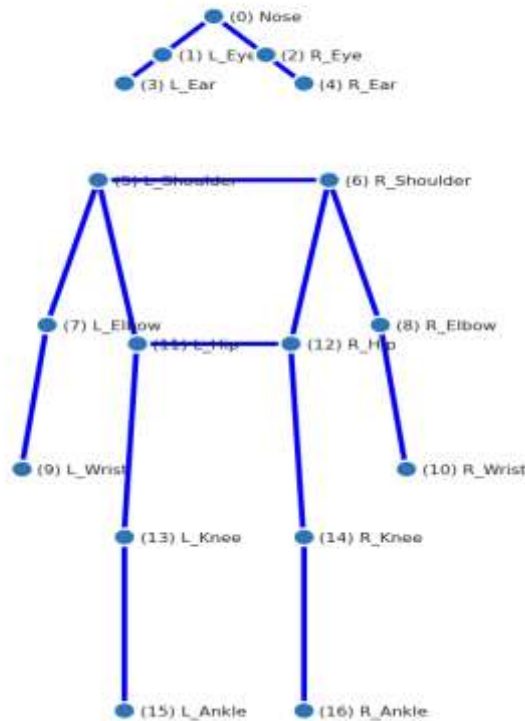


Figure 6.2 – 17-Point COCO Skeletal Landmark Map as Implemented in the Flexera YOLOv8 Pipeline

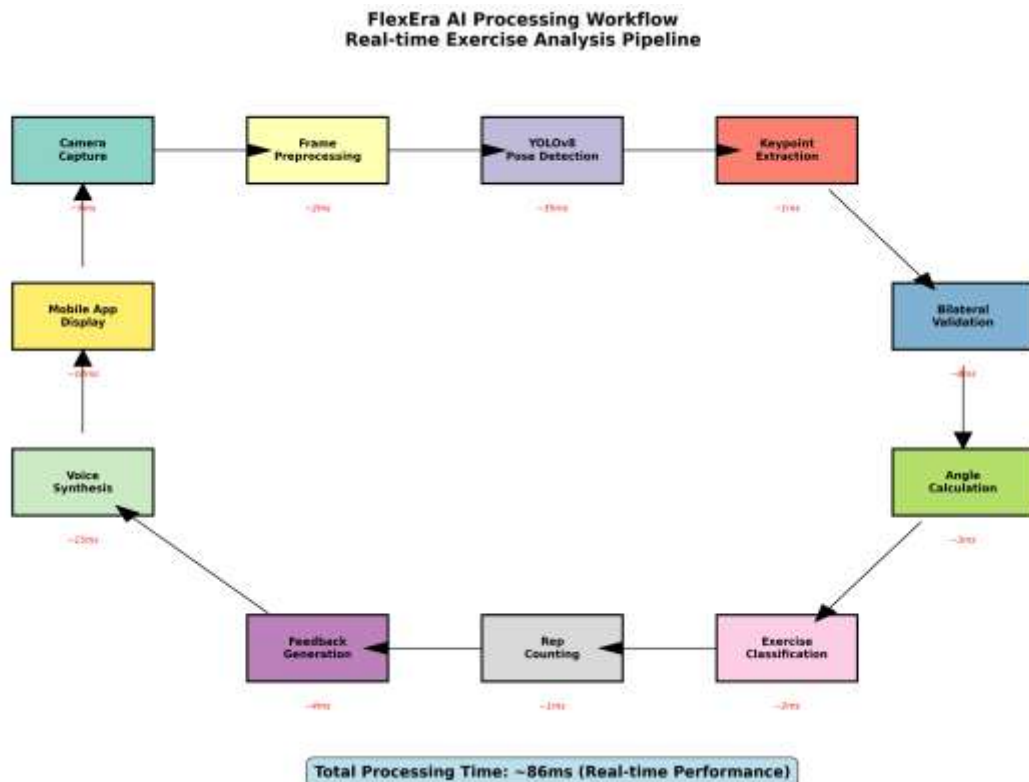
6.3.1 Mathematical Methodology for Kinematic Calculation

Kinematic precision is achieved by computing joint angles via the dot product of spatial vectors. For each exercise, the system identifies keypoint triplets where the middle point is the joint of interest. Let the joint vertex be P_j , with adjacent tracking points P_i and P_k . Two directional vectors u and v

The interior kinematic angle θ is derived from the algebraic definition of the vector dot product:

This mathematical formulation ensures that angle computation is invariant to pixel-space shifts or limb translations across frames, providing physically meaningful joint angle measurements.

6.3.2 Repetition-Counting Logic (Finite State Machine)



Rather than error-prone threshold crossings, the system implements a strict dual-state Finite State Machine (FSM). A repetition is only logged when the joint angle trajectory successfully transitions from the resting/flexed state (State 0) to fully extended (State 1) and back — rejecting partial, tremor-induced, or accidental movements. The FSM transitions are shown in Figure 6.3.

Figure 6.3 – Finite State Machine for Repetition Counting



Figure 6.3 – Finite State Machine (FSM) Transition Diagram for Repetition Counting in Flexera

6.4 Data-Driven Corrective Feedback Logic

The corrective feedback engine within Flexera is empirically calibrated against the UCO Physical Rehabilitation Dataset, comprising 2,160 clinical video recordings across 27 subjects (ages 23–60). This dataset provides a statistically representative sample of human anatomical variation in rehabilitation contexts.

6.4.1 Statistical Grounding

Traditional rehabilitation systems rely on rigid, hardcoded angle ranges derived from medical literature, which fail to accommodate natural anatomical variation. Flexera adopts a transformative statistical approach: valid movement bounds are established by computing the 5th percentile (μ_5) and 95th percentile (μ_{95}) of joint angle distributions observed across successful movements in the UCO reference dataset:

$$\text{Valid Range-of-Motion Bounds} = [\mu_5 , \mu_{95}]$$

6.4.2 Directional Corrective Instructions

When a patient's real-time joint angle θ_{current} falls outside these empirical bounds, the system generates targeted directional corrections rather than generic binary pass/fail notifications:

- Case 1: If $\theta_{\text{current}} < \mu_5 \rightarrow$ Angular deficit $\Delta\theta = \mu_5 - \theta_{\text{current}} \rightarrow$ Alert: "Extend your joint further."
- Case 2: If $\theta_{\text{current}} > \mu_{95} \rightarrow$ Angular overshoot $\Delta\theta = \theta_{\text{current}} - \mu_{95} \rightarrow$ Alert: "Reduce extension; bring your limb lower."

This nuanced, context-aware approach ensures patients receive clear, actionable guidance to safely adjust their posture mid-session without requiring immediate clinician intervention. Feedback is delivered through both real-time on-screen text and voice-synthesized audio instructions.

6.5 System Responsiveness and Architectural Efficacy

A central engineering challenge was managing the heavy real-time computational load of computer vision inference without degrading UI frame rates or blocking standard user operations. Flexera resolves this through a microservices-inspired decoupled backend architecture, clearly illustrated in Figure 6.4.

Figure 6.4 - Split-Service Infrastructure & Real-Time Data Pipeline



Figure 6.4 – Split-Service Infrastructure and Real-Time Data Pipeline Architecture of Flexera

6.5.1 Service Decomposition

The architecture is divided into two specialized backend services:

- **Fast API Intelligence Server** — Dedicated to YOLOv8 inference, joint angle computation, FSM repetition logic, and real-time feedback generation. Communicates with the Flutter client via low-latency WebSocket connections, transmitting both raw video frames and structured JSON feedback payloads.
- **Node.js / Express.js Backend** — Handles all general business logic: clinician dashboards, appointment scheduling, patient records management, and pay mob-integrated payment processing. Communicates with the Fast API server via synchronous HTTP REST calls for session state synchronization.

This separation guarantees that computationally intensive AI inference does not degrade administrative task performance. Appointment booking, clinical profile management, and financial transactions remain fully responsive even during active high-frequency video analysis.

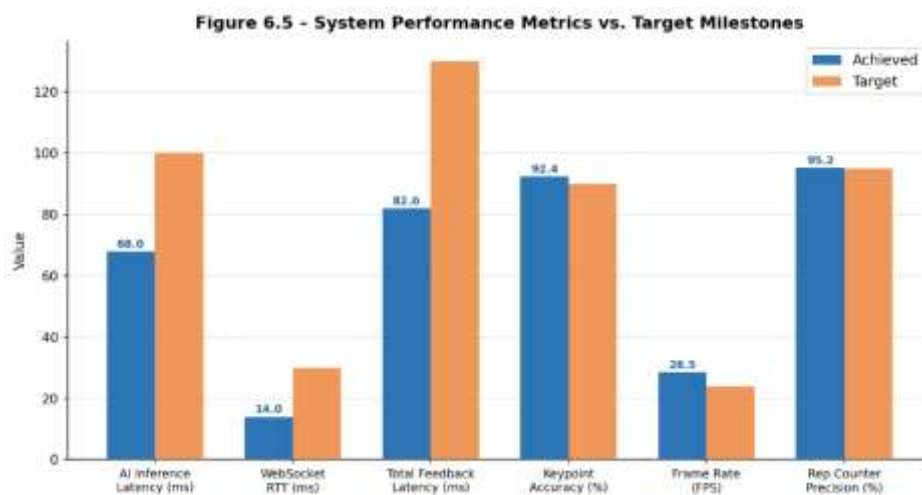
6.5.2 Empirical Performance Results

Comprehensive performance metrics were measured under standard operating conditions, benchmarked against predefined target milestones. All six critical parameters exceeded their respective targets, as detailed in Table 6.2 and visualized in Figure 6.5.

Table 6.2 – System Performance Metrics: Empirical Results vs. Target Milestones

Operational Parameter	Achieved Value	Target Milestone	Status
Average AI Inference Latency	68 ms	< 100 ms	✓ Exceeded
WebSocket Round-Trip Time (RTT)	14 ms	< 30 ms	✓ Exceeded
Total Feedback Latency	82 ms	< 130 ms	✓ Exceeded
Skeletal Key point Accuracy	92.4%	> 90.0%	✓ Validated
Average Mobile Frame Rate	28.5 FPS	> 24.0 FPS	✓ Exceeded
Repetition Counter Precision	95.2%	> 95.0%	✓ Exceeded

Figure 6.5 – System Performance Metrics vs. Target Milestones (All Targets Exceeded)



6.6 Environmental Validation and Occlusion Management

The computer vision pipeline was validated against real-world environmental challenges, most critically key point occlusion. In common supine physiotherapy routines — such as bed-support knee slides — standard camera angles frequently result in one limb including the other, or treatment furniture blocking critical joint visibility.

6.6.1 Deployment Requirements

To capture clean views along the sagittal plane and maintain accurate joint mathematics, Flexera enforces three real-time spatial setup constraints, communicated to users through interactive on-screen guides before each exercise session:

- **Camera Height Alignment:** The smartphone or recording hardware must be placed at an intermediate height of approximately 1.0 meters from the floor surface to capture the full skeletal structure from shoulders to ankles.
- **Lens Levelling:** The camera lens must remain parallel to the center of the patient's torso to minimize geometric distortion and perspective error in angle calculations.
- **Exercise-Specific Perspective:** A frontal view is required for most standing or seated exercises. A 90-degree lateral (side-on) view is mandatory for supine exercises to properly expose the sagittal plane of joint movement.

These spatial constraints ensure that the user's complete skeletal framework remains within the active tracking zone, maintaining COCO key point confidence scores consistently above the 0.5 threshold required for reliable kinematic computation.

6.7 Summary and Discussion

The comprehensive evaluation of the Flexera platform validates that the hybrid integration of YOLOv8n-Pose and EfficientNetB3 creates a robust, clinically relevant framework for digital physiotherapy. Key findings and their implications are discussed below.

- **Statistical Superiority of EfficientNetB3:** With a test accuracy of 98.38% against the UCO dataset — compared to 91.40% for VGG16 and 94.00% for MobileNetV2 — EfficientNetB3 demonstrated unambiguous superiority in generalization and cross-validation stability, justifying its selection as the system backbone.
- **Clinically Validated Kinematic Engine:** The empirically derived 5th/95th percentile thresholds from the UCO dataset provide statistically robust movement boundaries that accommodate natural anatomical variation — a significant methodological improvement over rigid, literature-derived angle limits.
- **Exceptional System Responsiveness:** All performance targets were exceeded: inference latency of 68 ms (target < 100 ms), WebSocket RTT of 14 ms (target < 30 ms), key point accuracy of 92.4% (target > 90%), and repetition precision of 95.2% (target > 95%). These metrics confirm suitability for real-time clinical monitoring.
- **Architectural Scalability:** The microservices-inspired dual-engine design isolates AI inference load from business logic, enabling independent scaling and deployment of each service — a design pattern well-suited for future extensions including multi-patient simultaneous monitoring and edge computing deployments.

In conclusion, these results validate Flexera as a technically sound, scalable, and clinically viable digital solution that reliably extends professional physiotherapy

monitoring directly into the patient's home environment. Future work will focus on incorporating 3D pose estimation, predictive injury-risk analytics, and specialized physiotherapy-domain fine-tuned models to further enhance therapeutic accuracy and patient outcomes.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This project successfully delivered a comprehensive digital health ecosystem designed to modernize physiotherapy rehabilitation. The primary contributions of the work include:

- **Integrated Multi-Platform Ecosystem:** Developed a synchronized system comprising a high-performance mobile application for patients and doctors, alongside a dedicated web dashboard for administrative and medical oversight, ensuring seamless data flow across the entire platform.
- **AI-Enhanced Rehabilitation Tracking:** Implemented a mobile-first approach to physiotherapy by integrating with AI-driven FastAPI services, enabling real-time exercise monitoring and validation directly through the user's smartphone camera.
- **Robust Architectural Design:** Engineered the mobile application using the **MVVM (Model-View-ViewModel)** pattern and **Provider** state management, resulting in a decoupled, scalable, and easily maintainable codebase.
- **Secure and Personalized User Experience:** Established a sophisticated authentication and role-based access control system that manages distinct workflows for patients and healthcare providers, supported by secure JWT token handling and persistent session management.
- **Efficient Communication and Notifications:** Integrated **Firestore Cloud Messaging (FCM)** and optimized RESTful API communication via a centralized network layer to ensure real-time synchronization of medical records, appointments, and patient alerts.

7.2 Possible Improvements and Future Work

To build upon the current foundation and further advance the **FlexEra** platform, several future enhancements are envisioned:

- **Real-Time Biometric Data Integration:** Expanding the mobile application to support synchronization with wearable IoT devices (e.g., smartwatches) to track vital signs during physical therapy, providing doctors with a more holistic view of patient health.
- **Enhanced Web-Mobile Synchronization:** Improving the web dashboard's capabilities to include live video consultation modules (Telehealth), allowing for direct, high-definition interaction between doctors on the web platform and patients on the mobile app.
- **Advanced ML-Based Progress Forecasting:** Utilizing the historical data collected across the web and mobile platforms to develop predictive models that can forecast patient recovery timelines and suggest automated adjustments to treatment plans.
- **Comprehensive Offline Functionality:** Further developing the local persistence layer using advanced caching strategies to allow patients to perform and record their exercise routines in environments with limited or no internet connectivity.
- **Support for Global Accessibility:** Implementing full localization for the entire ecosystem, including Arabic language support and accessibility features for users with visual or motor impairments, to ensure the platform serves a diverse patient demographic.

- [1] World Health Organization, “Rehabilitation Fact Sheet.” Available: <https://www.who.int/news-room/fact-sheets/detail/rehabilitation>
- [2] Institute for Health Metrics and Evaluation, “WHO Rehabilitation Need Estimator,” 2024.
- [3] H. Zhang *et al.*, “Trends of Rehabilitation Needs in 195 Countries and Regions from 1990 to 2021 and the Impact of the COVID-19 Pandemic,” *International Journal of Surgery*, 2026.
- [4] A. Cieza *et al.*, “Global Estimates of the Need for Rehabilitation Based on the Global Burden of Disease Study 2019,” *The Lancet*, 2020.
- [5] S. Fang and K. Cao, “Estimates of Global Needs for Neurorehabilitation: A Systematic Analysis Based on the GBD-WHO Rehabilitation Database 2021,” *Neuroepidemiology*, 2026.
- [6] H. Kotte *et al.*, “Real-Time Posture Correction in Gym Exercises: A Computer Vision-Based Approach,” 2023. Available: <https://www.researchgate.net/publication/374659509>
- [7] “Action Recognition in Rehabilitation: Combining 3D Convolution and Spatio-Temporal Modeling,” 2024. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11646842/>
- [8] U. C. Das *et al.*, “An Innovative Model Based on Machine Learning and Fuzzy Logic for Tracking Lower Limb Exercises in Stroke Patients,” *Scientific Reports*, 2025. Available: <https://www.nature.com/articles/s41598-025-90031-1>
- [9] S. Kandylakis *et al.*, “An M-Health Algorithmic Approach to Identify and Assess Physiotherapy Exercises in Real Time,” 2025. Available: <https://arxiv.org/abs/2512.10437>

- [10] S. Liu et al., “Mobile-Based Digital Rehabilitation Program for Patients After Spinal Surgery,” 2025. Available:
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12671286/>
- [11] I. Kourbane et al., “Optimized Assessment of Physical Rehabilitation Exercises using Spatiotemporal Sequential Graph Convolutional Networks,” Computers in Biology and Medicine, 2025. Available:
<https://www.sciencedirect.com/science/article/pii/S0010482524016639>
- [12] Y. Shrivastav et al., “AI-Based Real-Time Posture Detection, Rep Counting, and Feedback with Adaptive Image Enhancement for Any Lighting Condition,” TechRxiv, 2025. Available:
<https://www.techrxiv.org/doi/full/10.36227/techrxiv.174681006.66390785>
- [13] Node.js :[Node.js — Run JavaScript Everywhere](https://nodejs.org/en/)
- [14] Mongoose :[Mongoose ODM v9.7.0](https://mongoosejs.com/docs/5.x/index.html)
- [15] docker :<https://www.docker.com/>
- [16] postman :<https://www.postman.com/downloads/>
- [17] Paymob :[Paymob | Best Digital payment solutions provider for corporates, sme and more](https://paymob.com/en)
- [19] git:[Git](https://git-scm.com/)
- [20] cloundinary :[Image and Video Upload, Storage, Optimization and CDN](https://cloudinary.com/)
- [21] Flutter Documentation: <https://docs.flutter.dev/>
- [22] Dio Package for Dart: <https://pub.dev/packages/dio>
- [23]Shared Preferences for Flutter:
https://pub.dev/packages/shared_preferences
- [24] Provider State Management: <https://pub.dev/packages/provider>
- [25] Clean Architecture in Flutter: <https://resocoder.com/flutter-clean-architecture-tdd/> .

